

Can We Break Fine-Grained and NP-Hardness Barriers if We’ve Seen the Graph Before? The Isomorphic-Priors Model

Dani Dorfman 

Max Planck Institute for Informatics
Saarland Informatics Campus

Simon Döring 

Max Planck Institute for Informatics
Saarland Informatics Campus

Martin G. Herold* 

Max Planck Institute for Informatics
Saarland Informatics Campus

Daniel Neuen 

TU Dresden

Joachim Spoerhase 

University of Liverpool

Danupon Nanongkai 

Max Planck Institute for Informatics
Saarland Informatics Campus

Zihang Wu 

Max Planck Institute for Informatics
Saarland Informatics Campus

Abstract

If we run a heavy-duty computation on prior data, can we avoid repeated computation for similar future inputs? Inspired by this question, we introduce a new computational model for graph problems called *algorithms with isomorphic priors*. Solving a graph problem Π in this model involves two phases: (i) The *preprocessing* phase quickly analyzes *prior graphs* $G_1, \dots, G_k$ along with the (previously computed) exact optimal values $OPT(G_i)$. (ii) Subsequently, given a new graph H , a fast *query* phase must either (a) output the exact solution $OPT(H)$, or (b) correctly report that H is not isomorphic to any G_i .¹ *Can we avoid computing $OPT(H)$ from scratch when H is isomorphic to some G_i ?* We show that this is the case for a number of problems; for many others, we establish conditional lower bounds.

(1) Some *NP-hard* problems, including Constrained Shortest Path and ℓ_p -Shortest Path and Constrained Spanning Tree, admit polynomial preprocessing and query times in our model. In contrast, almost all of Karp’s 21 NP-complete problems and $(2 - \varepsilon)$ -approximate k -Center, for every fixed $\varepsilon > 0$, admit no such algorithms unless Graph Isomorphism (GI) is in P, even with $O(1)$ priors. Unless $GI \in \text{coMA}$, these lower bounds persist with *unbounded preprocessing* provided the data structure size and query time remain polynomial.

(2) In contrast to conditional $n^{3-o(1)}$ fine-grained lower bounds, our framework achieves an $O(n^\omega)$ query time for Negative Triangle and a near-linear query time for Replacement Path. It also achieves near-linear query time for Maximum Flow.

(3) While it remains a major open problem whether *infinite-duration games* (such as Parity Game, Mean Payoff Game, Energy Game, and Stochastic Game) admit polynomial-time algorithms, they can be easily solved in near-linear time within our model.

Our proofs rely on a simple combination of existing tools and are accessible to readers without specialized background in these methods. The model leads to open questions about, e.g., approximation guarantees and fine-grained speedups, including whether isomorphic priors enable faster algorithms for k -Clique. We formulate a k -Clique hypothesis which implies $GI \notin \text{P}$.

*Martin G. Herold was initially funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – Project number 399223600.

¹Our algorithms sometimes return $OPT(H)$ even though no graph G_i is isomorphic to H . We focus on problems Π for which isomorphic instances yield identical optimal values, a property that holds for most natural graph problems.

Contents

1	Introduction	1
1.1	Our Results	2
1.2	Organization	5
2	Overview	5
2.1	Algorithms via Randomized Polynomial Evaluation (Details in Section 6)	5
2.2	Algorithms via Weisfeiler-Leman (WL) Coloring (Details in Sections 7 and 8)	7
2.3	Lower Bounds (Details in Section 9)	7
3	Open Problems	10
4	Related Works	13
5	Our Model of Computation	16
5.1	Preliminary: Attributed Graphs and Isomorphism	16
5.2	The Isomorphic-Priors Model	16
5.3	Our Focused Approach: Invariant Representations	17
6	Algorithms via Randomized Polynomial Evaluation	18
6.1	Negative Triangle	19
6.2	Constrained Spanning Tree	21
7	Algorithms via 1-WL (Elementary Arguments)	23
7.1	Constrained Shortest Path and Energy Game	24
7.2	Preliminaries: The Weisfeiler–Leman Algorithm	25
7.3	1-WL Equivalence Implies Equal Optimality	27
7.4	Proof of Theorems 7.2 and 7.4	29
8	More on Weisfeiler-Leman (Advanced Arguments)	31
8.1	The Weisfeiler-Leman Algorithm	31
8.2	ℓ -WL Invariant Functions are in the Isomorphic-Priors Model	32
8.3	CSP is 1-WL Invariant	33
8.4	CST is 2-WL Invariant	35
8.5	Examples when Weisfeiler-Leman Fails	37
9	Conditional Lower Bounds	39
9.1	Hardness Results for GI	41
9.2	Iso-Problems and GI-Completeness	42
9.3	Isomorphism-Preserving Reductions	43
9.4	The Hardness of 3-SAT	47
9.4.1	Lovász’s Reduction for Colorability	47
9.4.2	Reduction from 3-Coloring to 3-SAT	51
9.5	Hardness in the Isomorphism-Priors Model	53
9.6	Proof of Hardness for Gap-Max- k -Coverage	54
10	Conclusion	58
10.1	Proof of the Main Theorems	58
10.2	k -Clique Hypothesis and Consequence	58
10.3	Acknowledgments and AI Disclosure	59

Bibliography	59
A Further Problems that are Invariant under 1-WL	67
A.1 Color-Consistent Strategies for Energy Game	67
A.2 More Games on Graphs	68
A.3 The Replacement Path Problem	71
A.4 Maximum Flow	72
B More Conditional Lower Bounds	75
B.1 Additional Problem Definitions	75
B.2 Additional Isomorphism-Preserving Reductions	78

1 Introduction

If we invest in heavy-duty computation on prior data, can we avoid repeating this effort for similar future inputs? In an era defined by data abundance, this has become a central algorithmic challenge. There is a pressing need for data-driven algorithms that exploit knowledge of existing instances, fundamentally departing from the traditional worst-case design paradigm where inputs are analyzed without any prior context.

In this paper, we explore this question theoretically by proposing a new computational model for graphs and related objects. While this motivation is shared by existing paradigms, such as average-case analysis over input distributions or learning-augmented algorithms with oracles, our model uniquely grants direct access to raw prior data without relying on learned oracles or distributions.

Model: Algorithms with Isomorphic Priors. Solving a graph problem Π in our model proceeds in two phases. In the *preprocessing* phase, the algorithm is given a set of *prior graphs* $G_1, \dots, G_k$ along with their exact optimal values $OPT(G_i)$, which may have been computationally expensive to obtain. Subsequently, in the *query* phase, the algorithm receives a new *query graph* H . The algorithm must either output $OPT(H)$ if H is isomorphic to some G_i , or correctly report that H is not isomorphic to any prior graph.² Throughout, we let n and m denote the number of vertices and edges in H , respectively. We assume without loss of generality that all prior graphs $G_1, \dots, G_k$ also contain exactly n vertices and m edges (because graphs of different sizes cannot be isomorphic).

For concreteness, let us consider two examples of problems Π :

1. **Negative Triangle:** Given a weighted undirected graph G , we want to know if G contains a negative triangle, i.e., three vertices a, b, c such that the sum of the weights of edges (a, b) , (b, c) , and (c, a) is less than zero. This problem can be easily solved in $O(n^3)$ time. Breaking this cubic barrier would be a major breakthrough in fine-grained complexity [VW10, Vas18].

In our model, the algorithm is given weighted undirected graphs $G_1, \dots, G_k$ alongside binary indicators $OPT(G_i)$ denoting whether each G_i contains a negative triangle. After preprocessing this information, the algorithm is given another graph H and must either return whether H contains a negative triangle, or report that H is not isomorphic to any G_i .

2. **Constrained Shortest Path (CSP):** The input consists of a directed graph G where each edge e has a non-negative length $\ell(e)$ and a non-negative cost $c(e)$, two designated vertices s and t , and a budget B . We want to find a path from s to t whose total cost is at most B and whose total length is minimized. Let $OPT(G)$ denote the length of the optimal path.

CSP is NP-hard [GJ79, Han80] and has been extensively studied under various names, such as Restricted Shortest Path or Resource Constrained Shortest Path, from both the theoretical [ABK25, Ber12, PY00, LR01, MZ00, Has92] and applied [YRZ⁺24, WW23, ATHK21, WXYL16, Sto12, MZ01, Xue00, LT92, BC89, AAN83, HZ80] perspectives.

In our model, the algorithm is given a budget³ B and preprocesses prior input graphs $G_1, \dots, G_k$ with their respective length and cost functions, along with their precomputed solutions $OPT(G_1), \dots, OPT(G_k)$. After preprocessing this information, the algorithm receives a query

²Recall that graphs G and H are isomorphic (denoted $G \cong H$) if one graph can be obtained simply by renaming the vertices of the other; i.e., there exists a bijection $\pi: V(G) \rightarrow V(H)$ such that $(u, v) \in E(G)$ if and only if $(\pi(u), \pi(v)) \in E(H)$. Furthermore, if the edges carry additional attributes (such as weights, lengths, or costs), the bijection must preserve these values; i.e., every attribute α satisfies $\alpha_G(u, v) = \alpha_H(\pi(u), \pi(v))$ for all $(u, v) \in E(G)$.

³We assume for simplicity that the budget is the same for all input instances. Algorithms for this case can be trivially extended to handle varying budgets since, naturally, two instances are isomorphic if and only if their input graphs are isomorphic and their budgets are identical.

graph H with its own length and cost functions, and must either return the constrained shortest path distance $OPT(H)$ or report that H is not isomorphic to any G_i .⁴

Note that an algorithm may still correctly return the optimal value $OPT(H)$ even if no G_i is isomorphic to H ; indeed, the algorithms in this paper exhibit this behavior.

Can we Avoid Computing $OPT(H)$ from Scratch when H is Isomorphic to some G_i ?

Ideally, if H is isomorphic to one of the prior graphs $G_1, \dots, G_k$, the algorithm should compute $OPT(H)$ significantly faster than traditional worst-case bounds guarantee. Conversely, if H is not isomorphic to any prior graph, the algorithm should quickly report this so that a heavy-duty fallback computation can be invoked from scratch. For instance, if a weighted query graph H is isomorphic to a prior graph G_i (i.e., identical up to a renaming of vertices), can we determine whether H contains a negative triangle in truly subcubic time $O(n^{3-\epsilon})$? Similarly, for CSP, can we achieve polynomial preprocessing and query times in our model?

Trivially, every problem in our model can be solved via Graph Isomorphism (GI); however, the best known algorithm for GI requires quasipolynomial time [Bab16] and is efficient only on restricted graph classes [HW74, Luk82, Bod90]. This is prohibitively slow for a query phase even for NP-hard problems like CSP, and is inferior to the classical worst-case time for problems in P like Negative Triangle. *Is this new model capable of bypassing worst-case barriers without invoking computationally expensive graph isomorphism algorithms?*

1.1 Our Results

Algorithmic Results. We show that our model has the power to bypass worst-case time complexity barriers for various classes of computational problems.

The first class of problems we consider encompasses **NP-hard graph optimization**. Besides the aforementioned Constrained Shortest Path (CSP) problem, ℓ_p -Shortest Path (ℓ_p -SP) stands as another classic multi-objective challenge [PY00, YY98, BLM⁺08, CMM26, MOT24, LXZ24]. Started by Papadimitriou and Yannakakis [PY00] and the subject of recent progress characterizing its exact approximation landscape [CMM26, MOT24, LXZ24], this problem requires finding an s - t path in a graph with d -dimensional edge weights such that the ℓ_p -norm of the path’s total aggregated weight vector is minimized. Closely related are Constrained Spanning Tree (CST) and ℓ_p -Spanning Tree (ℓ_p -ST) [HCP04, Xue00, RG96, MRS⁺98, Sho83, AAN82, GJ79], where the optimization goal shifts from finding an s - t path to computing a spanning tree.

All these problems are NP-hard. In contrast, in our model, we only require near-linear preprocessing and query times to solve the problems for paths and subcubic time for trees.

Theorem 1.1 (Algorithms for NP-hard Problems). *There exist deterministic isomorphic-priors algorithms that solve Constrained Shortest Path and ℓ_p -Shortest Path using $\tilde{O}(km)$ preprocessing time and $\tilde{O}(m)$ query time.⁵ For Constrained Spanning Tree and ℓ_p -Spanning Tree, there exist randomized⁶ algorithms requiring $\tilde{O}(kn^\omega)$ preprocessing time and $\tilde{O}(n^\omega)$ query time, where $\omega < 2.37134$ is the matrix multiplication exponent.*

⁴Following the definition in Footnote 2, an isomorphism between G_i and H must strictly preserve the source and target nodes, mapping s to s and t to t .

⁵ $\tilde{O}$ hides polylogarithmic factors.

⁶The algorithms are correct with high probability. We say an event occurs *with high probability* (w.h.p.) if it holds with probability at least $1 - 1/N^c$ for any arbitrarily large constant $c \geq 1$, where N denotes the input size. Furthermore, our algorithms are analyzed under the standard *oblivious adversary* model; that is, we assume the sequence of queries is generated independently of the algorithm’s internal random choices (see, e.g., [NY15] and [KL14, Definition 3.4]).

Next, we consider **problems in P**. Beyond Negative Triangle, Replacement Path is another fundamental problem in P that requires $n^{3-o(1)}$ time in the worst case under standard fine-grained hypotheses [VW10, Vas18]. In this problem, given a shortest s - t path P in a graph G , the goal is to compute, for every edge $e \in P$, the length of the shortest s - t path in $G - e$. Our Isomorphic-Priors model allows us to bypass this classical cubic barrier, achieving near-linear query time for Replacement Path and truly subcubic query time for Negative Triangle:

Theorem 1.2 (Algorithms for Problems in P). *There exists a deterministic isomorphic-priors algorithm that solves Replacement Path using $\tilde{O}(km)$ preprocessing time and $\tilde{O}(m)$ query time. For Negative Triangle, there exists a randomized⁶ isomorphic-priors algorithm requiring $\tilde{O}(kn^\omega)$ preprocessing time and $\tilde{O}(n^\omega)$ query time.*

Interestingly, while Negative Triangle and Replacement Path are subcubic equivalent in the classical setting [VW10, AGV15], their current complexities diverge in the Isomorphic-Priors model.

Additionally, we can solve Maximum Flow with near-linear preprocessing and query times, which is slightly faster than the current worst-case complexity [CKL⁺22, vdBCP⁺23, vdBLL⁺21, vdBLN⁺20].

Our third class of problems concerns the **frontier between P and NP**, namely **infinite-duration games** played on graphs, including Parity Game, Mean Payoff Game, Energy Game, and Simple Stochastic Game. In these games, two players take turns moving a token along the edges of a directed graph to form an infinite path. For example, in an Energy Game (played on a bipartite graph with integer edge weights), the token starts at vertex s with initial energy e_0 . Traversing an edge adds its weight to the running total. Player A wins if they can keep the energy non-negative indefinitely; otherwise, Player B wins. We defer formal definitions to Section 7.

Originating from automata theory, reactive systems synthesis, and model checking [EJ91, Tho97], these games lie in $\text{NP} \cap \text{coNP}$ (and often $\text{UP} \cap \text{coUP}$ [Jur98, Con92]), making NP-hardness highly unlikely. While a major breakthrough (STOC 2017 best paper) showed Parity Game are solvable in quasipolynomial time [CJK⁺17], harder variants (Mean Payoff Game, Energy Game, and Simple Stochastic Game) still rely on randomized subexponential simplex-like methods [Hal07, BV07]. Determining whether any of these games admit polynomial-time algorithms remains a prominent open question.

In our setting, however, all these problems can be easily solved in near-linear time.

Theorem 1.3 (Algorithms for Infinite-Duration Games). *There exist deterministic Isomorphic-Priors algorithms that solve Parity Game, Mean Payoff Game, Energy Game, and Simple Stochastic Game using $\tilde{O}(km)$ preprocessing time and $\tilde{O}(m)$ query time.*

More broadly, a similar result holds for every infinite-duration game with a well-defined value (value-determined game; see Definition A.1)

In all the results above, we compute only the optimal *values*. If the corresponding optimal *solutions* are provided with the priors, then our algorithms can also compute an optimal solution for the query graph.

Conditional Lower Bounds. We observe that many known reductions can be naturally adapted to our framework to establish conditional lower bounds, which hold even when the database contains only a single prior graph ($k = 1$). Specifically, we show that for many NP-hard problems, achieving both polynomial preprocessing and query times would imply the major algorithmic breakthrough of $\text{GI} \in \text{P}$. Moreover, barring a breakthrough proving $\text{GI} \in \text{coMA}$, these lower bounds hold even in a stronger model allowing unbounded preprocessing time.

Theorem 1.4 (Informal). *Unless $\text{GI} \in \text{P}$, none of the following problems can be solved in the Isomorphic-Priors model using simultaneously polynomial preprocessing time and polynomial query time, even when restricted to a single prior graph ($k = 1$): 3-SAT⁷, Max 2-SAT, Clique, Coloring, 3-Coloring, Clique Cover, Exact Cover, Set Cover, Set Packing, Hitting Set, VC, MaxCut, Steiner Tree, Feedback Arc Set, Feedback Vertex Set, Directed Hamiltonian Cycle, Undirected Hamiltonian Cycle, 3D-Matching, 0/1-Integer Linear Programming, k -Median, Metric TSP, TSP within any approximation factor, and $(2 - \epsilon)$ -approximate k -Center.*

Unless $\text{GI} \in \text{coMA}$, these lower bounds hold even against algorithms with unbounded preprocessing time, provided they are restricted to a polynomial-size data structure for answering queries.

The list above includes almost all of Karp’s 21 NP-complete problems [Kar72], except a few problems on numbers that are trivial in our model such as Knapsack and Partition problems.

Theorem 1.4 already includes hardness of approximation results for the Traveling Salesman Problem (TSP) and k -Center. If the PCP theorem can be transferred to argue that Max 3-SAT cannot be approximated within an arbitrary constant factor in our model, we can additionally argue that neither a $(1 - 1/e + \epsilon)$ -approximation for Max- k -Coverage nor a $(1 + 2/e - \epsilon)$ -approximation for k -Median can be achieved using simultaneously polynomial preprocessing time and polynomial query time (see Section 9). We view the deeper exploration of the PCP theorem, inapproximability bounds, and improved approximation algorithms within our model as an exciting future research direction.

Proving hardness in our model requires *isomorphism-preserving reductions* (Section 9.3): a reduction f must map an instance G to a target instance $f(G)$ while guaranteeing that if $G \cong H$, then $f(G) \cong f(H)$. This is related to the existing notion of *strong isomorphism reductions* [BCF⁺11], which demands the biconditional $G \cong H \iff f(G) \cong f(H)$. Some standard reductions preserve isomorphism, such as the reduction from Graph Isomorphism to finding a maximum clique in a modular product graph [Koz78] (which immediately implies the hardness of Maximum Clique/Maximum Independent Set in our model). However, many classical reductions are not isomorphism-preserving, such as the standard transformation from SAT to 3-SAT or the PCP theorem (e.g., the assignment tester in [Din07]). Determining whether the rich landscape of classical reductions can be systematically adapted into isomorphism-preserving variants remains a formidable research program.

Recent Developments. Since we completed the initial draft of this paper on April 1, 2026, the following related developments have emerged.

First, in the All-Pairs Shortest Paths problem (APSP), the input is an edge-weighted graph, and the task is to compute all pairwise distances. Fox [Fox26] showed that APSP can be solved in the Isomorphic-Priors model by a randomized algorithm with $\tilde{O}(kn^\omega)$ preprocessing time and $\tilde{O}(n^\omega)$ query time.

Neuen independently observed that APSP, Negative Triangle, and Constrained Spanning Tree admit a unified randomized approach with $\tilde{O}(kn^\omega)$ query time and no nontrivial preprocessing beyond storing the prior instances and their answers. Although the query bound retains a linear dependence on k , the approach handles all three problems using the same comparison procedure, based on only $O(\log n)$ rounds of 2-WL which can be performed in time $\tilde{O}(n^\omega)$ using Skresanov’s randomized implementation of 2-WL [Skr21].⁸

⁷For SAT, we say that two CNF formulas are isomorphic if one can be obtained from the other by changing the names of variables and clauses.

⁸More precisely, Skresanov’s randomized implementation of 2-WL [Skr21] allows r rounds of refinement to be computed in $\tilde{O}(rn^\omega)$ time, with high probability. The key observation is that $O(\log n)$ rounds suffice to transfer the required answers between instances that are not distinguished by the refinement. For Negative Triangle and Constrained Minimum Spanning Tree, such instances have the same decision answer and optimal value, respectively.

1.2 Organization

We provide an overview of our arguments in Section 2, which we believe already gives readers the key ideas and enough intuition to reconstruct the full proofs. We discuss open problems, related work and formally define the model and necessary notations in Sections 3 to 5. In Section 6, we present algorithms based on randomized polynomial evaluation for **Negative Triangle**, **Constrained Spanning Tree** and ℓ_p -**Spanning Tree**. Then, we switch to Weisfeiler-Leman techniques: Section 7 presents an elementary argument based on WL coloring for **Constrained Shortest Path** and **Energy Game** and Section 8 present the same results based on connections to graph homomorphism counting, as well as a deterministic algorithm for **Constrained Spanning Tree** and ℓ_p -**Spanning Tree**. Our lower bounds are presented in Section 9. We conclude in Section 10 by presenting the proofs of our main theorems, referring to the relevant results established throughout the paper, and by discussing some hypotheses within our model and their consequences. Sections A and B provide additional algorithmic results based on WL coloring and additional lower bound results.

2 Overview

In this section, we outline the core technical ideas underlying our main results. We emphasize again that our primary contribution lies not in the invention of these individual techniques, but in their synthesis across different fields to demonstrate how worst-case computational barriers can be bypassed within the Isomorphic-Priors framework, alongside the establishment of tight conditional lower bounds.

We present three main ideas below: (1) algorithms via randomized polynomial evaluation, (2) algorithms via Weisfeiler-Leman (WL) color refinement, and (3) conditional lower bounds via isomorphism-preserving reductions. We strive to present these concepts without assuming a specialized background, with the goal of enabling a general theoretical computer science audience to reconstruct our proofs after reading this section.

2.1 Algorithms via Randomized Polynomial Evaluation (Details in Section 6)

Let us consider **Negative Triangle** as a guiding example. We define the *triangle polynomial* of a weighted undirected graph G as

$$P_{\Delta}(G) := \sum_{w_1 \leq w_2 \leq w_3} c_{w_1, w_2, w_3}(G) x_{w_1} x_{w_2} x_{w_3}$$

where $c_{w_1, w_2, w_3}(G)$ is the number of triangles in G whose edges possess the weight multiset $\{w_1, w_2, w_3\}$. For example, the first term of $P_{\Delta}(G)$ in Figure 1 is $2x_{-2}^2 x_1$, because there are two triangles in G with edge weights $\{-2, -2, 1\}$.

Obviously, if $P_{\Delta}(G) \equiv P_{\Delta}(H)$ (i.e., they are identically equal as polynomials), then G contains a negative triangle if and only if H does. (Note that this does not imply $G \cong H$, as exemplified in Figure 1). Our goal is therefore to efficiently identify a prior graph G_i such that $P_{\Delta}(G_i) \equiv P_{\Delta}(H)$ or rule out the existence of such G_i .

To achieve this without explicitly constructing the polynomials, we rely on randomized Polynomial Identity Testing. By the Schwartz-Zippel lemma, since P_{Δ} is a multivariate polynomial of degree 3, evaluating it at a random point drawn uniformly from a sufficiently large finite field $\mathbb{F}$ distinguishes non-identical polynomials with high probability. Specifically, we assign each possible variable x_w a random value $r_w \in \mathbb{F}$.

For **APSP**, ordered pairs receiving the same color have the same distance, allowing the stored distances to be transferred to the query graph.

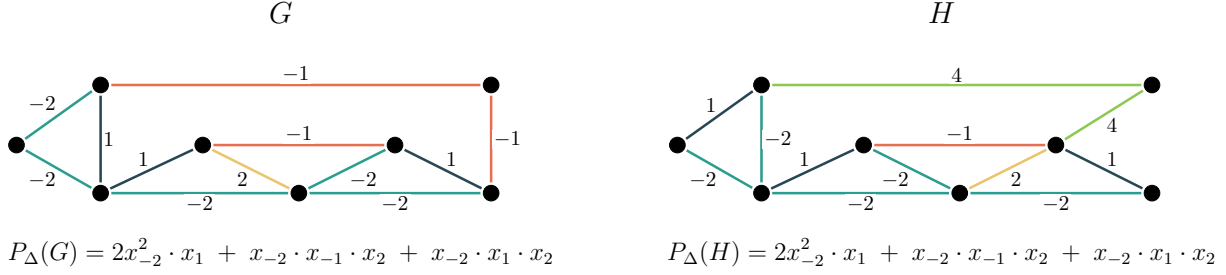


Figure 1: Examples of triangle polynomials in weighted graphs. Edges with the same color have the same weight. Note that G and H are not isomorphic (and in fact not even WL-equivalent).

Given this random assignment, we need to evaluate $P_{\Delta}(G)$ at $\mathbf{r} = (r_{w_1}, \dots, r_{w_m})$ in $\tilde{O}(n^{\omega})$ time. To this end, we construct an $n \times n$ matrix A where $A_{u,v} = r_{w(u,v)}$ if $(u,v) \in E(G)$, and 0 otherwise. The value of the polynomial evaluated at $\mathbf{r}$ is simply $\frac{1}{6}\text{Tr}(A^3)$ which is equal to the sum of weighted triangles in the graph with adjacency matrix A and can be computed via fast matrix multiplication in $\tilde{O}(n^{\omega})$ query time.

To summarize, during the preprocessing phase, we evaluate $P_{\Delta}(G_i)(\mathbf{r})$ for each prior G_i and store this field element as its invariant representation. This requires $\tilde{O}(kn^{\omega})$ total preprocessing time. When we receive a query graph H , we evaluate $P_{\Delta}(H)(\mathbf{r})$ using the exact same random assignment $\mathbf{r}$ in $\tilde{O}(n^{\omega})$ time. We then simply find a prior graph G_i that evaluates to the identical scalar value and return its precomputed solution $OPT(G_i)$. This takes $\tilde{O}(n^{\omega})$ query time.⁹ We can guarantee that, with high probability⁶, the random point $\mathbf{r}$ chosen during the preprocessing phase is such that, for every i , $P_{\Delta}(G_i)(\mathbf{r}) = P_{\Delta}(H)(\mathbf{r})$ if and only if $P_{\Delta}(G_i) \equiv P_{\Delta}(H)$. Thus, the answer given in the query phase is correct with high probability.

Additionally, if the actual negative triangle T_i in G_i is provided, we can find a negative triangle in H by reducing the problem to unweighted triangle detection, which takes $O(n^{\omega})$ time [Ita77]. (Roughly speaking, this is done by detecting an unweighted triangle in a tripartite graph consisting only of edges whose weights match those of T_i .)

To solve the Constrained Spanning Tree (CST) problem, we similarly define the *spanning tree polynomial* of a graph G as

$$P_{ST}(G) := \sum_{w_1 \leq \dots \leq w_{n-1}} c_{w_1, \dots, w_{n-1}}(G) \prod_{i=1}^{n-1} x_{w_i}$$

where $c_{w_1, \dots, w_{n-1}}(G)$ is the number of spanning trees with the weight multiset $\{w_1, \dots, w_{n-1}\}$.

We leverage the fact that this multivariate polynomial can be efficiently evaluated at a random point r by invoking Kirchhoff's Matrix Tree Theorem. Specifically, this evaluation reduces to computing the determinant of a randomized Laplacian matrix, which takes $\tilde{O}(n^{\omega})$ time via fast matrix multiplication. If the optimal tree T_i in G_i is provided, we can reconstruct a corresponding optimal tree T in H via matroid intersection in polynomial time [Sch03], and specifically in $\tilde{O}(m\sqrt{n})$ time via recent advancements [BMNT23, CLS⁺19, GS85]. Full details and the formal algebraic constructions are deferred to Section 6.2.

⁹There are some polylogarithmic factors needed to guarantee high success rate and for searching for the desired graph G_i .

2.2 Algorithms via Weisfeiler-Leman (WL) Coloring (Details in Sections 7 and 8)

We sketch an elementary argument here, designed to be accessible to readers unfamiliar with WL coloring, with full details deferred to Section 7. An alternative, more concise proof based on a more general perspective is provided in Section 8.

We use **Constrained Shortest Path (CSP)** as our guiding example. In the preprocessing phase, we color the vertices of each prior graph G_i using the Weisfeiler-Leman (WL) color refinement process, which runs in time $O((n + m) \log n)$ [CC82, PT87, BBG17]. An essential property of the resulting coloring is that *every two vertices u and v that receive the same color will have identically colored neighborhoods*.

For example, in Figure 2, every pink vertex (labeled p) is adjacent to the target t via an edge of attributes (length and cost) $(5, 2)$, to two cyan vertices (labeled c) via edges of attributes $(2, 4)$, and to one green vertex (labeled g) via an edge of attributes $(3, 3)$. Note that we initialize the process such that the source s and target t are assigned uniquely identifiable colors (red and blue in the example). The edge colors shown in the figure represent the edge attributes (and are not a result of the WL coloring).

Building on this property, we can easily establish the following claim (see the caption of Figure 2 for a concrete example).

Claim 2.1 (Informal). *Let G and H be two WL-equivalent graphs, meaning that for every color c , both graphs contain exactly the same number of vertices assigned color c . If there exists an s - t walk (i.e., an s - t path that is not necessarily simple) $W = (s = v_0, v_1, \dots, v_p = t)$ in G , then there exists a corresponding walk $W' = (s = v'_0, v'_1, \dots, v'_p = t)$ in H such that, for every step $i \in \{0, \dots, p-1\}$, the attributes (length and cost) of the edges (v_i, v_{i+1}) and (v'_i, v'_{i+1}) are identical.*

Proof Idea of Claim 2.1. Imagine placing a token at the source vertex s in both graphs G and H . At each step, we move the token in G along the walk W from a vertex of color c to a neighbor of color c' via an edge with attribute α . The definition of WL-equivalence guarantees that the token in H (which currently resides on a vertex of the exact same color c) has an identically attributed edge to a neighbor of color c' . By matching these moves step-by-step, we successfully build the corresponding walk W' in H . $\square$

As an immediate consequence of the claim above, every two WL-equivalent graphs G and H yield the exact optimal value, i.e., $OPT(G) = OPT(H)$.¹⁰ Naturally, during the query phase, our algorithm simply computes the WL coloring of the query graph H , searches for a prior graph G_i that is WL-equivalent to H , and returns $OPT(G_i)$. This process achieves the desired near-linear query time. Additionally, if each prior graph G_i is equipped with its optimal path P_i , we can efficiently reconstruct the corresponding optimal path in H using the token-tracking procedure described in the proof sketch.

Other path-centric optimization problems, such as ℓ_p -Shortest Path, Replacement Path, and Infinite-Duration Games, can be solved using a similar approach. Furthermore, we note that the Constrained Spanning Tree (CST) problem can be solved deterministically in $O(n^3 \log n)$ query time via 2-WL, a higher-dimensional variant of the WL coloring; see Section 8.

2.3 Lower Bounds (Details in Section 9)

Graph Isomorphism with Promise. Our starting point is the Graph Isomorphism (GI) problem itself, formulated within our model under a single prior instance ($k = 1$). In the preprocessing phase,

¹⁰To establish this, we rely on the standard fact that any s - t walk can be transformed into a simple s - t path by iteratively removing cycles, which does not increase the total cost or length of the walk since edge lengths and costs are non-negative.

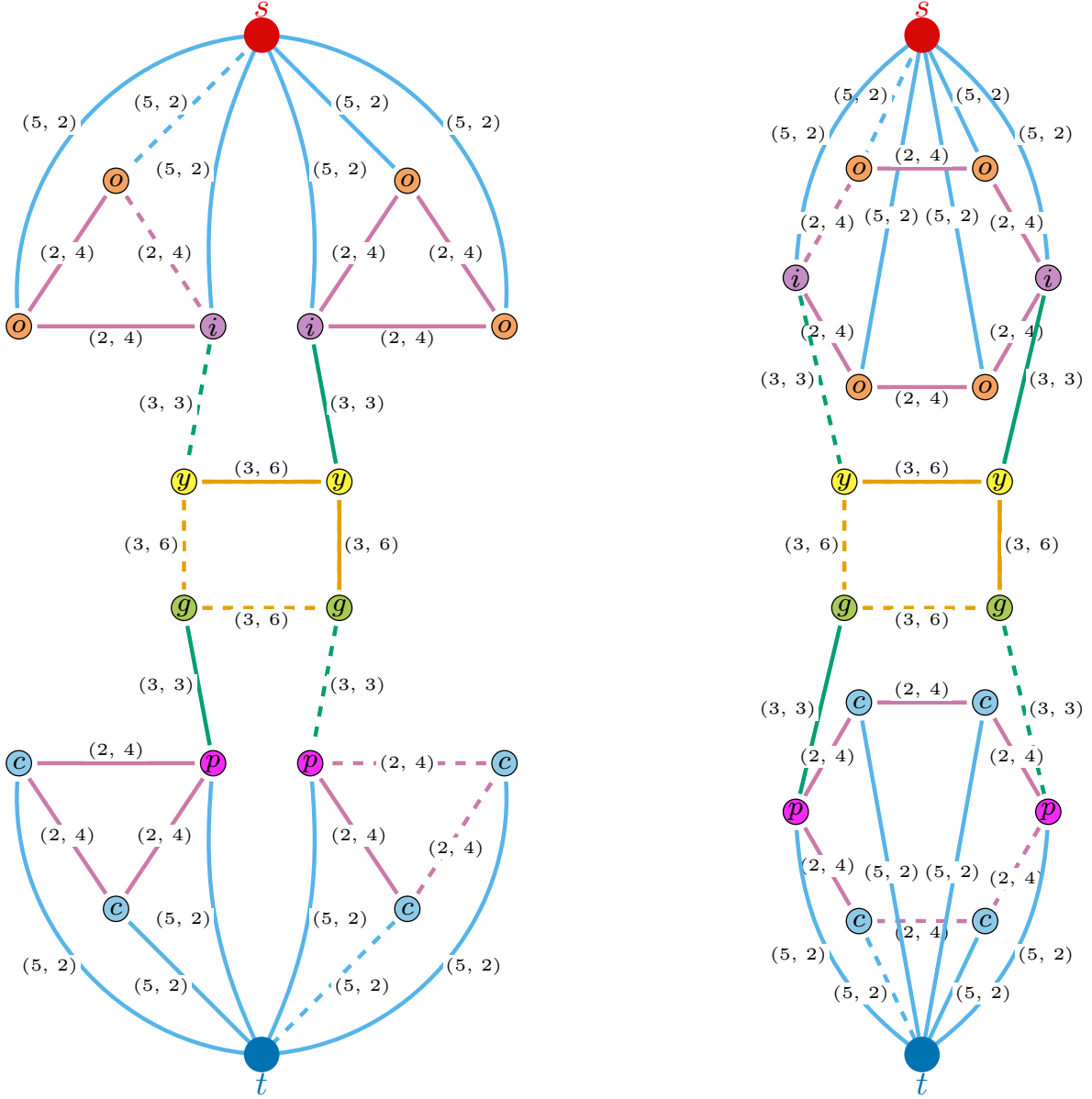


Figure 2: Two WL-equivalent but non-isomorphic graphs. Excluding s and t , each graph contains the exact same color distribution: 4 orange (o), 2 indigo (i), 2 yellow (y), 2 green (g), 2 pink (p), and 4 cyan (c) vertices. The dashed edges in the left graph illustrate an s - t walk with the edge-attribute sequence $((5, 2), (2, 4), (3, 3), (3, 6), (3, 6), (3, 3), (2, 4), (2, 4), (5, 2))$. As guaranteed by Claim 2.1, a corresponding walk with the exact same attribute sequence must exist in the right graph, which is realized by the dashed edges shown there.

the algorithm receives a prior pair of graphs (G, H) along with $OPT(G, H)$, which indicates whether $G \cong H$. During the query phase, the algorithm receives a new pair (G', H') and must either (i) decide whether $G' \cong H'$, or (ii) correctly declare that $(G, H) \not\cong (G', H')$ (i.e., $G' \not\cong G$ or $H' \not\cong H$).

We argue that this problem cannot be solved with polynomial preprocessing and query times unless $GI \in P$, even under the promise that $G' \cong G \cong H$. Let (X, Y) be an arbitrary worst-case instance of GI . We define the prior instance simply as $G = H = X$, meaning $OPT(G, H)$ trivially indicates that $G \cong H$. We then present the query instance as $G' = X$ and $H' = Y$. Observe that every algorithm capable of answering this query in polynomial preprocessing and query time would immediately decide whether $X \cong Y$ from scratch without prior, implying $GI \in P$.

From GI with Promise to Maximum Independent Set. Now we reduce GI with the $G' \cong G \cong H$ promise from above to **Maximum Independent Set** (equivalently, **Maximum Clique**) with a single prior. (This reduction already exists in [Koz78].) Assume for contradiction that there exists an algorithm A that takes a prior graph Z alongside the size of its maximum independent set¹¹, denoted $\alpha(Z)$, preprocesses this information in polynomial time, and subsequently, for every query graph Z' , either computes $\alpha(Z')$ or declares that $Z' \not\cong Z$ in polynomial time. We use A to solve GI under our single-prior setting as follows.

Constructing Prior Graph Z : Upon receiving the prior pair (G, H) for GI , our reduction constructs a prior graph Z as follows. For every $u \in V(G)$ and $v \in V(H)$, we create a vertex $z_{u,v}$ in Z . We add an edge between distinct vertices $z_{u,v}$ and $z_{u',v'}$ if the assignments $u \mapsto v$ and $u' \mapsto v'$ violate the isomorphism condition; i.e., we add an edge if either: (i) $(u = u') \oplus^{12} (v = v')$, or (ii) the adjacency is not preserved, meaning $(u, u') \in E(G)$ but $(v, v') \notin E(H)$, or vice versa. Observe that $\alpha(Z) = |V(G)|$ if and only if $G \cong H$.¹³ By the $G \cong H$ promise, we know that $\alpha(Z) = |V(G)|$. Thus, we pass Z and its solution $\alpha(Z) = |V(G)|$ to algorithm A for preprocessing.

Constructing Query Graph Z' : Upon receiving a query pair (G', H') for GI , we construct a graph Z' using the exact same procedure; hence, $\alpha(Z') = |V(G')|$ if and only if $G' \cong H'$. We then query algorithm A with Z' . If A returns $\alpha(Z') = |V(G')|$, we output $G' \cong H'$; otherwise, we declare that $G' \not\cong H'$.

We claim that this is always correct. If $(G, H) \cong (G', H')$, our construction guarantees that $Z \cong Z'$ (meaning our reduction is *isomorphism-preserving*). Consequently, A must output $\alpha(Z') = |V(G')|$. Conversely, if $(G, H) \not\cong (G', H')$, the promise that $G' \cong G \cong H$ forces the conclusion that $G' \not\cong H'$.¹⁴ Thus, A must either return $\alpha(Z') < |V(G')|$ or declare $Z \not\cong Z'$, both of which trigger our reduction to correctly declare $G' \not\cong H'$.

Finally, observe the crucial role of the isomorphism-preserving property here. If our reduction was not isomorphism-preserving, the condition $Z \not\cong Z'$ could arise regardless of whether $(G, H) \cong (G', H')$ or $(G, H) \not\cong (G', H')$. In such a scenario, if algorithm A outputs $Z \not\cong Z'$, we would be unable to distinguish whether $(G, H) \cong (G', H')$ (which dictates we must return $G' \cong H'$ due to the $G' \cong G \cong H$ promise) or if $(G, H) \not\cong (G', H')$.

Many NP-hardness reductions are naturally isomorphism-preserving, meaning many standard lower bounds carry over to our framework. A prime exception is Karp's standard reduction from SAT to 3-SAT [Kar72], which fails to preserve isomorphism. We circumvent this by routing our reduction

¹¹Recall that an independent set of a graph is a set of mutually non-adjacent vertices, and $\alpha(Z)$ denotes the size of the maximum independent set in Z .

¹²We use the logical xor to avoid self loops (when both $u = u'$ and $v = v'$).

¹³To see this, note that condition (i) ensures every independent set selects at most one vertex for each $u \in V(G)$ and $v \in V(H)$, bounding its size by $|V(G)|$. If $G \cong H$ via an isomorphism f , the set $\{z_{u,f(u)} \mid u \in V(G)\}$ forms an independent set of size exactly $|V(G)|$. Conversely, every independent set of size $|V(G)|$ defines a bijection, and condition (ii) guarantees this bijection preserves all edges and non-edges, forming a valid isomorphism.

¹⁴To see this, assume for contradiction that $G' \cong H'$. Due to the $G' \cong G \cong H$ promise, we have $H' \cong G' \cong G \cong H$. We would therefore have $(G', H') \cong (G, H)$, contradicting our initial assumption.

through graph coloring. We begin with Karp’s SAT to Coloring reduction, which is isomorphism-preserving. In the second step, we employ Lovász’s reduction from Coloring to 3-Coloring [Lov73].¹⁵ Finally, a standard mapping from 3-Coloring back to 3-SAT completes the chain, yielding the desired isomorphism-preserving reduction from SAT to 3-SAT. Figure 3 provides an overview of our reduction chains.

3 Open Problems

Our results raise several questions about the power and limitations of the Isomorphic-Priors model. We begin with four central directions: which fine-grained complexity barriers persist in our model, an Isomorphic-Priors k -Clique hypothesis with consequences for Graph Isomorphism, the approximability of hard optimization problems, and extensions beyond exact isomorphism. We then discuss structural questions about the sources of tractability and the broader scope of the framework.

Fine-Grained Complexity. Many questions remain for problems in P. For example, our algorithm for Negative Triangle uses randomness to achieve $\tilde{O}(kn^\omega)$ preprocessing time and $\tilde{O}(n^\omega)$ query time. Can the same bounds be achieved deterministically? Can we improve them to near-linear time?

More broadly, the standard weighted versions of Negative Triangle, Replacement Path, All-Pairs Shortest Paths (APSP), Minimum-Weight Cycle, Second Shortest Simple Path, and Radius are equivalent under subcubic reductions in the classical setting without priors [VW10, AGV15]. Yet our algorithms achieve different query bounds for Negative Triangle and Replacement Path, and it remains unclear whether the classical equivalences extend to our model. Doing so requires isomorphism-preserving reductions. Can we solve all the problems in this equivalence class in near-linear query time?

Beyond the APSP equivalence class, it is natural to ask whether fine-grained barriers based on OV, Hitting Set, and combinatorial Triangle Detection persist in the Isomorphic-Priors model, and whether such lower bounds can be transferred via isomorphism-preserving fine-grained reductions.

Isomorphic-Priors k -Clique Hypothesis. For every fixed integer $k \geq 3$ divisible by three, fast matrix multiplication solves k -Clique in $n^{\omega k/3 + o(1)}$ time. The standard k -Clique Hypothesis rules out any constant improvement in the exponent [Vas18]. We hypothesize that this barrier persists with isomorphic priors.

Hypothesis 3.1. (*Isomorphic-Priors k -Clique Hypothesis*) For every fixed integer $k \geq 3$ and constant $\epsilon > 0$, no isomorphic-priors algorithm can detect a k -clique in an n -vertex graph with both preprocessing and query times $O(n^{\omega k/3 - \epsilon})$, even with $O(1)$ prior graphs.

This hypothesis implies $GI \notin P$: A polynomial-time algorithm for Graph Isomorphism would allow us to compare a query graph against $O(1)$ priors and retrieve a matching prior’s answer, or report that none exists, in $n^{O(1)}$ time. This would contradict the hypothesis for sufficiently large fixed k . We provide further details and discuss weaker variants in Section 10. An important open direction is to refute these hypotheses or establish them under more standard complexity assumptions.

Approximability. Although we establish conditional lower bounds for exact computation and for some approximation problems, much of the approximation landscape remains open. For example, can we achieve a 0.876-approximation for Max 3-SAT, a 0.879-approximation for MaxCut, a 1.99-approximation for Vertex Cover, or an $n^{0.99}$ -approximation for Max Clique, with polynomial preprocessing and query times? Can isomorphic priors also enable a 1.385-approximation for Steiner Tree

¹⁵We refer the interested reader to <https://users.encs.concordia.ca/~chvatal/notes/color.html>

or a $4/3$ -approximation for metric TSP? These questions ask whether prior computations can help us surpass classical hardness thresholds or achieve approximation guarantees that remain elusive without priors. Conversely, which classical inapproximability bounds persist in our setting? In particular, can PCP constructions and gap-preserving reductions be adapted to establish such bounds while preserving the isomorphisms required by our framework?

Beyond Exact Isomorphism. A natural direction is to relax the strict isomorphism requirement. Our invariant-representation approach already allows exact answers to be transferred between some non-isomorphic graphs. Can this principle be extended to graphs whose representations are similar, rather than identical? For example, what happens if H is isomorphic to G'_i obtained G_i by small perturbations of its edge weights or adding/deleting a few edges? Is there a natural, efficiently computable distance between Weisfeiler–Leman (WL) histograms [SSvL⁺11] that yields useful bounds on the difference between $OPT(H)$ and $OPT(G_i)$? Under what assumptions could such bounds lead to approximation guarantees for **Constrained Shortest Path** with polynomial query time? Since small perturbations can change feasibility or cause large changes in the optimal value, meaningful guarantees may require additional assumptions on the instances or controlled relaxations of the constraints. Establishing such results would provide a principled way to reuse prior answers on new instances, with approximation guarantees tied to their similarity to the priors.

A complementary approach is to replace isomorphism with other structural relations, such as attribute-preserving homomorphisms or subgraph embeddings, and ask what information these relations allow us to transfer. For **Negative Triangle**, a weight-preserving homomorphism of a prior graph G_i into a query graph H transfers a positive answer, whereas a weight-preserving homomorphism from H to G_i transfers a negative answer. Can we efficiently identify and exploit such relations without being given the corresponding maps? What additional restrictions would allow these one-sided implications to yield fast query algorithms? More generally, which problem-specific relations support exact answers, approximation guarantees, or useful one-sided conclusions beyond the isomorphism classes of the priors?

Isomorphism-Preserving Algorithms. Our hardness results rely on isomorphism-preserving reductions. This raises a broader question: how much computational overhead is required to make basic algorithmic constructions consistent across isomorphic inputs? For example, a spanning tree of a connected graph can be computed in $O(n + m)$ time. Can we efficiently select such a tree while preserving isomorphism? Specifically, we seek an algorithm A that returns a spanning tree $T_G = A(G)$ of each connected graph G , such that whenever $G \cong H$, there exists an isomorphism $\varphi: G \rightarrow H$ satisfying $\varphi(T_G) = T_H$. Equivalently, we require

$$G \cong H \implies (G, T_G) \cong (H, T_H),$$

where an isomorphism of pairs must preserve both the graph edges and the distinguished spanning-tree edges. Can such a selection be computed in polynomial, or even near-linear, time? More generally, which efficiently solvable search problems admit efficient solution-selection procedures that are consistent in this sense, and how does their complexity relate to graph canonization? We can ask similar questions for other problems such as shortest paths, minimum cut, maximal and maximum matching, and $\Delta + 1$ coloring.

Beyond NP. Which natural problems complete for classes such as PSPACE, EXPTIME, or NEXPTIME admit polynomial preprocessing and query times in the Isomorphic-Priors model?¹⁶ How does this question extend to exact counting problems complete for #P, when the exact counts are supplied with the priors? More generally, which structural properties allow answers to classically hard problems to be efficiently reused, and which problems admit conditional lower bounds through isomorphism-preserving reductions? Understanding this landscape would clarify how the complexity of reusing prior answers differs from that of computing them from scratch.

The Power of Invariant Representations. Our algorithmic results rely on efficiently computable invariant representations (e.g., WL coloring or spanning tree polynomial) whose equality guarantees equality of optimal values. Does this approach characterize polynomial-time solvability in the Isomorphic-Priors model? Specifically, if a problem admits polynomial preprocessing and query times, must there exist a polynomial-time computable function f such that

$$G \cong H \implies f(G) = f(H) \implies \text{OPT}(G) = \text{OPT}(H)?$$

Here, f must be computable from the input graph alone, without access to its optimal value or to a database of priors. Alternatively, can jointly processing a query and the supplied prior answers yield algorithms that cannot be captured by such representations?

Characterizing Tractability. Our algorithmic results for NP-hard problems (consider for example Constrained Shortest Path or Constrained Spanning Tree) exploit numerical edge attributes, whereas problems on unweighted graphs such as Vertex Cover and Hamiltonian Cycle remain conditionally hard. To what extent do these attributes account for the difference? Can we identify natural NP-hard problems on unweighted undirected graphs, with no additional vertex or edge attributes, that admit polynomial preprocessing and query times? Here, we seek positive results beyond those that follow immediately from efficient isomorphism testing on restricted graph classes.

Constrained Shortest Path and Constrained Spanning Tree provide examples of weakly NP-hard problems that become tractable in our model. Do all problems admitting pseudopolynomial algorithms admit polynomial preprocessing and query times with isomorphic priors? Conversely, are there natural weakly NP-hard problems that remain conditionally intractable in our model? Understanding these questions would help distinguish the roles of numerical information and graph structure in enabling the reuse of prior computations.

From Values to Solutions. Our algorithms can reconstruct optimal solutions when suitable optimal witnesses are supplied with the priors, but these reconstruction arguments are problem-specific. Is there a general principle that converts efficient value-retrieval algorithms into efficient solution-reconstruction algorithms in our model? Standard decision-to-search reductions do not immediately suffice: the modified instances they generate need not be isomorphic to any prior, so the algorithm may abstain on them. Which properties of the witnesses or invariant representations permit efficient reconstruction? Can we identify a natural separation between retrieving optimal values and reconstructing optimal solutions, even when optimal prior solutions are supplied?

More Concrete Problems. Can makespan scheduling of unit-length jobs with precedence constraints on a fixed number of identical machines [NSW25], or Rabin games [MST24], be solved

¹⁶NFA Universality and Existential Length Universality for NFAs are PSPACE-complete and NEXPTIME-complete, respectively [GLR⁺20]. We believe that both admit polynomial preprocessing and query times in our setting by adapting our 1-WL techniques, under isomorphisms that preserve transition labels and initial and accepting states. However, a formal treatment of these extensions is beyond the scope of this paper.

with polynomial preprocessing and query times in the Isomorphic-Priors model? Can we obtain near-linear preprocessing time per prior instance and near-linear query time for global minimum vertex cuts in weighted directed graphs [CMT26, LNP⁺25], or for generalized Büchi games [CDHL16]?¹⁷

4 Related Works

A Possible Connection to Graph Machine Learning. While machine learning (ML) is not a focus of this work, we briefly discuss a conceptual connection between our model and graph learning. This discussion is exploratory: our results have formal implications for abstract isomorphism-invariant predictors, but whether these implications can inform the design or training of practical ML models remains to be established.

Consider an ML model M that takes a graph G as input and outputs some value $M(G)$. We call such a model *isomorphism-invariant* if, for every pair of isomorphic input graphs $G \cong H$, we have $M(G) = M(H)$. Isomorphism-invariance is a natural requirement for graph-level prediction tasks in which vertex identifiers carry no semantic meaning, and is a standard design principle in graph representation learning [MBSL19, KP19]. Examples of isomorphism-invariant prediction targets include structural graph quantities such as triangle counts, diameter, and chromatic number, as well as the optimal values of combinatorial optimization problems such as Maximum Cut or Traveling Salesman Problem. Many commonly used graph-learning architectures satisfy this symmetry by design: message-passing Graph Neural Networks (GNNs), when equipped with a permutation-invariant graph-level readout, yield isomorphism-invariant predictors [XHLJ19, MRF⁺19]; higher-order invariant GNN architectures have been developed as well [MBSL19, KP19]. Several Graph Transformer architectures similarly incorporate isomorphism-respecting structural or positional encodings [KBH⁺21]. The following observation gives a concrete connection between our model and the construction of isomorphism-invariant graph predictors.

Observation 4.1. *Let P be an isomorphism-invariant function on graph instances that does not admit an algorithm with preprocessing and query times polynomial in n and k in the Isomorphic-Priors model. Then there is no training procedure that, for every correctly labeled training set*

$$\mathcal{S} = \{(G_1, P(G_1)), \dots, (G_k, P(G_k))\},$$

constructs an isomorphism-invariant predictor $M_{\mathcal{S}}$ satisfying all of the following:

- *Efficiency: The training time, inference time, and description length of $M_{\mathcal{S}}$ are each polynomial in n and k .*
- *Training completeness: The predictor answers correctly on every training example: $M_{\mathcal{S}}(G_i) = P(G_i)$ for every $i \in [k]$.*
- *Soundness: The predictor never returns an incorrect answer: For every query graph H , $M_{\mathcal{S}}(H) \in \{P(H), \perp\}$, where $\perp$ denotes abstention.*

Proof. Suppose such a training procedure exists. We use it to compute P in the Isomorphic-Priors model.

During preprocessing, given the prior graphs $G_1, \dots, G_k$ together with $P(G_1), \dots, P(G_k)$, we run the training procedure on the corresponding training set $\mathcal{S}$ and store the resulting predictor $M_{\mathcal{S}}$.

Given a query graph H , we evaluate $M_{\mathcal{S}}(H)$. If the output is not $\perp$, we return it; otherwise, we report that H is not isomorphic to any prior graph. Soundness guarantees that every non-abstaining answer is correct. To justify the report upon abstention, suppose that $H \cong G_i$ for some $i \in [k]$. Then

$$M_{\mathcal{S}}(H) = M_{\mathcal{S}}(G_i) = P(G_i) = P(H),$$

¹⁷For strict definitions and discussions of Rabin games and generalized Büchi games in the Isomorphic-Priors model, see Section A.2.

where the equalities follow, respectively, from the isomorphism-invariance of M_S , the training completeness, and the isomorphism-invariance of P .

The resulting algorithm has polynomial preprocessing and query times, contradicting the assumed hardness of P in the Isomorphic-Priors model. $\square$

For example, assuming $GI \notin P$, our lower bound for TSP rules out an isomorphism-invariant learner that simultaneously guarantees efficiency, training completeness, and soundness for every correctly labeled training set. Such a learner must therefore relax at least one of these requirements. For example, one may allow incorrect predictions on some query graphs, or require the guarantees to hold only with high probability when the training set is sampled from a specified distribution, rather than for every possible training set. The observation alone does not rule out useful learning-based methods under such relaxations.

In the other direction, our invariant-representation algorithms for, e.g., CSP and Energy Game construct predictors satisfying all three conditions for every correctly labeled training set. These predictors answer correctly on every graph isomorphic to a training example and can also transfer answers to non-isomorphic graphs with matching representations. Thus, these constructions establish the feasibility of the abstract predictor requirements, but do not by themselves show that a particular neural architecture can represent or learn the resulting predictors. Natural next questions are whether concrete graph-learning architectures can learn or exploit the identified representations, and under what structural or distributional assumptions they can provide non-abstaining answers on graphs not isomorphic to any training example.

Beyond Worst-Case Analysis. Our work contributes to the long-standing effort in theoretical computer science to define models that bypass overly pessimistic worst-case lower bounds. Classical approaches in this vein include average-case analysis and smoothed analysis, which assume inputs are drawn from specific distributions or subjected to random perturbations to model typical real-world data rather than adversarial instances. Other paradigms, such as instance optimality and universal optimality, abandon uniform worst-case bounds altogether; instead, they require an algorithm’s performance to be competitive with the best possible algorithm tailored specifically to the given input instance or network topology. In the emerging field of learning-augmented algorithms (or algorithms with predictions), an algorithm receives a prediction (often learned from prior data) to guide its execution, with its guarantees degrading gracefully depending on the noise or error in the prediction.

Our model differs from these frameworks in that we explicitly allow the algorithm to preprocess prior data without assuming that this data comes from a certain distribution or that there is an oracle that provides predictions. We believe our model offers an orthogonal angle to study beyond-worst-case analysis, directly answering whether we can bypass existing lower bounds for graphs that are structurally equivalent to those we have seen before.

Graph Isomorphism (GI). Clearly, if we can test whether two n -vertex, m -edge graphs are isomorphic in $T(n, m)$ time, then every problem in our model can be solved with $O(k \cdot T(n, m))$ query time and zero preprocessing time. Furthermore, if a more demanding problem called *graph canonization* can be solved in $T(n, m)$ time, we can eliminate the dependence on k in the query phase entirely, reducing the query time to $O(T(n, m))$ at the expense of an $O(k \cdot T(n, m))$ preprocessing time.¹⁸

¹⁸In graph canonization, an algorithm maps every graph G to a canonical string $f(G)$ such that $f(G) = f(H)$ if and only if G and H are isomorphic. Since the length of $f(G)$ is bounded by $T(n, m)$, we can spend $O(k \cdot T(n, m))$ preprocessing time to insert the k canonical forms into a deterministic trie. For a query graph H , computing $f(H)$ and traversing the trie takes exactly $O(T(n, m))$ time, matching the overall query bound.

This implies that all problems in our setting can be solved in quasipolynomial time using Babai’s landmark GI algorithm [Bab16, Bab19], and even in polynomial time for restricted graph classes, such as planar [HW74], bounded-degree [Luk82, GNS23], or bounded-treewidth graphs [Bod90, LPPS17, GNSW20], and more generally, all classes that exclude a fixed graph as a topological minor [GM15, Neu24b]. However, it remains a major open question whether **Graph Isomorphism** is solvable in polynomial time for general graphs. Moreover, to beat the $O(n^3)$ bound for **Negative Triangle**, we would need a canonization algorithm running in $O(n^{3-\epsilon})$ time, which appears even more elusive.¹⁹

The core technical challenge of our model is therefore: *which specific problem structures allow us to bypass the general Graph Isomorphism barrier?* This relationship goes both ways: Any failure to achieve fast algorithms in our model can be viewed as a barrier to fast graph isomorphism testing. Ultimately, we hope that future studies within the Isomorphic-Priors framework will reveal the fundamental combinatorial structures that make **Graph Isomorphism** inherently difficult.

Weisfeiler-Leman (WL) Color Refinement. The Weisfeiler-Leman (WL) algorithm is a fundamental heuristic for isomorphism testing (see, .e.g, [GN21, Kie20b, Kie20a]). On the positive side, Atserias and Maneva [AM13] established relationship between Sherali-Adams (SA) hierarchy of linear programming relaxations and the levels of the WL algorithm. Consequently, higher-dimensional WL colorings can serve as invariant representations for polynomial-time solvable problems formulated via basic LP relaxations, such as *maximum bipartite matching* and **Maximum Flow**. However, since these problems already admit efficient algorithms, this connection does not yield new speedups in our model; in fact, we show in this paper that the use of WL coloring can be refined to achieve near-linear time in our setting.

Another line of work, which is largely orthogonal to our focus, aims to demonstrate that WL conceptually subsumes other polynomial-time graph isomorphism heuristics (a motivation explicitly discussed by Arvind et al. [AFKV22]).

Strikingly, beyond these LP-based equivalences, positive algorithmic results using WL invariants to accelerate exactly or approximately hard problems appear virtually nonexistent in the literature. In this light, a core conceptual contribution of our work is demonstrating that this well-known technique can be easily leveraged to bypass traditional worst-case computational barriers.

In contrast to the scarcity of positive results, there is a rich lineage of negative and inapproximability bounds associated with the WL hierarchy. As opposed to relying on standard complexity assumptions (e.g., $P \neq NP$ or the Unique Games Conjecture), this line of work leverages WL, often formulated via its equivalence to infinitary first-order logic with counting, as a mathematical proxy to unconditionally rule out algorithms expressible in fixed-point logic with counting (FPC). Through this lens of unconditional inexpressibility, Atserias and Dawar [AD19] proved that k -WL cannot approximate **Vertex Cover** to a factor better than $7/6$ (alongside other inapproximability results), while Tucker-Foltz [Tuc24] recently established inapproximability thresholds for Unique Games under WL refinement. For exact computation, Göbel et al. [GGR24] showed that finding a dominating set of size k requires a WL dimension of at least k , fitting into a broader understanding of WL’s limitations on conjunctive queries.

Another related line of work explores the relationship between WL coloring and homomorphism counts [Dvo10, Dvo10, DGR18, GRS22], as well as (induced) subgraph counts [Neu24a, LB24, CN25]. This homomorphism framework has recently become a central tool in the machine learning community for characterizing the expressiveness of Graph Neural Networks (GNNs) [BFZB23, ZGD⁺24]. We purposefully present both elementary combinatorial arguments (Section 7) and the more succinct homomorphism formulation (Section 8) to ensure accessibility for a broad algorithmic audience. In a

¹⁹A simple isomorphism test running in $O(n^{3-\epsilon})$ time would not suffice, as it yields an $O(k \cdot n^{3-\epsilon})$ query time in our setting. This trivially exceeds the naive $O(n^3)$ baseline whenever $k = \omega(n^\epsilon)$.

similar spirit, it is known that the 2-dimensional Weisfeiler-Leman (2-WL) algorithm subsumes the spectral information of the adjacency matrix (as well as other associated graph matrices) [Für10, RS23, AFKV25], which we exploit for our deterministic algorithms for Constrained Spanning Tree and ℓ_p -Spanning Tree.

5 Our Model of Computation

5.1 Preliminary: Attributed Graphs and Isomorphism

Throughout this work, we consider graphs whose vertices and edges may carry *attributes*. For instance, a vertex attribute may be used to designate sources or sinks, while an edge may carry a vector attribute representing multiple edge weights. The specific attributes depend on the problem under consideration.

For example, in **Negative Triangle**, the attribute of each edge is simply its weight, and there are no vertex attributes (i.e., every vertex receives the same default attribute, denoted by $\perp$). For **Constrained Shortest Path**, the attribute of each edge e with length $\ell(e)$ and cost $c(e)$ is the vector $\alpha(e) = (\ell(e), c(e))$. The vertex attributes designate the terminals: $\alpha(v) = \mathbf{s}$ if v is the source, $\alpha(v) = \mathbf{t}$ if v is the sink, and $\alpha(v) = \perp$ otherwise.

We adopt the natural notion of *isomorphism* for attributed graphs: an isomorphism between two graphs G and H is a bijection between their vertex sets that preserves adjacency as well as all vertex and edge attributes.

Definition 5.1 (Isomorphism of attributed graphs). Let

$$G = (V_G, E_G, \alpha_G^V, \alpha_G^E) \quad \text{and} \quad H = (V_H, E_H, \alpha_H^V, \alpha_H^E)$$

be two graphs, where V_G, V_H are vertex sets, E_G, E_H are edge sets, and α^V, α^E assign attributes from alphabets Σ_V, Σ_E to the vertices and edges, respectively.

We say that G and H are *isomorphic* (denoted $G \cong H$) if there exists a bijection $\varphi : V_G \rightarrow V_H$ such that the following conditions hold.

1. **Adjacency is preserved:** $(u, v) \in E_G \iff (\varphi(u), \varphi(v)) \in E_H$ for all $u, v \in V_G$.
2. **Vertex attributes are preserved:** $\alpha_G^V(u) = \alpha_H^V(\varphi(u))$ for all $u \in V_G$.
3. **Edge attributes are preserved:** $\alpha_G^E(u, v) = \alpha_H^E(\varphi(u), \varphi(v))$ for all $(u, v) \in E_G$.

Remark 5.2 (Isomorphism-closed problems). This paper focuses on *isomorphism-closed problems*, a class encompassing virtually all natural computational tasks on graphs and related objects. Let Π be a decision or optimization problem (e.g., **Constrained Shortest Path**, **Negative Triangle**, **SAT**, or **Energy Game**). Each problem Π we consider is equipped with an equivalence relation $\cong$ defining when two problem instances are isomorphic.²⁰ We assume that Π is inherently *closed under isomorphism*. For a decision problem, this means $I \cong J$ implies that I is a YES-instance if and only if J is a YES-instance. For an optimization problem, $I \cong J$ implies that both instances share the identical optimal value, meaning $\text{OPT}_\Pi(I) = \text{OPT}_\Pi(J)$.

5.2 The Isomorphic-Priors Model

We study every problem Π within a model consisting of two phases:

²⁰For graph problems (e.g., **GI**, **CSP**, **CST**, **IS**, **MaxCut**), this corresponds to standard or attributed graph isomorphism (see Definition 5.1). For problems like **SAT**, two CNF formulas Φ and Φ' are isomorphic if Φ' can be obtained from Φ via a bijection of variables (preserving negations) alongside arbitrary permutations of clauses and literals.

Preprocessing Phase. We are given a database $\mathcal{S} = \{G_1, G_2, \dots, G_k\}$ of k attributed graphs, referred to as *prior graphs*.²¹ We assume every graph in $\mathcal{S}$ has exactly n vertices and m edges, and that all global problem parameters are fixed across all instances. Each $G_i \in \mathcal{S}$ is provided alongside its exact solution, denoted by $OPT(G_i)$. For decision problems (e.g., **Negative Triangle**), $OPT(G_i) \in \{\text{YES}, \text{NO}\}$. For optimization problems (e.g., **Constrained Shortest Path**), $OPT(G_i)$ defaults to the optimal objective *value*. Note, however, that if the database equips G_i with an explicit *optimal solution* (e.g., the actual optimal path or vertex set), all our algorithms are capable of reconstructing and outputting the exact optimal solution for the query graph as well.

The goal of the preprocessing phase is to compute a data structure over $\mathcal{S}$ to accelerate future queries. Generally, we consider polynomial preprocessing time as acceptable.

Query Phase. Upon receiving a query graph H , the algorithm must satisfy the following correctness guarantees. If H is isomorphic to some prior graph $G_i \in \mathcal{S}$, the algorithm must output the correct solution $OPT(H) = OPT(G_i)$. Otherwise, if H is not isomorphic to any graph in $\mathcal{S}$, the algorithm must either (i) output the correct solution $OPT(H)$, or (ii) correctly declare that $H \not\cong G_i$ for all $G_i \in \mathcal{S}$.

We say an algorithm is correct *with high probability* (w.h.p.) if it answers the query correctly with probability at least $1 - 1/N^c$ for any arbitrarily large constant $c \geq 1$, where N denotes the input size (of a single query). This probability is taken over the internal randomness of the algorithm. Throughout, our algorithms are analyzed under the standard *oblivious adversary* model; that is, we assume the sequence of queries is generated independently of the algorithm’s internal random choices (see, e.g., [NY15] and [KL14, Definition 3.4]).

Definition 5.3. We say that a problem Π is *polynomial-time solvable inside the Isomorphic-Priors model* if it can be solved in polynomial preprocessing time and polynomial query time.

5.3 Our Focused Approach: Invariant Representations

All our algorithms employ essentially the same unified design framework called *invariant representation*. In the following, we give an abstract description in its most basic form. Let $\mathcal{G}$ be the space of all valid input graphs for a computational problem Π . Algorithms in this framework compute a function $f: \mathcal{G} \rightarrow \mathcal{X}$, for some $\mathcal{X}$ called the *feature space*, that satisfies the following.

1. **Permutation Invariance:** For every $G, H \in \mathcal{G}$, if $G \cong H$, then $f(G) = f(H)$.
2. **Solution-Preserving:** For every $G, H \in \mathcal{G}$, if $f(G) = f(H)$, then $OPT_\Pi(G) = OPT_\Pi(H)$.

We say that G and H are *f-equivalent* if $f(G) = f(H)$. We call such an f an *Invariant and Solution-Preserving Signature* for Π . It is easy to see that Invariant and Solution-Preserving Signatures that are efficiently computable suffice to solve problems in our setting. For the next theorem, we assume that $\mathcal{X} = \{0, 1\}^*$ (which can always be achieved by encoding objects from $\mathcal{X}$ using binary strings).

Theorem 5.4. *Let Π be a graph optimization or decision problem and let $f: \mathcal{G} \rightarrow \{0, 1\}^*$ be an Invariant and Solution-Preserving Signature for Π . Also let $T(n)$ denote the time required to evaluate f on inputs of size at most n .*

Then there is an algorithm that solves Π with k prior instances each of size at most n in $O(k \cdot T(n))$ preprocessing time and $O(T(n))$ query time.

²¹For simplicity, we focus on defining the model for graphs, though this framework naturally extends to every combinatorial object equipped with a well-defined notion of isomorphism.

Proof. Let $G_1, \dots, G_k$ denote the prior instances, which all have size at most n . In the preprocessing phase, we compute all signatures $f(G_i) \in \{0, 1\}^*$ and store them together with the corresponding OPT_i in a trie. Note that storing a single signature string in a trie takes time proportional to the length of the string, which is bounded by $O(T(n))$. Hence, the preprocessing phase runs in time $O(k \cdot T(n))$.

In the query phase, we are getting an instance H . We first compute the signature $f(H)$ and search this string (together with the corresponding optimal value) in the trie. If there is some $i \in [k]$ such that $f(G) = f(H)$, then $OPT_{\Pi}(G_i) = OPT_{\Pi}(H)$, and we return the stored optimal value $OPT_{\Pi}(G_i)$. If no matching signature is found, then we return that $H \not\cong G_i$ for ever $i \in [k]$. $\square$

We stress that, although all our algorithms follow the basic approach outlined above, we usually do not rely on Theorem 5.4 as a black-box. First, in several cases, our algorithms are randomized, i.e., the output of the signature function f depends on random bits. In this case, the preservation of solutions is usually achieved only with high probability.

Also, in all cases discussed in this paper, the Invariant and Solution-Preserving Signatures we build can even be used to reconstruct optimal solutions (and not only the value). However, as already discussed above, the reconstruction is problem-specific, and does not follow from the general argument above (see also [From Values to Solutions](#)).

6 Algorithms via Randomized Polynomial Evaluation

In this section, we show how to utilize basic ideas from the field of randomized algorithm to obtain fast algorithms in the Isomorphic-Priors model. Our approach maps the multiset of edge weights of specific subgraphs (e.g. triangles, spanning trees) to the monomials of a formal multivariate polynomial. To efficiently compare these polynomials without explicitly expanding them, we rely on Polynomial Identity Testing (PIT) over finite fields. To this end, we use the Schwartz-Zippel Lemma.

Lemma 6.1 (Schwartz-Zippel Lemma). *Let $P(x_1, \dots, x_m)$ be a non-zero multivariate polynomial of total degree d over a field $\mathbb{F}$. If $r_1, \dots, r_m$ are chosen independently and uniformly at random from a finite subset $S \subseteq \mathbb{F}$, then $\Pr[P(r_1, \dots, r_m) = 0] \leq \frac{d}{|S|}$.*

We start with **Negative Triangle** where we are presented with a weighted graph G and have to decide if G contains a negative triangles (see Definition 6.4). The assumption that no algorithm solves this problem in time $O(n^{3-\varepsilon})$ for any $\varepsilon > 0$ is a widely used hardness assumption in fine-grained complexity theory (see [VW10]). However, we show that inside the Isomorphic-Priors model, we can solve the problem in time $O(n^\omega)$, where $\omega < 2.373$ denote the matrix multiplication exponent.

Theorem 6.2. *There is an algorithm that solves **Negative Triangle** with k prior instances each having at most n vertices in $\tilde{O}(k \cdot n^\omega)$ preprocesses time and $\tilde{O}(n^\omega)$ query time with high probability.*

Additionally, if each prior G_i that contains a negative triangle is equipped with a weight multiset $\{w_1^{(i)}, w_2^{(i)}, w_3^{(i)}\}$, then we can output the specific vertices of a negative triangle in H in $\tilde{O}(n^\omega)$ time with high probability.

Next, we study the **Constrained Spanning Tree (CST)** problem. Here, the input is graph with cost and length attributes attached to each edge and a budget B . The problem is to find a spanning tree that minimizes the cost and respect a budget constraint (see Definition 6.6). It is well-known that this problem is NP-complete. However, we show that inside the Isomorphic-Priors model, we can solve the problem in time $O(n^\omega)$.

Theorem 6.3. *There is an algorithm that solves the **CST** problem with k prior instances each having at most n vertices in $\tilde{O}(k \cdot n^\omega)$ preprocesses time and $\tilde{O}(n^\omega)$ query time with high probability.*

Additionally, if each prior instance G_i is equipped with a multiset of cost-distance pairs $W_i := \{w_1^{(i)}, \dots, w_{n-1}^{(i)}\}$, we can explicitly output the corresponding spanning tree in H in $\tilde{O}(n^\omega)$ time with high probability.

6.1 Negative Triangle

Definition 6.4 (Negative Triangle). An instance is defined by an undirected, edge-weighted graph $G = (V, E, w)$, where $w : E \rightarrow \mathbb{R}$ assigns a real weight to each edge. The objective is to determine whether there exist three vertices $u, v, x \in V$ forming a triangle such that the sum of their edge weights is strictly negative: $w(u, v) + w(v, x) + w(x, u) < 0$.

Two problem instances G and H of **Negative Triangle** are isomorphic if their underlying graphs are isomorphic. Note that an isomorphism has to preserve edge weights. Therefore, **Negative Triangle** is closed under isomorphism.

In order to use Lemma 6.1, we have to find a way how to detect (or count) negative triangles via polynomial. To this end, we associate a formal variable x_w with every distinct edge weight w . The *triangle polynomial* of G is defined as:

$$P_\Delta(G) := \sum_{w_1 \leq w_2 \leq w_3} c_{w_1, w_2, w_3}(G) x_{w_1} x_{w_2} x_{w_3}$$

where $c_{w_1, w_2, w_3}(G)$ is the exact number of triangles in G whose edges possess the weight multiset $\{w_1, w_2, w_3\}$. See Figure 1. We now, obtain that $P_\Delta(G)$ encodes all different kinds of triangles inside of G and thus especially the negative triangles.

Lemma 6.5. *If G and H are two graphs such that $P_\Delta(G) \equiv P_\Delta(H)$, then G contains a negative triangle if and only if H contains a negative triangle.*

Proof. If $P_\Delta(G) \equiv P_\Delta(H)$ then the coefficients of every corresponding monomial match exactly, i.e. $c_{w_1, w_2, w_3}(G) = c_{w_1, w_2, w_3}(H)$ for all combinations of weights. Consequently, G contains a weight combination with $c_{w_1, w_2, w_3}(G) > 0$ and $w_1 + w_2 + w_3 < 0$ if and only if H contains the exact same number of such negative triangles. $\square$

Proof of Theorem 6.2. Let $\mathcal{S} := \{G_i = (V_i, E_i, w_i)\}_{i=1\dots k}$ be a database of prior instances and assume that for each $i \in [k]$ we are given $\text{OPT}(G_i)$ that indicates whether G_i contains a negative triangle.

Preprocessing: Given $\mathcal{S}$, we first compute a prime number $p \geq 3k^2 n^{\log(n)}$ and construct a finite field $\mathbb{F}_p$. Note that all basic operations in $\mathbb{F}_p$ can be performed in time $O(\log p) = \tilde{O}(1)$. Next, we assign arbitrarily a unique non-zero element in $\mathbb{F}_p$ to each edge-weight of the graphs G_i and store the edge-weights together with the corresponding field elements in a binary search tree. Formally, we construct a function $\text{enc}_\mathcal{S} : \mathbb{R} \rightarrow \mathbb{F}_p \cup \{\perp\}$ that, on input x , either outputs an unique element in $\mathbb{F}_p$ if x is the weight of some graph in $\mathcal{S}$, or returns $\perp$ otherwise. Note that $\text{enc}_\mathcal{S}$ can be computed in time $O(k \cdot n^2 \cdot \log p)$, and evaluating $\text{enc}_\mathcal{S}$ on some x takes time $O(\log(kn^2) \cdot \log p) = \tilde{O}(1)$.

Given a graph G_i , we write $\text{enc}_\mathcal{S}(G_i)$ for graph where each edge weight w was replaced with $\text{enc}_\mathcal{S}(w)$. This can be done in time $\tilde{O}(kn^2)$.

We construct a $(k \cdot n^2)$ -wise independent family of hash functions $h : \mathbb{F}_p \rightarrow \mathbb{F}_p$ using [Vad12, Construction 3.32]. Concretely, we sample a random polynomial over $\mathbb{F}_p$ of degree at most $k \cdot n^2 - 1$. Note that such a hash function can be constructed in time $O(k \cdot n^2)$. Given an edge-weight w , we define $r(w) := h(\text{enc}_\mathcal{S}(w))$. For a graph G_i , we write $r(G_i)$ for the graph obtained by replacing each edge weight w with $r(w)$.

Let $x_1, \dots, x_m \in \mathbb{F}_p$, where $m \leq k \cdot n^2$, be a sequence of values. Using the Fast Fourier Transformation, we can evaluate $h(x_i)$ for all i in time $\tilde{O}(k \cdot n^2)$. Thus, we can compute $\tilde{G}_1 :=$

$r(G_1), \dots, \tilde{G}_k := r(G_k)$ in time $\tilde{O}(kn^2)$. To be more specific, let $w_1, \dots, w_m \in \mathbb{R}$ be an enumeration of all edge-weights of graphs in $\mathcal{S}$. Note that $m \leq k \cdot n^2$. We compute the vector $\mathbf{r} = (r_1 := r(w_1), \dots, r_m := r(w_m))$ in time $\tilde{O}(kn^2)$. Further, we store each r_i with the corresponding w_i into a binary data structure $R_{\mathcal{S}}: \mathbb{R} \rightarrow \mathbb{F}_p$. Note that we can evaluate $R_{\mathcal{S}}(w)$ in time $O(\log(kn^2)) = \tilde{O}(1)$. We write $A_{\tilde{G}_i}$ for the adjacency matrix of $\tilde{G}_i$. Note that this is a matrix over $\mathbb{F}_p$.

Next, we define $f_{\Delta}(G) := \frac{1}{6} \text{Tr}(A_{\tilde{G}}^3)$.²² Computing $A_{\tilde{G}}^3$ takes time $\tilde{O}(n^{\omega})$. Therefore, calculating all $f_{\Delta}(G_i)$ is in time $\tilde{O}(kn^{\omega})$. For each $i \in [k]$, we compute $f_{\Delta}(G_i)$ and store the value together with $OPT(G_i)$ in a linear data structure. Formally, we define a data structure $D_{\mathcal{S}}$ that on input x either returns $D_{\mathcal{S}}(x) = OPT(G_i)$ for a graph G_i with $f_{\Delta}(G_i) = x$ or returns that no such graph exist. $D_{\mathcal{S}}$ can be constructed in time $\tilde{O}(k)$ and can be queried in time $\tilde{O}(1)$. Hence, the preprocessing phase takes time $\tilde{O}(kn^{\omega})$.

Query: Given an input graph H , we use $\text{enc}_{\mathcal{S}}$ to check that all weights of H correspond to weights of matrices in $\mathcal{S}$. If this is not the case then our algorithm claims that H is not isomorphic to any graph in $\mathcal{S}$. Next, we use the data structure $R_{\mathcal{S}}$ to compute $\tilde{H} := r(H)$ in time $\tilde{O}(n^2)$. We then again apply fast matrix multiplication to compute $f_{\Delta}(H)$ in time $\tilde{O}(n^{\omega})$. Lastly, we check if $x := f_{\Delta}(H)$ is in $D_{\mathcal{S}}$. If this is the case then we return $D_{\mathcal{S}}(x)$, otherwise we report that H is not isomorphic to any graph in $\mathcal{S}$. This can be done in time $\tilde{O}(1)$. Hence, the whole query time is in $\tilde{O}(n^{\omega})$.

Correctness: As a first step, we show

$$f_{\Delta}(G) := \frac{1}{6} \text{Tr}(A_{\tilde{G}}^3) = \sum_{w_1 \leq w_2 \leq w_3} c_{w_1, w_2, w_3} r(w_1) r(w_2) r(w_3) = P_{\Delta}(G)(\mathbf{r}), \quad (1)$$

where $P_{\Delta}(G)(\mathbf{r})$ denotes the evaluation of $P_{\Delta}(G)$ where we assign the value $r(w_i)$ to each variable x_{w_i} . A standard result (e.g. [FH15, Fact 1]) states that

$$\text{Tr}(A_{\tilde{G}}^3) = \sum_{x_1 \in V(G)} \sum_{x_2 \in V(G)} \sum_{x_3 \in V(G)} r(w(x_1, x_2)) \cdot r(w(x_2, x_3)) \cdot r(w(x_3, x_1)),$$

where $w(x_i, x_j)$ is equal to the weight of the edge from x_i to x_j and $r(w(x_i, x_j)) = 0$ if there is no edge from x_i to x_j . Next, observe that the vertices $x_1, x_2, x_3 \in V(G)$ form a triangle in G if and only if $r(w(x_1, x_2)) \cdot r(w(x_2, x_3)) \cdot r(w(x_3, x_1))$ appears exactly six times in the sum above.²³ This yields Equation (1).

Let H be a graph on which we are queried. The graph H can be isomorphic to some $G_i \in \mathcal{S}$ only if all edge-weights of H appear as edge-weights in $\mathcal{S}$. If this is not the case then our query algorithm rejects. Thus, we assume that all edge-weights in H appear in the list $w_1, \dots, w_m$.

We now consider two cases. First, suppose that H is isomorphic to some G_i . Then, $f_{\Delta}(G_i) = f_{\Delta}(H)$ due to Equation (1) and the fact that $P_{\Delta}(G_i) \equiv P_{\Delta}(H)$ since $H \cong G_i$. Thus, $D_{\mathcal{S}}(f_{\Delta}(H))$ returns $OPT(G_i)$ as long as there aren't any different graphs G_j with $f_{\Delta}(G_i) = f_{\Delta}(G_j)$ and $OPT(G_i) \neq OPT(G_j)$. By Lemma 6.5, this only occurs if $P_{\Delta}(G_i) \neq P_{\Delta}(G_j)$ but $P_{\Delta}(G_i)(\mathbf{r}) = P_{\Delta}(G_j)(\mathbf{r})$. Since all r_i are chosen independently and uniformly at random over $\mathbb{F}_p$, Lemma 6.1 yields that the event $P_{\Delta}(G_i) \neq P_{\Delta}(G_j)$ and $P_{\Delta}(G_i)(\mathbf{r}) = P_{\Delta}(G_j)(\mathbf{r})$ has probability at most $\frac{3}{p}$ for every fixed index $j \neq i$. Using union bound, we obtain that the failure probability is at most $\frac{3k}{p} \leq k^{-1} n^{-\log(n)}$. Thus, our success probability is at least $1 - k^{-1} n^{-\log(n)}$ and there w.h.p with respect to the query size n .

Second, suppose H is non-isomorphic to every $G_i \in \mathcal{S}$ and let $OPT(H)$ denote the value indicating whether H contains a negative triangle. The only way that the algorithm fails is if there

²²Without loss of generality, we assume $p > 6$.

²³The factor six is a result of the six possible symmetries that each orientated triangle has.

exist a G_i with $f_\Delta(G_i) = f_\Delta(H)$ and $OPT(G_i) \neq OPT(H)$. By using the same argument as above, we obtain that the probability of this happening is at most $\frac{3k}{p} \leq k^{-1}n^{-\log(n)}$. Thus, our success probability is at least $1 - k^{-1}n^{-\log(n)}$ and there w.h.p with respect to the query size n .

Reconstruction: Assume that for each graph $G_i \in \mathcal{S}$ containing a negative triangle, we are provided with a multiset of edge-weights $W_i := \{w_1^{(i)}, w_2^{(i)}, w_3^{(i)}\}$ that form such a triangle. Suppose the algorithm is queried on a graph H such that $P_\Delta(H) \equiv P_\Delta(G_i)$ for some $G_i \in \mathcal{S}$. By Lemma 6.5, H must also contain a triangle with the identical multiset of edge-weights W_i ; i.e., such a triangle has total weight $w_1^{(i)} + w_2^{(i)} + w_3^{(i)} < 0$.

To find this specific negative triangle in H , we construct an auxiliary tripartite graph H' with vertex sets V_1, V_2, V_3 , where each V_j is a copy of $V(H)$. For every vertex $v \in V(H)$, let v_j denote its copy in V_j . For every edge $\{u, v\}$ in H with weight $w_1^{(i)}$, we add an undirected unweighted edge $\{u_1, v_1\}$ to H' . Similarly, for every edge $\{u, v\}$ in H with weight $w_2^{(i)}$, we add an undirected unweighted edge $\{u_2, v_2\}$ to H' , and for every edge with weight $w_3^{(i)}$, we add an undirected unweighted edge $\{u_3, v_3\}$ to H' . Observe that any triangle in H' must consist of one vertex from each partition. Thus, an unweighted triangle in H' corresponds to a triangle in H with the desired edge-weights. Since triangle detection in unweighted graphs can be performed in $O(n^\omega)$ time [Ita77], we can find the negative triangle in H in $O(n^\omega)$ total time. $\square$

6.2 Constrained Spanning Tree

Definition 6.6 (Constrained Spanning Tree (CST) Problem [AAN82, RG96]). Let $G = (V, E, c, d, B)$ be an undirected graph with two non-negative edge-weight functions

$$c : E \rightarrow \mathbb{R}_{\geq 0} \quad \text{and} \quad d : E \rightarrow \mathbb{R}_{\geq 0},$$

and let $B \in \mathbb{R}_{\geq 0}$ be a budget. The *Constrained Spanning Tree Problem* asks to find a spanning tree $T \subseteq E$ that

$$\begin{aligned} &\text{minimizes} \quad c(T) := \sum_{e \in T} c(e) \\ &\text{subject to} \quad \sum_{e \in T} d(e) \leq B. \end{aligned}$$

A spanning tree satisfying the constraint is called *budget-feasible*. The optimal value is $OPT(G)$ for the optimal value of a problem instance G . If no spanning-tree satisfies the budget constraint then $OPT(G) = \infty$.

Two problem instances $G = (V, E, c, d, B)$ and $G' = (V', E', c', d', B')$ of CST are isomorphic if $B = B'$ their underlying graphs are isomorphic. Note that an isomorphism has to preserve vertex attributes and edge attributes. Therefore, CST is closed under isomorphism.

Note that each edge $e \in E$ is associated with two different attributes $c(e)$ and $d(e)$. We combine these two attributes and say that $w(e) := \alpha_G^E(e) = (c(e), d(e))$ is the weight of the edge e . Note that isomorphism respect weights.

Like in the CSP case, we need to develop a method to find a spanning with certain weight constraints via polynomials. To this end, we associate a formal variable x_w with every distinct edge weight w . The *spanning tree polynomial* of G is defined as

$$P_{ST}(G) := \sum_{w_1 \leq \dots \leq w_{n-1}} c_{w_1, \dots, w_{n-1}}(G) \prod_{i=1}^{n-1} x_{w_i}$$

where $c_{w_1, \dots, w_{n-1}}(G)$ is the number of spanning trees with the weight multiset $\{w_1, \dots, w_{n-1}\}$. Note that this polynomial is closely related to the *Kirchhoff polynomial* (See [CY00]).

Lemma 6.7. *Let G and H be two graphs. If $P_{ST}(G) \equiv P_{ST}(H)$ then for any target B , G contains a spanning tree of total weight B if and only if H contains a spanning tree of total weight B .*

Proof. If $P_{ST}(G) \equiv P_{ST}(H)$ then the coefficients of every corresponding monomial match exactly, i.e. $c_{w_1, \dots, w_{n-1}}(G) = c_{w_1, \dots, w_{n-1}}(H)$ for all combinations of weights. Consequently, G contains a spanning tree that uses weights $w_1, \dots, w_{n-1}$ if and only if H contains the same number spanning tree. $\square$

Proof of Theorem 6.3. Let $\mathcal{S} := \{G_i = (V_i, E_i, c_i, d_i, B_i)\}_{i=1 \dots k}$ be a database of prior instances and assume that for each $i \in [k]$ we are given OPT_i that indicates the optimal value of CST.

Preprocessing: Given $\mathcal{S}$, we first compute a prime number $p \geq k^2 n^{\log(n)+1}$ and construct a finite field $\mathbb{F}_p$. Note that all basic operations in $\mathbb{F}_p$ can be performed in time $O(\log p) = \tilde{O}(1)$. Next, we assign arbitrarily a unique non-zero element in $\mathbb{F}_p$ to each edge-weight of the graphs G_i and store the edge-weights together with the corresponding field elements in a binary search tree. Remember that the edge weight $w(e)$ of an edge $e \in E_i$ is defined as the tuple $(c_i(e), d_i(e))$. Formally, we construct a function $\text{enc}_{\mathcal{S}}: \mathbb{R}^2 \rightarrow \mathbb{F}_p \cup \{\perp\}$ that, on input x , either outputs a unique element in $\mathbb{F}_p$ if x is the weight of some graph in $\mathcal{S}$, or returns $\perp$ otherwise. Note that $\text{enc}_{\mathcal{S}}$ can be computed in time $O(k \cdot n^2 \cdot \log p)$, and evaluating $\text{enc}_{\mathcal{S}}$ on some x takes time $O(\log(kn^2) \cdot \log p) = \tilde{O}(1)$.

Given a graph G_i , we write $\text{enc}_{\mathcal{S}}(G_i)$ for graph where each edge weight w was replaced with $\text{enc}_{\mathcal{S}}(w)$. This can be done in time $\tilde{O}(kn^2)$.

We construct a $(k \cdot n^2)$ -wise independent family of hash functions $h: \mathbb{F}_p \rightarrow \mathbb{F}_p$ using [Vad12, Construction 3.32]. Concretely, we sample a random polynomial over $\mathbb{F}_p$ of degree at most $k \cdot n^2 - 1$. Note that such a hash function can be constructed in time $O(k \cdot n^2)$. Given an edge-weight w , we define $r(w) := h(\text{enc}_{\mathcal{S}}(w))$. For a graph G_i , we write $r(G_i)$ for the graph obtained by replacing each edge weight w with $r(w)$.

Let $x_1, \dots, x_m \in \mathbb{F}_p$, where $m \leq k \cdot n^2$, be a sequence of values. Using the Fast Fourier Transformation, we can evaluate $h(x_i)$ for all i in time $\tilde{O}(k \cdot n^2)$. Thus, we can compute $\tilde{G}_1 := r(G_1), \dots, \tilde{G}_k := r(G_k)$ in time $\tilde{O}(kn^2)$. To be more specific, let $w_1, \dots, w_m \in \mathbb{R}^2$ be an enumeration of all edge-weights of graphs in $\mathcal{S}$. Note that $m \leq k \cdot n^2$. We compute the vector $\mathbf{r} = (r_1 := r(w_1), \dots, r_m := r(w_m))$ in time $\tilde{O}(kn^2)$. Further, we store each r_i with the corresponding w_i into a binary data structure $R_{\mathcal{S}}: \mathbb{R}^2 \rightarrow \mathbb{F}_p$. Note that we can evaluate $R_{\mathcal{S}}(w)$ in time $O(\log(kn^2)) = \tilde{O}(1)$. We write $L^{\tilde{G}_i}$ for the Laplacian matrix of $\tilde{G}_i$. Further, for each $j \in [n]$,²⁴ we write $\tilde{L}_j^{\tilde{G}_i}$ for the matrix that is obtained by removing the j -th row and the j -th column from $L^{\tilde{G}_i}$. Note that $\tilde{L}_j^{\tilde{G}_i}$ is a matrix over $\mathbb{F}_p$.

Next, we define $f_{ST}(G) := \det(\tilde{L}_1^{\tilde{G}_i})$ which can be computed in time $\tilde{O}(n^\omega)$. For each $i \in [k]$, we compute $f_{\Delta}(G_i)$ and store the value together with OPT_i in a linear data structure. Formally, we define a data structure $D_{\mathcal{S}}$ that on input x either returns $D_{\mathcal{S}}(x) = OPT_i$ for a graph G_i with $f_{\Delta}(G_i) = x$ or returns that no such graph exist. $D_{\mathcal{S}}$ can be constructed in time $\tilde{O}(k)$ and can be queried in time $\tilde{O}(1)$. Hence, the preprocessing phase takes time $\tilde{O}(kn^\omega)$.

Query: Given an input graph H , we use $\text{enc}_{\mathcal{S}}$ to check that all weights of H correspond to weights of matrices in $\mathcal{S}$. If this is not the case then our algorithm claims that H is not isomorphic to any graph in $\mathcal{S}$. Next, we use the data structure $R_{\mathcal{S}}$ to compute $\tilde{H} := r(H)$ in time $\tilde{O}(n^2)$. We then compute $f_{ST}(H)$ in time $\tilde{O}(n^\omega)$. Lastly, we check if $x := f_{\Delta}(H)$ is in $D_{\mathcal{S}}$. If this is the case then we return $D_{\mathcal{S}}(x)$, otherwise we report that H is not isomorphic to any graph in $\mathcal{S}$. This can be done in time $\tilde{O}(1)$. Hence, the whole query time is in $\tilde{O}(n^\omega)$.

²⁴For simplicity, we assume that G_i has n vertices which is why L^{G_i} is a $n \times n$ -matrix.

Correctness: As a first step, we show that

$$f_{ST}(G) = \det(\tilde{L}_1^{\tilde{G}_i}) = \sum_{w_1 \leq \dots \leq w_{n-1}} c_{w_1, \dots, w_{n-1}}(G) \prod_{i=1}^{n-1} r(w_i) = P_{ST}(G)(\mathbf{r}). \quad (2)$$

Here $P_{ST}(G)(\mathbf{r})$ denotes the evaluation of $P_{ST}(G)$ obtained by assigning the value $r(w_i)$ to each variable x_{w_i} . A standard result (e.g. [CY00, Page 3]) states that for any $j \in [n]$

$$\det(\tilde{L}_j^{\tilde{G}_i}) = \sum_T \prod_{w \in T} r(w),$$

where the sum is over all spanning trees T of G and $\prod_{w \in T}$ is the product over all edge-weights in T . By combining spanning trees with the same edge-weights, we obtain Equation (2).

Let H be a graph on which we are queried. The graph H is isomorphic to some $G_i \in \mathcal{S}$ only if all edge-weights of H appear as edge-weights in $\mathcal{S}$. If this is not the case, then our query algorithm rejects. Thus, we assume that all edge-weights in H appear in the list $w_1, \dots, w_m$.

We consider two cases. First, suppose that H is isomorphic to some G_i . Then, $f_{ST}(G_i) = f_{ST}(H)$ due to Equation (2) and the fact that $P_{ST}(G_i) \equiv P_{ST}(H)$ since $H \cong G_i$. Thus, $D_{\mathcal{S}}(f_{ST}(H))$ returns OPT_i provided there is no other graph G_j such that $f_{ST}(G_i) = f_{ST}(G_j)$ and $OPT_i \neq OPT_j$. By Lemma 6.5, this only occurs if $P_{ST}(G_i) \not\equiv P_{ST}(G_j)$ but $P_{ST}(G_i)(\mathbf{r}) = P_{ST}(G_j)(\mathbf{r})$. Since all r_i are chosen independently and uniformly at random over $\mathbb{F}_p$, Lemma 6.1 yields that the event $P_{ST}(G_i) \not\equiv P_{ST}(G_j)$ and $P_{ST}(G_i)(\mathbf{r}) = P_{ST}(G_j)(\mathbf{r})$ has probability at most $\frac{n-1}{p}$ for every fixed index $j \neq i$. Using union bound, we obtain that the failure probability is at most $\frac{(n-1)k}{p} \leq k^{-1}n^{-\log(n)}$. Thus, our success probability is at least $1 - k^{-1}n^{-\log(n)}$ and there w.h.p with respect to the query size n .

Second, suppose H is non-isomorphic to every $G_i \in \mathcal{S}$ and let OPT_H denote the value of the optimal spanning tree that satisfies the budget constraint. The only way that the algorithm fails is if there exist a G_i with $f_{ST}(G_i) = f_{ST}(H)$ and $OPT_i \neq OPT_H$. By using the same argument as above, we obtain that the probability of this happening is at most $\frac{(n-1)k}{p} \leq k^{-1}n^{-\log(n)}$. Thus, our success probability is at least $1 - k^{-1}n^{-\log(n)}$ and there w.h.p with respect to the query size n .

Reconstruction: Assume that each graph G_i is equipped with a multiset of cost-distance pairs $W_i := \{w_1^{(i)}, \dots, w_{n-1}^{(i)}\}$ corresponding to an optimal spanning tree in G_i . Suppose the algorithm is queried on a graph H such that $P_{ST}(H) \equiv P_{ST}(G_i)$ for some $G_i \in \mathcal{S}$. By Lemma 6.7, H must contain a spanning tree with the same multiset of cost-distance pairs W_i . Upon retrieving W_i , let $k_{c,d}$ denote the multiplicity of the pair (c, d) in W_i . We formulate the reconstruction of an optimal tree in H as a matroid intersection problem over the ground set $E(H)$ between: (1) a graphic matroid $\mathcal{M}_G$ (where independent sets are forests), and (2) a partition matroid $\mathcal{M}_P$ (where independent sets contain at most $k_{c,d}$ edges of type (c, d)). Consequently, the desired spanning tree in H can be computed in polynomial time using classical matroid intersection algorithms [Sch03], and specifically in $\tilde{O}(m\sqrt{n})$ time via recent advancements [BMNT23, CLS⁺19, GS85].

□

7 Algorithms via 1-WL (Elementary Arguments)

In this section, we show that Constrained Shortest Path and Energy Game can be solved in near-linear time in the Isomorphic-Priors model using the 1-dimensional Weisfeiler-Leman (1-WL) algorithm. We start by formally defining both problems and stating the main algorithmic results for both problems. After that, we give an introduction to the 1-WL algorithm and prove our algorithmic results.

7.1 Constrained Shortest Path and Energy Game

Definition 7.1 (Constrained Shortest Path (CSP)). An instance of Constrained Shortest Path is defined by a tuple $G = (V, E, c, \ell, s, t, B)$, where:

- (V, E) is a finite directed graph.
- $c : E \rightarrow \mathbb{R}_{\geq 0}$ is a cost function assigning a non-negative cost to each edge.
- $\ell : E \rightarrow \mathbb{R}_{\geq 0}$ is a length (or delay) function assigning a non-negative length to each edge.
- $s, t \in V$ are the distinguished source and target vertices.
- $B \in \mathbb{R}_{\geq 0}$ is the maximum allowable path cost constraint.

For a directed path $P = (e_1, \dots, e_r)$ from s to t , the total cost is $c(P) = \sum_{i=1}^r c(e_i)$ and the total length is $\ell(P) = \sum_{i=1}^r \ell(e_i)$. The objective is to find a feasible path P^* minimizing $c(P^*)$ subject to $\ell(P^*) \leq B$. The optimal value is $OPT(G) = c(P^*)$. If no path satisfies the length constraint, $OPT(G) = \infty$.

Two problem instances $G = (V, E, c, \ell, s, t, B)$ and $G' = (V', E', c', \ell', s', t', B')$ of CSP are isomorphic if $B = B'$ and their underlying graphs are isomorphic. Note that an isomorphism has to preserve vertex attributes and edge attributes (e.g., s maps s' , t maps t' , etc.).

Naturally, CSP with isomorphic priors is formalized as follows. In the preprocessing phase, we are given a database $\mathcal{S} = \{G_1, \dots, G_k\}$ of k prior instances. Each instance is given as a tuple $G_i = (V_i, E_i, c_i, \ell_i, s_i, t_i, B_i)$, provided together with its optimal value $OPT(G_i)$. In the query phase, we receive a query graph $G = (V, E, c, \ell, s, t, B)$.

Theorem 7.2 (CSP in the Isomorphic-Priors Model). *There is an algorithm that solves the CSP problem with k prior instances each having at most n vertices and m edges in $O(k(n+m) \log n)$ preprocesses time and $O((n+m) \log n)$ query time. Additionally, if each prior G_i is equipped with an optimal path, then the optimal path for the query graph can be found in $O((n+m) \log n)$ query time.*

Definition 7.3 (Energy Game). An Energy Game instance is defined by a tuple $G = (V, E, V_1, V_2, w, v_0)$, where:

- (V, E) is a finite directed graph with no terminal states.
- V_1, V_2 is a partition of V ($V_1 \cup V_2 = V$, $V_1 \cap V_2 = \emptyset$), where Player 1 controls V_1 and Player 2 controls V_2 .
- $w : E \rightarrow \mathbb{Z}$ is a weight function assigning an integer energy change to each edge.
- $v_0 \in V$ is the designated initial vertex.

A play $\pi = v_0 v_1 v_2 \dots$ is formed by moving a token: at step $k \geq 0$, if $v_k \in V_i$, Player i chooses an outgoing edge $(v_k, v_{k+1}) \in E$. Given an initial energy $E_0 \in \mathbb{N}$, the energy evolves via $E_{k+1} = E_k + w(v_k, v_{k+1})$. A strategy for Player 1 is winning from v_0 with initial energy E_0 if it ensures that $E_k \geq 0$ for all $k \geq 0$, regardless of the choices of Player 2. The objective is to find the minimum initial energy $E_0^* \in \mathbb{N}$ for which Player 1 has a winning strategy from v_0 . The optimal value is $OPT(G) = E_0^*$. If no finite initial energy allows Player 1 to win, we set $OPT(G) = \infty$.

Two Energy Game instances G and G' are isomorphic if their underlying graphs are isomorphic. Note that an isomorphism has to preserve vertex attributes and edge attributes. Therefore, Energy Game is closed under isomorphism.

Theorem 7.4 (Energy Game in the Isomorphic-Priors Model). *There is an algorithm that solves Energy Game with k prior instances each having at most n vertices and m edges in $O(k(n+m)\log n)$ preprocesses time and $O((n+m)\log n)$ query time. Additionally, if each prior G_i , that is a winning instance, is equipped with a memoryless winning strategy σ_i for Player 1 from every starting vertex, then the algorithm can also compute a memoryless optimal winning strategy σ_H with the same guarantees for H in $O((n+m)\log n)$ time.*

7.2 Preliminaries: The Weisfeiler–Leman Algorithm

To solve constrained reachability and game dynamics, we model our problem instances as attributed directed graphs. Let $G = (V, E, \alpha_V, \alpha_E)$ be a directed graph where $\alpha_E: E \rightarrow \Sigma_E$ assigns an *attribute* to each edge (e.g., costs, lengths, or weights), and $\alpha_V: V \rightarrow \Sigma_V$ assigns an *attribute* to each vertex (e.g. source, sink, or what player a vertex belongs to).

When initializing the *1-dimensional Weisfeiler–Leman (1-WL) algorithm*, the vertex attributes are directly translated into an initial vertex *coloring*, denoted by $\chi_0^G: V \rightarrow \mathcal{C}$, such that $\chi_0^G(u) = \chi_0^G(v)$ if and only if $\alpha_V(u) = \alpha_V(v)$. The 1-WL algorithm then iteratively *refines* these colors (e.g. by recolor some vertices of color c with a new color d), ultimately returning a *stable color* for every vertex u , denoted by $\chi_\infty^G(u)$, which possesses the structural properties detailed in Lemma 7.6.

The 1-WL Algorithm: A Primer. The 1-WL algorithm can be intuitively understood as an iterative message-passing process. Initially, every vertex only knows its own starting color. In each round, every vertex “broadcasts” its current color to its neighbors. Simultaneously, it listens to the incoming messages, tallying up exactly how many neighbors of each color it has, grouped by the attributes and directions of the connecting edges. It then bundles its own current color with this exact tally to form a unique “signature”. If two vertices that previously shared the same color generate different signatures (meaning their local neighborhoods differ), they are split into two different new colors. This process continues until no pair of vertices split into two different new colors. The final color is called the *stable color*.

Formally, let $G = (V, E, \alpha_V, \alpha_E)$ be an attributed directed graph. For two colorings $\chi, \chi': V \rightarrow \mathcal{C}$, we write $\chi \preceq \chi'$ if $\chi'(v) = \chi'(w)$ for all $v, w \in V$ such that $\chi(v) = \chi(w)$ (in other words, the partition into color classes of χ refines the corresponding partition for χ'). We write $\chi \equiv \chi'$ if $\chi \preceq \chi'$ and $\chi' \preceq \chi$. The initial coloring computed by 1-WL is defined via $\chi_0^G(v) = \alpha_V(v)$ for all $v \in V(G)$. For $i \geq 0$ the coloring is iteratively refined by setting

$$\chi_{i+1}^G(v) := \left(\chi_i^G(v), \left\{ \left\{ (\chi_i^G(u), \alpha_E(u, v)) \mid u \in N_{in}(v) \right\}, \left\{ (\chi_i^G(w), \alpha_E(v, w)) \mid w \in N_{out}(v) \right\} \right\} \right),$$

where $\{\dots\}$ denotes a multiset. This refinement stabilizes when the partition of V induced by the colors stops changing. More concretely, since $\chi_{i+1}^G \preceq \chi_i^G$ for every $i \geq 0$, there is some minimal $i_\infty \geq 0$ such that $\chi_{i_\infty}^G \equiv \chi_{i_\infty+1}^G$, and we set $\chi_\infty^G := \chi_{i_\infty+1}^G$. Note that we perform one extra iteration after stabilization, which is necessary for some technical reasons.

The 1-WL algorithm can be used as a simple heuristic to test isomorphisms of graphs. If some color appears a different number of times in the stable coloring of the two graphs, then they cannot be isomorphic (the converse is not true in general).

Definition 7.5 (1-WL Equivalence). Two (attributed) graphs G and H are *1-WL equivalent* ($G \equiv_{1\text{-WL}} H$) if $\{\{\chi_\infty^G(v) \mid v \in V(G)\}\} = \{\{\chi_\infty^H(w) \mid w \in V(H)\}\}$.

Properties of 1-WL. Below are some basic facts about 1-WL.

Lemma 7.6 (1-WL Properties; Folklore). *For every two (attributed) graphs $G = (V_G, E_G, \alpha_G^V, \alpha_G^E)$ and $H = (V_H, E_H, \alpha_H^V, \alpha_H^E)$ such that $G \equiv_{1\text{-WL}} H$, the following holds:*

1. **Refinement:** *For every $v \in V_G$ and $v' \in V_H$, if $\chi_\infty^G(v) = \chi_\infty^H(v')$, then their initial colors match: $\alpha_G^V(v) = \alpha_H^V(v')$.*
2. **Neighborhood Preservation:** *For every $v \in V_G$ and $v' \in V_H$, if $\chi_\infty^G(v) = \chi_\infty^H(v')$, then for every stable color C and edge attribute ℓ , there exists an incoming edge $(u, v) \in E_G$ in G such that $\alpha_G^E(u, v) = \ell$ and $\chi_\infty^G(u) = C$ if and only if there exists an incoming edge $(u', v') \in E_H$ in H such that $\alpha_H^E(u', v') = \ell$ and $\chi_\infty^H(u') = C$. The analogous bi-conditional holds for outgoing edges.*

Remark 7.7. The 1-WL algorithm actually guarantees stronger properties than the above (see Lemma A.10). However, the properties above suffice to get results in this section.

Proof Sketch of Lemma 7.6. Let $v \in V_G$ and $v' \in V_H$ such that $\chi_\infty^G(v) = \chi_\infty^H(v')$. Also let $i_\infty \geq 0$ be minimal such that $\chi_{i_\infty}^G \equiv \chi_{i_\infty+1}^G$. So $\chi_\infty^G \equiv \chi_{i_\infty+1}^G$ and, since $G \equiv_{1\text{-WL}} H$ and $\chi_\infty^G(v) = \chi_\infty^H(v')$, we also get that $\chi_\infty^H \equiv \chi_{i_\infty+1}^H$. By definition, it holds for all $i \geq 0$ that, if $\chi_{i+1}^G(v) = \chi_{i+1}^H(v')$, then $\chi_i^G(v) = \chi_i^H(v')$. So the first part follows by induction.

To prove Neighborhood Preservation, consider the coloring $\chi^*: V(G) \uplus V(H) \rightarrow \mathcal{C}$ defined on the disjoint union of G and H via $\chi^*(w) := \chi_\infty^G(w)$ for all $w \in V(G)$, and $\chi^*(w') := \chi_\infty^H(w')$ for all $w' \in V(H)$. Since $G \equiv_{1\text{-WL}} H$, it can be verified that the coloring χ^* is stable for the disjoint union of G and H , and in fact $\chi_\infty^{G \uplus H} \equiv \chi^*$ (since all refinements being made on G or H also happen on $G \uplus H$). In particular, we get that $\chi_\infty^{G \uplus H}(v) = \chi_\infty^{G \uplus H}(v')$. By definition, we conclude that

$$\left\{ \left(\chi_\infty^{G \uplus H}(u), \alpha_G^E(u, v) \right) \mid u \in N_{in}(v) \right\} = \left\{ \left(\chi_\infty^{G \uplus H}(u'), \alpha_H^E(u', v') \right) \mid u' \in N_{in}(v') \right\}.$$

In particular, a pair (C, ℓ) appears in the first set if and only if it appears in the second set. $\square$

By repeatedly applying Lemma 7.6 (Neighborhood Preservation), every sequence of valid steps in G can be identically mirrored step-by-step in H . This sequential matching gives rise to the Walk Lifting property.

Lemma 7.8 (Walk Lifting). *Let $G = (V_G, E_G, \alpha_G^V, \alpha_G^E)$ and $H = (V_H, E_H, \alpha_H^V, \alpha_H^E)$ be (attributed) graphs such that $G \equiv_{1\text{-WL}} H$. Also let $v_0 \in V_G$ and $v'_0 \in V_H$ such that $\chi_\infty^G(v_0) = \chi_\infty^H(v'_0)$. For every directed walk $W = (v_0, e_1, v_1, \dots, e_r, v_r)$ in G , there exists a corresponding directed walk $W' = (v'_0, e'_1, v'_1, \dots, e'_r, v'_r)$ in H such that $\chi_\infty^G(v_i) = \chi_\infty^H(v'_i)$ for all $0 \leq i \leq r$, and $\alpha_G^E(e_j) = \alpha_H^E(e'_j)$ for all $1 \leq j \leq r$.*

Proof. We proceed by induction on the length of the walk r .

In the case case $r = 0$, the walk W consists of a single vertex v_0 . We set W' to be the walk consisting only of v'_0 . By the lemma's hypothesis, $\chi_\infty^G(v_0) = \chi_\infty^H(v'_0)$, so the condition holds trivially.

So assume the claim holds for walks of length $k < r$. Let $W_k = (v_0, \dots, e_k, v_k)$ be the prefix of length k in G , which by the inductive hypothesis lifts to a valid walk $W'_k = (v'_0, \dots, e'_k, v'_k)$ in H where $\chi_\infty^G(v_k) = \chi_\infty^H(v'_k)$.

The next step in W traverses the edge $e_{k+1} = (v_k, v_{k+1})$ with some attribute $\ell = \alpha_G^E(e_{k+1})$. Let $C = \chi_\infty^G(v_{k+1})$ be the stable color of the destination vertex. By Lemma 7.6 (Neighborhood Preservation), since $\chi_\infty^G(v_k) = \chi_\infty^H(v'_k)$, the existence of an outgoing edge from v_k to a vertex of color C with attribute ℓ guarantees the existence of an outgoing edge $e'_{k+1} = (v'_k, v'_{k+1})$ in H with attribute $\alpha_H^E(e'_{k+1}) = \ell$ such that $\chi_\infty^H(v'_{k+1}) = C$. Appending this edge e'_{k+1} and vertex v'_{k+1} to W'_k yields a valid walk of length $k + 1$ satisfying all conditions. $\square$

Runtime. The straightforward mathematical definition of 1-WL constructs nested multisets as colors, leading to exponentially growing color representations (as strings) that are highly inefficient to compute and store. However, by replacing these unwieldy nested tuples with canonical integer identifiers at each iterative step, one obtains the near-linear runtimes required for our analysis [CC82, PT87, BBG17]. More precisely, we rely on the following result.

Theorem 7.9 ([BBG17]). *There is an algorithm, that given an edge-labeled graph G , outputs a vertex-coloring $\chi^G: V(G) \rightarrow [|V(G)|]$ in time*

$$O((|V(G)| + |E(G)|) \log |V(G)|)$$

such that the following properties hold:

1. *For every graph G it holds that $\chi^G \equiv \chi_\infty^G$.*
2. *For every two graphs G, H such that $G \equiv_{1\text{-WL}} H$ it holds that $(G, \chi^G) \equiv_{1\text{-WL}} (H, \chi^H)$.*
3. *For every two isomorphic graphs G, H and every isomorphism $\varphi: G \cong H$ it holds that $\chi^G(v) = \chi^H(\varphi(v))$ for all $v \in V(G)$.*

Note that the output coloring can effectively be treated as the 1-WL coloring. One key difference is that, although $\chi_\infty^G(v) \neq \chi_\infty^H(w)$, it may be that $\chi^G(v) = \chi^H(w)$ since the exponentially growing color representations are compressed into a set of size n . However, this usually does not cause any problems.

The coloring from Theorem 7.9 can be used to obtain a *complete invariant* for 1-WL equivalence (see, e.g., [Ott17]). For the Constrained Shortest Path and Energy Game (and other problems; see Appendix A), this gives a Invariant and Solution-Preserving Signature.

Theorem 7.10. *There is a deterministic algorithm that, given an attributed graph G , outputs a string $\text{WL}_1(G) \in \{0, 1\}^*$ in time $O((|V(G)| + |E(G)|) \log |V(G)|)$, such that for every two input graphs G, H it holds that $G \equiv_{1\text{-WL}} H$ if and only if $\text{WL}_1(G) = \text{WL}_1(H)$.*

Proof Sketch. On the input $G = (V, E, \alpha_V, \alpha_E)$, we can compute the coloring χ^G via Theorem 7.9. Now, we build the string $\text{WL}_1(G) \in \{0, 1\}^*$ to encode the following information:

- For each color a in the image χ^G , the size of $(\chi^G)^{-1}(a)$ and $\alpha_V(v)$ for some (and thus for all) $v \in (\chi^G)^{-1}(a)$ (in increasing order of colors $a \in [|V(G)|]$).
- For each pair of colors a, a' in the image χ^G , and each edge attribute ℓ in the image of α_E , the number of edges from $(\chi^G)^{-1}(a)$ to $(\chi^G)^{-1}(a')$ with edge attribute ℓ . If this number is zero, then we do not explicitly encode this information in the string.

Note that the resulting string has length at most $O((|V(G)| + |E(G)|) \log |V(G)|)$, and can be computed in the same time. It is easily seen that $G \equiv_{1\text{-WL}} H$ if and only if $\text{WL}_1(G) = \text{WL}_1(H)$. $\square$

7.3 1-WL Equivalence Implies Equal Optimality

Now, let us return to Constrained Shortest Path and Energy Game. In order to prove Theorems 7.2 and 7.4 we argue that 1-WL equivalent instances achieve the same optimal value. As a result, it suffices to compute complete invariants for 1-WL for all priors, and store optimal solutions for them.

Theorem 7.11. *Let*

$$\begin{aligned} G &= (V, E, c : E \rightarrow \mathbb{R}_{\geq 0}, \ell : E \rightarrow \mathbb{R}_{\geq 0}, s, t, B) \quad \text{and} \\ G' &= (V', E', c' : E' \rightarrow \mathbb{R}_{\geq 0}, \ell' : E' \rightarrow \mathbb{R}_{\geq 0}, s', t', B) \end{aligned}$$

be two instances of the Constrained Shortest Path (CSP) problem. If $G \equiv_{1\text{-WL}} G'$, then they have the same optimal value for CSP.

Before turning to the proof, let us recall how instances $G = (V, E, c : E \rightarrow \mathbb{R}_{\geq 0}, \ell : E \rightarrow \mathbb{R}_{\geq 0}, s, t, B)$ of CSP are interpreted as attributed graphs. The attribute of each edge $e \in E$ with length $\ell(e)$ and cost $c(e)$ is the vector $\alpha_E(e) = (\ell(e), c(e))$. The vertex attributes designate the terminals: $\alpha_V(v) = s$ if v is the source, $\alpha_V(v) = t$ if v is the sink, and $\alpha_V(v) = \perp$ otherwise.

Proof. Let $OPT(G)$ and $OPT(G')$ denote the optimal values. By symmetry, it suffices to show $OPT(G') \leq OPT(G)$. If no feasible path exists in G , then $OPT(G) = \infty$ and the inequality trivially holds.

Assume a feasible optimal path exists, and let $P = (v_0, e_1, v_1, \dots, e_r, v_r)$ be an optimal directed path in G from $v_0 = s$ to $v_r = t$ with total cost $C \geq 0$ and length $L \leq B$. Recall that each edge e_j carries the attribute $\alpha_E(e_j) = (c(e_j), \ell(e_j))$.

Because $G \equiv_{1\text{-WL}} G'$ and α_V isolates the distinguished vertices into singleton color classes, we obtain that $\chi_\infty^G(s)$ and $\chi_\infty^{G'}(s')$. By Lemma 7.8 (Walk Lifting), there exists a directed walk $W' = (v'_0, e'_1, v'_1, \dots, e'_r, v'_r)$ in G' starting at s' with the exact same sequence of edge attributes (c, ℓ) and stable vertex colors. In particular, $\chi_\infty^G(t) = \chi_\infty^G(v_r) = \chi_\infty^{G'}(v'_r)$. Since t and t' are distinguished via α_V and α'_V , we obtain that $v'_r = t'$ by Lemma 7.6.

The total cost and length of walk W' are identical to P , meaning W' is a feasible solution in G' with cost C . If W' contains cycles, they can be removed to form a simple directed path P' . Given that the edge lengths and costs are non-negative, removing cycles cannot increase the total cost or violate the length constraint. Thus, $OPT(G') \leq C = OPT(G)$. $\square$

Theorem 7.12. *Let $G = (V, E, V_1, V_2, w, v_0)$ and $G' = (V', E', V'_1, V'_2, w', v'_0)$ be two Energy Game instances. If $G \equiv_{1\text{-WL}} G'$, then Player 1 has a winning strategy in G from v_0 with initial energy E_0 if and only if Player 1 has a winning strategy in G' from v'_0 with initial energy E_0 .*

Again, let us first recall how instances $G = (V, E, V_1, V_2, w, v_0)$ are interpreted as attributed graphs. The attribute of each edge $e \in E$ is $\alpha_E(e) = w(e)$. The vertex attributes indicate the partition according to players and the initial vertex: $\alpha_V(v) = (i, s)$ if v is the initial vertex, and $\alpha_V(v) = (i, \perp)$ otherwise, where in both cases $i \in \{1, 2\}$ is the unique index such that $v \in V_i$.

Proof. By symmetry, it suffices to show that if Player 1 has a winning strategy σ in G starting at v_0 with energy E_0 , they also have a winning strategy σ' in G' starting at v'_0 with energy E_0 .

We define σ' dynamically by maintaining a simulated “shadow play” in G . Let the current finite prefix of the play in G' be $\pi' = (v'_0, \dots, v'_k)$ and the maintained shadow play in G be $\pi = (v_0, \dots, v_k)$. We enforce the invariant that $\chi_\infty^G(v_i) = \chi_\infty^{G'}(v'_i)$ and $w(v_{i-1}, v_i) = w'(v'_{i-1}, v'_i)$ for all $i \leq k$.

1. **Base Case:** For $k = 0$, the play begins at v'_0 . We set the shadow play to v_0 . Since $G \equiv_{1\text{-WL}} G'$, we conclude that $\chi_\infty(v'_0) = \chi_\infty(v_0)$ using Lemma 7.6 (Refinement).
2. **Player 1’s Turn:** Suppose the current play is at $v'_k \in V'_1$. Because colors strictly preserve initial partitions (by Lemma 7.6, Refinement), the shadow vertex v_k satisfies $v_k \in V_1$. In G , strategy σ dictates an outgoing edge (v_k, v_{k+1}) with weight W . By Lemma 7.6 (Neighborhood Preservation), since $\chi_\infty^{G'}(v'_k) = \chi_\infty^G(v_k)$, there exists an outgoing edge in G' to some vertex v'_{k+1} such that $\chi_\infty^{G'}(v'_{k+1}) = \chi_\infty^G(v_{k+1})$ and $w'(v'_k, v'_{k+1}) = W$. Player 1’s strategy σ' chooses this edge, and we append v_{k+1} to the shadow play.

3. **Player 2's Turn:** Suppose the current play is at $v'_k \in V'_2$, meaning the shadow vertex v_k satisfies $v_k \in V_2$. Player 2 chooses an arbitrary edge (v'_k, v'_{k+1}) with weight W . By Lemma 7.6 (Neighborhood Preservation), since $\chi_\infty^G(v_k) = \chi_\infty^{G'}(v'_k)$, there exists an outgoing edge from v_k to some vertex v_{k+1} in G such that $\chi_\infty^G(v_{k+1}) = \chi_\infty^{G'}(v'_{k+1})$ and $w(v_k, v_{k+1}) = W$. We append this v_{k+1} to the shadow play.

Through this induction, every infinite play π' in G' conforming to σ' maps to a valid shadow play π in G conforming to σ . Crucially, the sequence of edge weights traversed in π' is identical to that in π . Because σ is winning in G , the energy level along π satisfies $E_k \geq 0$ for all $k \geq 0$. Since the step-by-step weights match exactly, the energy level in G' also satisfies $E'_k = E_k \geq 0$ for all $k \geq 0$. Thus, σ' is a winning strategy for Player 1 in G' . $\square$

7.4 Proof of Theorems 7.2 and 7.4

Proof of Theorem 7.2. We describe the different phases of the algorithm.

Preprocessing: For each graph $G_i \in \mathcal{S}$, we run the 1-WL algorithm (Theorem 7.9) and compute its complete invariant for 1-WL equivalence (Theorem 7.10). Both subroutines run in time $O((n+m) \log n)$ time. We insert each string $\text{WL}_1(G_i) \in \{0,1\}^*$ into a binary search tree, where the i -th letter determines whether we take the left/right subtree on the i -th level. This takes time proportional to the length of the string $\text{WL}_1(G_i)$, i.e., it can be done in time $O((n+m) \log n)$. For the corresponding node where the key $\text{WL}_1(G_i)$ is saved, we save the OPT_i as well as an optimal solution path (if provided) and the stable coloring χ^{G_i} .

Note that, if two graphs $G_i, G_j \in \mathcal{S}$ yield the same invariant, they are 1-WL equivalent (Theorem 7.10). In particular, by Theorem 7.11, this implies $\text{OPT}_i = \text{OPT}_j$, and no conflicting values can be assigned to a single node of the search tree. Overall, processing all k graphs takes $O(k(n+m) \log n)$ time.

Query: Given the query graph H , we run the 1-WL algorithm (Theorem 7.9) and compute its complete invariant for 1-WL equivalence (Theorem 7.10) in $O((n+m) \log n)$ time. Looking up the computed invariant in the binary search tree takes $O((n+m) \log n)$ time, allowing us to return $\text{OPT}(H) = \text{OPT}_i$ (which is correct by Theorem 7.11). If the string $\text{WL}_1(H)$ is not contained in the search tree, then we can correctly claim that H is not isomorphic to any graph in $\mathcal{S}$ because graphs that are not 1-WL equivalent are not isomorphic. The total query time is $O((n+m) \log n)$.

Reconstruction: If optimal paths are stored, the lookup yields the optimal path $P_i = (v_0, e_1, v_1, \dots, e_r, v_r)$ from the equivalent graph G_i . Because $H \equiv_{1\text{-WL}} G_i$, analogous to the proof of Theorem 7.11, we can inductively construct a corresponding walk in H : if we are currently at $u' \in V(H)$ corresponding to $v_j \in P_i$, we scan the outgoing edges of u' to find an edge e' with the same attributes as e_{j+1} that leads to a vertex v' with the same 1-WL stable color as v_{j+1} . The existence of such an edge is guaranteed by Lemma 7.6 (Neighborhood Preservation). Scanning for the next edge takes time proportional to the out-degree of the current vertex. Over the entire path, this takes at most $O(n+m)$ time. This procedure yields a valid walk W' in H with identical cost and length to P_i . Finally, we remove cycles from W' in $O(n)$ time to produce a simple optimal path P_H . The total query time for the path remains $O((n+m) \log n)$. $\square$

We now turn from the proof of CSP to the proof of Theorem 7.4. The shadow play construction in the proof of Theorem 7.12 may use memory, even when the original strategy is memoryless. To reconstruct memoryless strategies, we therefore use color-consistent strategies, defined as follows.

Definition 7.13 (Color-Consistent Strategy). Let $G = (V, E, V_1, V_2, w, v_0)$ be an Energy Game instance with stable 1-WL coloring χ_∞^G . A memoryless strategy σ of Player 1 is *color-consistent* if,

for all $u, v \in V_1$ with $\chi_\infty^G(u) = \chi_\infty^G(v)$,

$$\chi_\infty^G(\sigma(u)) = \chi_\infty^G(\sigma(v)) \quad \text{and} \quad w(u, \sigma(u)) = w(v, \sigma(v)).$$

The following lemma, whose proof is deferred to Appendix A.1, shows that a memoryless strategy optimal from every vertex can be made color-consistent in linear time, given the stable 1-WL coloring.

Lemma 7.14. *Let $G = (V, E, V_1, V_2, w, v_0)$ be an Energy Game instance with n vertices and m edges. Suppose we are given its stable 1-WL coloring χ_∞^G and a memoryless strategy σ of Player 1 that wins from every vertex with the minimum possible initial energy. Then a color-consistent memoryless strategy $\hat{\sigma}$ with the same optimality property can be computed in $O(n + m)$ time.*

Proof of Theorem 7.4. We describe the different phases of the algorithm.

Preprocessing: For each graph $G_i \in \mathcal{S}$, we run the 1-WL algorithm (Theorem 7.9) and compute its complete invariant for 1-WL equivalence (Theorem 7.10). Both subroutines run in time $O((n + m) \log n)$. We insert each string $\text{WL}_1(G_i) \in \{0, 1\}^*$ into a binary search tree, where the i -th letter determines whether we take the left/right subtree on the i -th level. This takes time proportional to the length of the string $\text{WL}_1(G_i)$, i.e., it can be done in time $O((n + m) \log n)$. For the corresponding node where the key $\text{WL}_1(G_i)$ is saved, we save $\text{WIN}_i = \text{OPT}(G_i)$ and the stable coloring χ^{G_i} . If an optimal memoryless strategy σ_i is provided, we apply Lemma 7.14 to compute and store a color-consistent optimal memoryless strategy $\hat{\sigma}_i$. This additional step takes $O(m + n)$ time.

Note that, if two graphs $G_i, G_j \in \mathcal{S}$ yield the same invariant, they are 1-WL equivalent (Theorem 7.10). In particular, by Theorem 7.12, this guarantees $\text{WIN}_i = \text{WIN}_j$, so no conflicting values can be assigned to a single node of the search tree. Overall, processing all k graphs takes $O(k(n + m) \log n)$ time.

Query: Given the query graph H , we run the 1-WL algorithm (Theorem 7.9) and compute its complete invariant for 1-WL equivalence (Theorem 7.10) in $O((n + m) \log n)$ time. Looking up the computed invariant in the binary search tree takes $O((n + m) \log n)$ time, allowing us to return $\text{OPT}(H) = \text{WIN}_i$ (which is correct by Theorem 7.12). If the string $\text{WL}_1(H)$ is not contained in the search tree, then we can correctly claim that H is not isomorphic to any graph in $\mathcal{S}$ because graphs that are not 1-WL equivalent are not isomorphic. The total query time is $O((n + m) \log n)$.

Reconstruction: Suppose $\text{OPT}(G_i) < \infty$ and a color-consistent optimal memoryless strategy $\hat{\sigma}_i$ is stored. For each Player 1 color class C in G_i , choose a representative u_C and set $v_C = \hat{\sigma}_i(u_C)$. For every Player 1 vertex $u' \in V_H$ with $\chi_\infty^H(u') = C$, choose a successor v' satisfying

$$w_H(u', v') = w_i(u_C, v_C) \quad \text{and} \quad \chi_\infty^H(v') = \chi_\infty^{G_i}(v_C),$$

and define $\sigma_H(u') = v'$. Such a successor exists by Lemma 7.6 (Neighborhood Preservation). Selecting representatives and scanning the outgoing edges of Player 1 vertices takes $O(n + m)$ time, using the color identifiers computed by Theorem 7.9.

To prove correctness, consider any walk $u'_0 u'_1 \dots$ consistent with σ_H , and choose $u_0 \in V(G_i)$ with $\chi_\infty^{G_i}(u_0) = \chi_\infty^H(u'_0)$. As in the proof of Theorem 7.12, we construct a shadow walk $u_0 u_1 \dots$ in G_i with matching vertex colors and edge weights. At Player 1 vertices, color consistency ensures that $\hat{\sigma}_i(u_k)$ has the same successor color and edge weight as the move chosen by σ_H at u'_k . At Player 2 vertices, Neighborhood Preservation supplies a matching edge. Hence the shadow walk is consistent with $\hat{\sigma}_i$.

The shadow-play argument in both directions shows that equally colored vertices have the same minimum initial energy. Since $\hat{\sigma}_i$ is optimal from every starting vertex and the two walks have identical edge weights, σ_H is optimal from every starting vertex as well. In particular, it wins from the designated initial vertex with energy $\text{OPT}(H) = \text{OPT}(G_i)$. The total query time remains $O((n + m) \log n)$. $\square$

8 More on Weisfeiler-Leman (Advanced Arguments)

The two algorithms from Section 7 highlight a general approach for obtaining algorithms in the Isomorphic-Priors model: if optimal solution values are preserved under WL equivalence (see, e.g., Theorems 7.11 and 7.12), then we obtain efficient algorithms in our model. This strategy is not limited to 1-WL, but also holds for higher-dimensional Weisfeiler-Leman. More precisely, for each $\ell \geq 1$, the ℓ -dimensional Weisfeiler-Leman (ℓ -WL) algorithm is a heuristic for graph isomorphism testing that accumulates information by coloring ℓ -tuples of vertices. For increasing $\ell \geq 1$, the algorithm becomes more expressive, but it also becomes computationally more expensive.

In this section, we present a systematic way to use the ℓ -WL algorithm within the Isomorphic-Priors model. After describing the algorithm in Section 8.1, we begin by showing in Section 8.2 that every problem Π that is invariant under ℓ -WL is polynomial time solvable inside the Isomorphic-Priors model (see Theorems 8.3 and 8.4). We then develop a technique that helps us prove when a problem is in fact ℓ -WL invariant.

In Section 8.3 and Section 8.4, we apply this technique to the CSP and CST problems and show that they are 1-WL invariant and 2-WL invariant, respectively. As a result, we obtain fast algorithms for both problems in the Isomorphic-Priors model. Furthermore, additional examples are discussed in Appendix A. However, the Weisfeiler-Leman algorithm has its limits. In Section 8.5, we present an example of a problem that lies in P (and can therefore be efficiently solved in the Isomorphic-Priors model) but that is not ℓ -WL invariant for any constant ℓ . This means that Weisfeiler-Leman cannot be used to obtain a fast algorithm for this problem in the Isomorphic-Priors model.

8.1 The Weisfeiler-Leman Algorithm

We start with a description of the ℓ -WL algorithm (see, e.g., [GN21, Kie20b, Kie20a]). Fix some $\ell \geq 2$ and let $G = (V, E, \alpha_V, \alpha_E)$ be an attributed graph. For two colorings $\chi, \chi': V^\ell \rightarrow \mathcal{C}$, we say χ *refines* χ' , denoted $\chi \preceq \chi'$, if $\chi(\bar{v}) = \chi(\bar{w})$ implies $\chi'(\bar{v}) = \chi'(\bar{w})$ for all $\bar{v}, \bar{w} \in V^\ell$. The colorings χ and χ' are *equivalent*, denoted $\chi \equiv \chi'$, if $\chi \preceq \chi'$ and $\chi' \preceq \chi$.

We define the initial coloring $\chi_0^{G,\ell}: V^\ell \rightarrow \mathcal{C}$ computed by ℓ -WL as the coloring where each tuple is colored with the isomorphism type of its underlying ordered subgraph. More precisely, for a second attributed graph $G' = (V', E', \alpha'_V, \alpha'_E)$, and vertices $v_1, \dots, v_\ell \in V$ and $v'_1, \dots, v'_\ell \in V'$ we have $\chi_0^{G,\ell}(v_1, \dots, v_\ell) = \chi_0^{G',\ell}(v'_1, \dots, v'_\ell)$ if and only if, for all $i, j \in [\ell]$, we have

- $v_i = v_j$ if and only if $v'_i = v'_j$,
- $\alpha_V(v_i) = \alpha'_V(v'_i)$,
- $(v_i, v_j) \in E$ if and only if $(v'_i, v'_j) \in E'$, and
- if $(v_i, v_j) \in E$, then $\alpha_E(v_i, v_j) = \alpha'_E(v'_i, v'_j)$.

We then recursively define the coloring $\chi_i^{G,\ell}$ obtained after i rounds of the algorithm. For $\bar{v} = (v_1, \dots, v_k) \in V^\ell$, set

$$\chi_{i+1}^{G,\ell}(\bar{v}) := (\chi_i^{G,\ell}(\bar{v}), \mathcal{M}_i(\bar{v})),$$

where

$$\mathcal{M}_i(\bar{v}) := \left\{ \left(\chi_i^{G,\ell}(\bar{v}[w/1]), \dots, \chi_i^{G,\ell}(\bar{v}[w/\ell]) \right) \mid w \in V \right\}$$

and $\bar{v}[w/i] := (v_1, \dots, v_{i-1}, w, v_{i+1}, \dots, v_\ell)$ is the tuple obtained from substituting the i -th entry of $\bar{v}$ with w . By definition, $\chi_{i+1}^{G,\ell} \preceq \chi_i^{G,\ell}$ holds for all $i \geq 0$. So there is a minimal $i_\infty \geq 0$ such that $\chi_{i_\infty}^{G,\ell} \equiv \chi_{i_\infty+1}^{G,\ell}$, and we set $\chi_\infty^{G,\ell} := \chi_{i_\infty+1}^{G,\ell}$.

Definition 8.1 (ℓ -WL Equivalence). Two (attributed) graphs G and H are ℓ -WL equivalent ($G \equiv_{\ell\text{-WL}} H$) if $\{\{\chi_{\infty}^{G,\ell}(\bar{v}) \mid \bar{v} \in (V(G))^{\ell}\}\} = \{\{\chi_{\infty}^{H,\ell}(\bar{w}) \mid \bar{w} \in (V(H))^{\ell}\}\}$.

Observe that isomorphic (attributed) graphs G, H are always ℓ -WL equivalent.

Given an (attributed) graph G , the coloring $\chi_{\infty}^{G,\ell}$ can be computed in time $O(n^{\ell+1} \log n)$ [IL90] (more precisely, similar to Theorem 7.9, we compute a coloring equivalent to $\chi_{\infty}^{G,\ell}$). Moreover, similar to Theorem 7.10, we can compute a complete invariant for ℓ -WL equivalence (see, e.g., [Ott17]). In turn, this can be used as an Invariant and Solution-Preserving Signature for problems such as Constrained Spanning Tree (see Section 8.4).

Theorem 8.2. *Let $\ell \geq 2$. There is a deterministic algorithm that, given an attributed graph G , outputs a string $\text{WL}_{\ell}(G) \in \{0, 1\}^*$ in time $O(|V(G)|^{\ell+1} \log |V(G)|)$, such that for every two input graphs G, H it holds that $G \equiv_{\ell\text{-WL}} H$ if and only if $\text{WL}_{\ell}(G) = \text{WL}_{\ell}(H)$.*

8.2 ℓ -WL Invariant Functions are in the Isomorphic-Priors Model

A function f on (attributed) graphs is called ℓ -WL invariant if $f(G) = f(H)$ for all (attributed) graphs G, H with $G \equiv_{\ell\text{-WL}} H$.²⁵ We say that an optimization problem Π is ℓ -WL invariant if OPT_{Π} is ℓ -WL invariant (where OPT_{Π} denotes the function that maps an (attributed) input graph to the value of an optimum solution). For such problems, we obtain polynomial-time algorithm in the Isomorphic-Priors model using similar arguments as in Section 7.

Theorem 8.3. *Let $\ell \geq 2$ and let Π be an optimization problem that is ℓ -WL invariant.*

Then there is an algorithm that solves Π with k prior instances each of size at most n in $O(kn^{\ell+1} \log n)$ preprocessing time and $O(n^{\ell+1} \log n)$ query time.

Proof. Write $\mathcal{S} := \{G_i\}_{i=1\dots k}$ for the prior instances. We describe the different phases of the algorithm.

Preprocessing: For each graph $G_i \in \mathcal{S}$, we compute a complete invariant $\text{WL}_{\ell}(G_i) \in \{0, 1\}^*$ for G_i using Theorem 8.2 in time $O(n^{\ell+1} \log n)$ time. We insert each string $\text{WL}_{\ell}(G_i) \in \{0, 1\}^*$ into a binary search tree, where the i -th letter determines whether we take the left/right subtree on the i -th level. This takes time proportional to the length of the string $\text{WL}_{\ell}(G_i)$, i.e., it can be done in time $O(n^{\ell+1} \log n)$. For the corresponding node where the key $\text{WL}_{\ell}(G_i)$ is saved, we save $\text{OPT}_{\Pi}(G_i)$.

Note that, if two graphs $G_i, G_j \in \mathcal{S}$ have the same complete invariant, they are ℓ -WL equivalent (Theorem 8.2). In particular, since Π is ℓ -WL invariant, we get that $\text{OPT}_{\Pi}(G_i) = \text{OPT}_{\Pi}(G_j)$, and no conflicting values can be assigned to a single node of the search tree. Overall, processing all k graphs takes $O(kn^{\ell+1} \log n)$ time.

Query: Given the query graph H , we compute a complete invariant $\text{WL}_{\ell}(H) \in \{0, 1\}^*$ for G_i using Theorem 8.2 in time $O(n^{\ell+1} \log n)$ time. Looking up the computed invariant in the binary search tree takes $O(n^{\ell+1} \log n)$ time, allowing us to return $\text{OPT}_{\Pi}(H) = \text{OPT}_{\Pi}(G_i)$ (which is correct since Π is ℓ -WL invariant). If the string $\text{WL}_{\ell}(H)$ is not contained in the search tree, then we can correctly claim that H is not isomorphic to any graph in $\mathcal{S}$ because graphs that are not ℓ -WL equivalent are not isomorphic. The total query time is $O(n^{\ell+1} \log n)$. $\square$

For 1-WL invariant optimization problems, we obtain the following variant which is proved analogously using Theorem 7.10.

Theorem 8.4. *Let Π be an optimization problem that is 1-WL invariant. Then there is an algorithm that solves Π with k prior instances each of size at most n in $O(k(n + m) \log n)$ preprocessing time and $O((n + m) \log n)$ query time.*

²⁵In the literature (see, e.g., [GGR24]), the minimal $\ell \geq 1$ (if it exists) such that f is ℓ -WL invariant is also called the *WL dimension* of f .

We note that, in Theorems 7.2 and 7.4, we can also reconstruct an optimal solution of a query graphs if we are given optimal solutions for all priors. While it does not seem possible to obtain such a statement in the general setting of Theorems 8.3 and 8.4, we note that all our arguments for specific problems do imply that optimal solutions can indeed be reconstructed (see Appendix A for further examples).

In the following, we use an alternative way to characterize when an optimization problem is ℓ -WL invariant. We limit ourself to minimization problems. However, our approach can easily be extended to maximization problems. Remember that $\mathcal{G}$ is the set of valid attributed input graphs.

Definition 8.5. Let Π be an minimization problem. We call a triple $(W, \text{check}: \mathcal{G} \times W \rightarrow \mathbb{R}_{\geq 0}, \text{cost}: W \rightarrow \mathbb{R})$ a *witness function of OPT_{Π}* if:

- For all G and $X \in W$: if $\text{check}(G, X) > 0$ then $OPT_{\Pi}(G) \leq \text{cost}(X)$.
- If $OPT_{\Pi}(G) < \infty$ then there is a $X \in W$ with $OPT_{\Pi}(G) = \text{cost}(X)$ and $\text{check}(G, X) > 0$.

The intuition behind $(W, \text{check}: \mathcal{G} \times W \rightarrow \mathbb{R}_{\geq 0}, \text{cost}: W \rightarrow \mathbb{R})$ is that it characterizes the witnesses of the minimization problem Π . The function “check” checks for an element X if there exist a feasible solution (which is the case if $\text{check}(G, X) > 0$) with cost at most $\text{cost}(X)$. Further, if G has a feasible solution (i.e. $OPT_{\Pi}(G) < \infty$) then there is a *witness* $X \in G$ that attains the optimal value (i.e. $OPT_{\Pi}(G) = \text{cost}(X)$ and $\text{check}(G, X) > 0$).

In the following, we show that we can a witness function to show that OPT_{Π} is ℓ -WL. To this end, we show that the witness function itself is ℓ -WL invariant.

Definition 8.6. Let $\text{check}: \mathcal{G} \times W \rightarrow \mathbb{R}_{\geq 0}$ be a function. We say that check is ℓ -WL invariant on $\mathcal{G}$ if: for all graphs $G, H \in \mathcal{G}$ with $G \equiv_{\ell\text{-WL}} H$ and all $X \in W$ we obtain that $\text{check}(G, X) > 0$ if and only if $\text{check}(H, X) > 0$.

Lemma 8.7. Let Π be an minimization problem and $(W, \text{check}: \mathcal{G} \times W \rightarrow \mathbb{R}_{\geq 0}, \text{cost}: W \rightarrow \mathbb{R})$ be a witness function of OPT_{Π} . If check is ℓ -WL invariant on $\mathcal{G}$ then Π is ℓ -WL invariant.

Proof. Let $G, H \in \mathcal{G}$ be two graphs with $G \equiv_{\ell\text{-WL}} H$. Without loss of generality, we assume $OPT_{\Pi}(G) \leq OPT_{\Pi}(H)$ (otherwise we change to role of G and H). We have to show that $OPT_{\Pi}(H) \leq OPT_{\Pi}(G)$. If $OPT_{\Pi}(G) = \infty$ then we are already done. Otherwise, $OPT_{\Pi}(G) < \infty$. Thus, there exist a $X \in W$ with $OPT_{\Pi}(G) = \text{cost}(X)$ and $\text{check}(G, X) > 0$. Now, since check is ℓ -WL invariant, we obtain that $\text{check}(H, X) > 0$. Hence, $OPT_{\Pi}(H) \leq \text{cost}(X) = OPT_{\Pi}(G)$. $\square$

8.3 CSP is 1-WL Invariant

We first demonstrate the above approach on the **Constrained Shortest Path (CSP)** problem and present an alternative proof for Theorem 7.2 using a characterization of 1-WL via homomorphism counts [Dvo10, DGR18]. Write OPT for the optimal value function of **CSP** and $\mathcal{G}$ for the set of valid input graphs of **CSP**. We show that OPT is 1-WL invariant for all values.²⁶ This together with Theorem 8.4 immediately yields that **CSP** is in the Isomorphic-Priors model.

Theorem 8.8. *The Problem CSP is 1-WL invariant.*

To prove Theorem 8.8 we define a witness for OPT and show that this witness is 1-WL invariant. To this end, we start by defining graph homomorphisms.

²⁶There is the small technical detail that an input graph G has a length constraint B that is part of the input. Consistent with previous sections, we say that two input graphs G and H are 1-WL equivalent if G and H have the same length constraint B and $G \equiv_{1\text{-WL}} H$ for the underlying attributed graphs.

Definition 8.9 (Homomorphism of attributed Graphs). Let

$$G = (V_G, E_G, \alpha_G^V, \alpha_G^E) \quad \text{and} \quad H = (V_H, E_H, \alpha_H^V, \alpha_H^E)$$

be two graphs, where V_G, V_H are vertex sets, E_G, E_H are edge sets, and α^V, α^E assign attributes from alphabets Σ_V, Σ_E to the vertices and edges, respectively.

We say that a function $f: V_H \rightarrow V_G$ is a *homomorphism* from H to G if:

1. **Adjacency is preserved:** For all $(u, v) \in E_H$ it is true that $(\varphi(u), \varphi(v)) \in E_G$.
2. **Vertex attributes are preserved:** For all $u \in H$ it is true that $\alpha_H^V(u) = \alpha_G^V(f(u))$.
3. **Edge attributes are preserved:** For all $(u, v) \in E_H$ it is true that $\alpha_H^E(u, v) = \alpha_G^E(f(u), f(v))$.

We write $\#\text{Hom}(H \rightarrow G)$ for the number of homomorphism from H to G .

Next, we use homomorphism to define a witness function. Note that each input graph G has a length constraint B .

Definition of a Witness:

- $W = \{X \mid X \subseteq \mathbb{N}_{\geq 1} \times \mathbb{R}_{\geq 0} \times \mathbb{R}_{\geq 0}\}$,
- $\text{cost}(X) = \sum_{(n, c, \ell) \in X} n \cdot c$,
- Let P be a graph. We say that P is a *X -colored s, t -path* if
 - P is a path with colored start-vertex s and colored end-vertex t ,
 - each edge in P is colored with a color (c, ℓ) , and
 - for each $(n, c, \ell) \in X$ the path P contains exactly n edges of color (c, ℓ) .

Let $P_X := \{P \mid P \text{ is a } X\text{-colored } s, t\text{-path}\}$ be a set of paths then we define

$$\text{check}(G, X) = \begin{cases} \sum_{P \in P_X} \#\text{Hom}(P \rightarrow G) & \text{if } \sum_{(n, c, \ell) \in X} n \cdot \ell \leq B \\ 0 & \text{otherwise} \end{cases},$$

where B is length constraint of G .

Lemma 8.10. *The triple $(W, \text{check}: \mathcal{G} \times W \rightarrow \mathbb{R}, \text{cost}: W \rightarrow \mathbb{R})$ is a witness function of OPT .*

Proof. We start by showing that for all $X \in W$ with $\sum_{(n, c, \ell) \in X} n \cdot \ell \leq B$ we obtain

$$\text{check}(G, X) = \#\{\text{walks in } G \text{ from } s \text{ to } t \text{ s.t. for all } (n, c, \ell) \in X \text{ the color } (c, \ell) \text{ is used } n \text{ times}\}. \quad (3)$$

Let $P \in P_X$ be a path. Since P is a X -colored s, t -path, we obtain that $V(P) = \{v_1, \dots, v_t\}$ and $E(P) = \{e_1, \dots, e_{t-1}\}$ with $\alpha_P^V(v_1) = s$, $\alpha_P^V(v_t) = t$ and $\alpha_P^E(e_i) = (c_i, \ell_i)$. Now, $\#\text{Hom}(P \rightarrow G)$ is equal to the number of walks in G that start at s , end at t , and where we used an edge of color (c_i, ℓ_i) during the i th step of the walk. Thus, equation (3) follows since $\text{check}(G, X)$ computes the sum over all possible X -colored s, t -paths.

Next, we check that $(W, \text{check}: \mathcal{G} \times W \rightarrow \mathbb{R}, \text{cost}: W \rightarrow \mathbb{R})$ is a witness function of OPT . For the first property, let $G \in \mathcal{G}$, $X \in W$ and assume that $\text{check}(G, X) > 0$. By (3), we obtain that there is walk from s to t that uses the colors X . This walk induces a path P from s to t that is

obtained by deleting repeating vertices and edges of the walk. Note that for each $(n, c, \ell) \in X$ the path P uses edges of color (c, ℓ) at most n times. Further since $\text{check}(G, X) > 0$, we obtain

$$\ell(P) \leq \sum_{(n, c, \ell) \in X} n \cdot \ell \leq B \quad \text{and} \quad c(P) \leq \sum_{(n, c, \ell) \in X} n \cdot c = \text{cost}(X).$$

Thus, P is a feasible path of length at most B . Further, $\text{OPT}(G) \leq \text{cost}(X)$ since P has cost at most $\text{cost}(X)$.

Next, assume that $\text{OPT}(G) < \infty$ then there is a s, t -path P^* with $\ell(P^*) \leq B$ and $\text{OPT}(G) = c(P^*)$. Now, let X be the set that contains (n, c, ℓ) if and only if the edge color (c, ℓ) is used n times in P^* . Observe that $\sum_{(n, c, \ell) \in X} n \cdot \ell = \ell(P^*) \leq B$. Further note that $\text{OPT}(G) = c(P^*) = \sum_{(n, c, \ell) \in X} n \cdot c = \text{cost}(X)$. Since P^* is path s, t -path and therefore a walk, (3) yields that $\text{check}(G, X) \geq 1 > 0$. Thus, $\text{OPT}(G) = \text{cost}(X)$ and $\text{check}(G, X) > 0$. $\square$

The proof of the following lemma was discussed in [Bök19, Section 1.1]. The lemma is a generalization of Theorem 1 in [DGR18].

Lemma 8.11. *Let F be a tree and H, G be two graphs. If $H \equiv_{1\text{-WL}} G$ then $\#\text{Hom}(F \rightarrow G) = \#\text{Hom}(F \rightarrow H)$.*

Lemma 8.12. *The function $\text{check}: \mathcal{G} \times W \rightarrow \mathbb{R}$ is 1-WL invariant.*

Proof. Let $G, H \in \mathcal{G}$ be two graphs with $G \equiv_{1\text{-WL}} H$. According to Lemma 8.11, $\#\text{Hom}(F \rightarrow G) = \#\text{Hom}(F \rightarrow H)$ for all trees F . Since every path is a tree, we obtain that $\text{check}(G, X) = \text{check}(H, X)$ for all $X \in W$ that satisfy the length constraint, that is, $\sum_{(n, c, \ell) \in X} n \cdot \ell \leq B$. Note that G and H have the same length constraint since $G \equiv_{1\text{-WL}} H$. $\square$

Proof of Theorem 8.8. According to Lemma 8.10, the triple $(W, \text{check}: \mathcal{G} \times W \rightarrow \mathbb{R}, \text{cost}: W \rightarrow \mathbb{R})$ is a witness function of OPT . Further, Lemma 8.12 shows that check is 1-WL invariant. Thus, Lemma 8.7 implies that CSP is 1-WL invariant. $\square$

Corollary 8.13. *There is an algorithm that solves the CSP problem with k prior instances each of size at most n in $O(k(n + m) \log n)$ preprocessing time and $O((n + m) \log n)$ query time.*

Proof. This is a direct consequence of Theorem 8.4 combined with Theorem 8.8. $\square$

8.4 CST is 2-WL Invariant

Next, we again turn to the **Constrained Spanning Tree (CST)** problem. Recall that, in Theorem 6.3, we obtained a randomized algorithm in the Isomorphic-Priors model with $\tilde{O}(kn^\omega)$ preprocessing time and $O(n^\omega)$ query time. In the following, we obtain a deterministic algorithm via 2-WL. Write OPT for the optimal value function of CST and $\mathcal{G}$ for the set of valid input graphs of CST. We show that OPT is 2-WL invariant for all values.²⁷ This together with Theorem 8.3 this immediately yields a deterministic algorithm for CST in the Isomorphic-Priors model.

Theorem 8.14. *The problem CST is 2-WL invariant.*

²⁷As before, we say that two input graphs G and H are 2-WL equivalent if G and H have the same budget B value and $G \equiv_{2\text{-WL}} H$ for the underlying attributed graphs.

Definition of a Witness: To prove Theorem 8.14 we define a witness for OPT and show that this witness is 2-WL invariant.

- $W = \{X \mid X \subseteq \mathbb{N}_{\geq 0} \times \mathbb{R}_{\geq 0} \times \mathbb{R}_{\geq 0}\},$
- $\text{cost}(X) = \sum_{(n,c,d) \in X} n \cdot c,$
- Let T be a graph. We say that T is a X -colored tree if
 - T is a tree where each vertex has the same color,
 - each edge in T is colored with a color (c, d) , and
 - for each $(n, c, d) \in X$ the tree T contains exactly n edges of color (c, d) .

We define

$$\text{check}(G, X) = \left| \left\{ T : T \text{ is a spanning tree of } G \text{ and } T \text{ is } X\text{-colored and } \sum_{(n,c,d) \in X} n \cdot d \leq B \right\} \right|,$$

where B is the budget associated with the input graph G . This means that $\text{check}(G, X) = 0$, whenever X is outside the budget constraint of B

Lemma 8.15. *The triple $(W, \text{check}: \mathcal{G} \times W \rightarrow \mathbb{R}, \text{cost}: W \rightarrow \mathbb{R})$ is a witness function of OPT .*

Proof. Let $G \in \mathcal{G}$ and $X \in W$ and assume that $\text{check}(G, X) > 0$. By the definition of $\text{check}(G, X)$, we obtain that there is X -colored spanning tree T of G . For each $(n, c, d) \in X$, the tree T uses edges of color (c, d) exactly n times. Further, since $\text{check}(G, X) > 0$ we obtain

$$\ell(T) = \sum_{(n,c,d) \in X} n \cdot d \leq B \quad \text{and} \quad c(T) = \sum_{(n,c,d) \in X} n \cdot c = \text{cost}(X).$$

Thus, T is a feasible tree of length at most B . Additionally, $OPT(G) \leq \text{cost}(X)$ since T has cost $\text{cost}(X)$.

Next, assume that $OPT(G) < \infty$. Now, there is a spanning tree T^* with $\sum_{(n,c,d) \in X} n \cdot d \leq B$ and $OPT(G) = c(T^*)$. Let X be the set that contains (n, c, d) if and only if the edge color (c, d) is used n times in T^* . Note that $OPT(G) = c(T^*) = \sum_{(n,c,d) \in X} n \cdot d = \text{cost}(X)$. Since T^* is by definition a X -colored spanning tree, we obtain that $\text{check}(G, X) > 0$. Thus, $OPT_{\text{CST}}(G) = \text{cost}(X)$ and $\text{check}(G, X) > 0$. $\square$

We prove the following lemma which together with Lemma 8.15 immediately yields Theorem 8.14.

Lemma 8.16. *The function $\text{check}: \mathcal{G} \times W \rightarrow \mathbb{R}$ is 2-WL invariant on $\mathcal{G}$.*

Proof (sketch). To prove Lemma 8.16 we show that $\text{check}(G, X) = \text{check}(H, X)$ for all $G, H \in \mathcal{G}$ with $G \equiv_{2\text{-WL}} H$ and $X \in W$. Without loss of generality we assume that X satisfies $\sum_{(n,c,d) \in X} n \cdot d \leq B$ since otherwise $0 = \text{check}(G, X) = \text{check}(H, X) = 0$. We start with the Kirchhoff polynomial (see [CY00])

$$\text{Kir}(G) := \sum_T \prod_{e \in T} X_e,$$

where each edge e in G has its own variable X_e and the sum is over all spanning trees T of G . If we substituted each X_e with a new variable $c(e)$, where $c(e)$ is the color of the edge e , then we get the *Spanning Tree Polynomial* $P_{ST}(G)$ from Section 6.2. Formally, we have $\text{Kir}(G)|_{X_e=c(e)} \equiv P_{ST}(G)$, where $\text{Kir}(G)|_{X_e=c(e)}$ is obtained by substituting X_e with $c(e)$ in $\text{Kir}(G)$.

Further, note that the function $\text{check}(G, X)$ computes the coefficients of the *Spanning Tree Polynomial*. Specially, the coefficient $c_{w_1, \dots, w_{n-1}}$ of $P_{ST}(G)$ is equal to $\text{check}(G, X)$ where X is the set that contains a triple (n, c, d) if and only if the color (c, d) appears exactly n times in the multiset $\{w_1, \dots, w_{n-1}\}$ (note that each w_i is actually a (c, d) -tuple). Thus, if $P_{ST}(G) \equiv P_{ST}(H)$ then $\text{check}(G, X) = \text{check}(H, X)$ for all $X \in W$, proving the theorem.

To show $P_{ST}(G) \equiv P_{ST}(H)$, we use a result from [CY00] which states that for each $i \in [n]$

$$\text{Kir}(G) \equiv \det(\tilde{L}_i(G)),$$

where $\tilde{L}_i(G)$ is the Laplacian $L(G)$ of G with its i -th row and i -th column removed. Note that $\det(\tilde{L}_i(G))$ is a polynomial over the variables X_e . Now, $\text{Kir}(G)|_{X_e=c(e)} \equiv P_{ST}(G)$ and $\text{Kir}(G) \equiv \det(\tilde{L}_i(G))$ implies that $P_{ST}(G) \equiv \det(\tilde{L}_i^C(G))$ where $\tilde{L}_i^C(G)$ is obtained by substituted each variable X_e in $\tilde{L}_i(G)$ with $c(e)$.

Thus, it is enough to prove $P_{ST}(G) \equiv \det(\tilde{L}_i^C(G)) \equiv \det(\tilde{L}_i^C(H)) \equiv P_{ST}(H)$ whenever $H \equiv_{2\text{-WL}} G$. By [RS23]²⁸, $L^C(G)$ is cospectral to $L^C(H)$. Let d_i^G (respectively d_i^H) be the i th coefficient of the characteristic polynomial of $L^C(G)$ (respectively $L^C(H)$). Note that $d_i^H, d_i^G \in \mathbb{C}[c(e)]$.²⁹ Since $L^C(G)$ and $L^C(H)$ are cospectral, it follows that $d_i^H = d_i^G$. Note that d_1^H is the linear coefficient of the characteristic polynomial.³⁰ A well-known theorem from linear algebra (see e.g. [Alf18, Theorem 1.8]) implies

$$d_1^H \equiv -\sum_{i=1}^n \det(\tilde{L}_i^C(H)), \quad d_1^G \equiv -\sum_{i=1}^n \det(\tilde{L}_i^C(G)).$$

Now, since $\det(\tilde{L}_i^C(H)) \equiv \det(\tilde{L}_j^C(H))$ and $\det(\tilde{L}_i^C(G)) \equiv \det(\tilde{L}_j^C(G))$ for all $i, j \in [n]$, we obtain

$$P_{ST}(G) \equiv \det(\tilde{L}_i^C(G)) \equiv \frac{-d_1^G}{n} \equiv \frac{-d_1^H}{n} \equiv \det(\tilde{L}_i^C(H)) \equiv P_{ST}(H).$$

□

Proof of Theorem 8.14. Let $B \in \mathbb{R}_{>0}$. According to Lemma 8.15, the triple $(W, \text{check}: \mathcal{G} \times W \rightarrow \mathbb{R}, \text{cost}: W \rightarrow \mathbb{R})$ is a witness function of *OPT*. Further, Lemma 8.16 shows that check is 2-WL invariant on $\mathcal{G}$. Thus, Lemma 8.7 implies that CSP is 2-WL invariant. □

Corollary 8.17. *There is an algorithm that solves the CST problem with k prior instances each of size at most n in $O(kn^3 \log n)$ preprocessing time and $O(n^3 \log n)$ query time.*

Proof. This is a direct consequence of Theorem 8.3 combined with Theorem 8.14. □

8.5 Examples when Weisfeiler-Leman Fails

Given a graph G of order n and maximum degree $k \geq 3$. In [CFI92] Cai, Fürer, and Immerman introduced the idea of CFI graphs. A CFI graph is obtained by taking G , a parity $p \in \{\text{odd}, \text{even}\}$ and computing a labeled graph $\text{CFI}(G, p)$. This construction can be performed in time $O(2^k \cdot n)$.³¹ Further, $\text{CFI}(G, p)$ has a maximum degree at most $2^{k-2} + 1$.

²⁸See Theorem 26 combined with Example 22 that shows that Laplacian mapping $G \rightarrow L(G)$ is in $\mathfrak{E}$.

²⁹That is, the ring of polynomials with variables $c(e)$.

³⁰That is, the coefficient corresponding to x^1 in the characteristic polynomial of $L^C(H)$.

³¹See [CFI92, Section 6] for a detailed construction. Note that in this paper $X(G) = \text{CFI}(G, \text{even})$ and $\tilde{X}(G) = \text{CFI}(G, \text{odd})$.

Lemma 8.18. *Let $k \geq 3$. There is an algorithm that given a k -regular graph G , a parity $p \in \{\text{odd}, \text{even}\}$, and a labeled graph H checks whether $\text{CFI}(G, p)$ and H are isomorphic. Further the algorithm runs in time $O(|V(G)|^{f(k)})$ for some computable function f .*

Proof. By construction the graph $\text{CFI}(G, p)$ has a maximum degree of $2^{k-2} + 1$. By [GNS23], isomorphism testing on unlabeled graphs with maximum degree $2^{k-2} + 1$ can be done in time $O(n^{\log(2^{k-2}+1)^c}) \subseteq O(n^{k^c})$ for a fixed constant c that is independent of k . By [BC79], the problem of finding isomorphism in unlabeled graphs is polynomially time equivalent to the problem of finding isomorphism in labeled graphs. Thus, there is an algorithm that runs in time $O(|V(G)|^{f(k)})$, for some computable function f , and checks if a labeled graph H is isomorphic to $\text{CFI}(G, p)$. $\square$

Write $\mathcal{G}$ for the set of all graphs.

Lemma 8.19. *There is a function $\text{CFI}: \mathcal{G} \rightarrow \{0, 1\}$ with the following properties:*

- *The function CFI is computable in polynomial time: Given a colored graph G of order n , we can compute $\text{CFI}(G)$ in time $O(n^c)$ for some fixed constant c .*
- *For all isomorphic graphs G and H we obtain $\text{CFI}(G) = \text{CFI}(H)$.*
- *For all ℓ there are graphs G, H of order $\Theta(\ell)$ with $G \equiv_{\ell\text{-WL}} H$ and $\text{CFI}(G) \neq \text{CFI}(H)$.*

Proof. By [Alo21], for each n we can construct a 4-regular expander graph E_n with n vertices. To be more specific, there is an algorithm that given n constructs E_n in time $O(n^d)$ for some constant d . Further, it is well-known that expander graphs have linear separator.³² Formally, there is a global constant $\alpha > 0$ such that each separator of E_n has at least αn many vertices. We use this to define the following property on graphs:

$$\text{CFI}: \mathcal{G} \rightarrow \{0, 1\}, G \mapsto \begin{cases} 1 & \text{if there is an } n' \text{ with } G \cong \text{CFI}(E_{n'}, \text{odd}) \\ 0 & \text{otherwise} \end{cases}.$$

Next, we check that CFI has the promised properties

- Let G be an input graph of order n . We show how to compute $\text{CFI}(G)$. For each n' the graph $\text{CFI}(E_{n'}, \text{odd})$ has $(4 + 4 + 2^3)n' = 16n'$ many vertices. Thus, we compute $n' = n/16$. If n' is not an integer then we return 0 since G cannot be isomorphic to any $\text{CFI}(E_m, \text{odd})$. Otherwise, we use [Alo21] to compute $E_{n'}$ in time $O(n^d)$ and then construct $\text{CFI}(E_{n'}, \text{odd})$ in time $O(n)$. Note that G is either isomorphic to $\text{CFI}(E_{n'}, \text{odd})$ or G is non-isomorphic to every $\text{CFI}(E_m, \text{odd})$. Next, since $E_{n'}$ is by construction 4-regular Lemma 8.18 yields that we can check $G \cong \text{CFI}(E_{n'}, \text{odd})$ in time $O(n^{d'})$, for some fixed integer d' . If this is the case then return 1. Otherwise, we return 0. By setting $c = \max(d, d')$, we obtain that $\text{CFI}(G)$ can be computed in time $O(n^c)$.
- If G and H are isomorphic then $G \cong \text{CFI}(E_{n'}, \text{odd})$ if and only if $H \cong \text{CFI}(E_{n'}, \text{odd})$. Thus, $\text{CFI}(G) = \text{CFI}(H)$.
- Let $\ell \in \mathbb{N}$ and $\alpha > 0$ be a constant such that each separator of E_n has at least αn many vertices. Let $n := \lceil (\ell + 2)/\alpha \rceil$. We show that the graphs $\text{CFI}(E_n, \text{odd})$ and $\text{CFI}(E_n, \text{even})$ are sufficient. Note that both graphs have $\Theta(\ell)$ many vertices. Further, by construction each separator of E_n has at least $\ell + 1$ many vertices. Thus, [CFI92, Theorem 6.4] implies $\text{CFI}(E_n, \text{odd}) \equiv_{\ell\text{-WL}} \text{CFI}(E_n, \text{even})$.

³²A separator of a graph G is a subset $S \subseteq V(G)$ such that the induced subgraph with vertex set $V(G) \setminus S$ has no connected component with more than $|V(G)|/2$ vertices.

$\text{CFI}(E_n, \text{even})$.³³ Further, [CFI92, Theorem 6.2] yields that $\text{CFI}(E_n, \text{odd})$ and $\text{CFI}(E_n, \text{even})$ are non-isomorphic. Hence, $\text{CFI}(\text{CFI}(E_n, \text{odd})) = 1$ and $\text{CFI}(\text{CFI}(E_n, \text{even})) = 0$. $\square$

Theorem 8.20. *There exist a decision problem $\Pi = (\mathcal{G}, \text{CFI}: \mathcal{G} \rightarrow \{0, 1\})$ such that Π is polynomial time solvable and for all ℓ the problem Π is not ℓ -WL invariant.*

Proof. Let Π be the problem that given a graph G decides if $\text{CFI}(G) = 1$. By Lemma 8.19, Π is polynomial time solvable but for all ℓ the problem Π is not ℓ -WL invariant. $\square$

Theorem 8.3 shows that if a problem is ℓ -WL invariant then it is polynomial time solvable inside the Isomorphic-Priors model. However, Theorem 8.20 demonstrates that there are problems that are in P (and that are therefore polynomial time solvable inside the Isomorphic-Priors model) but for which our Weisfeiler-Leman approach fails.

Remark 8.21. The original CFI graph construction can be viewed as a graph-theoretic encoding of parity information. More specifically, the underlying idea of CFI graphs is closely related to the solvability of linear systems over the finite field $\mathbb{F}_2$, with the gadgets enforcing local parity constraints (see [DGH⁺13]). It follows that solving linear equations over $\mathbb{F}_2$ is not invariant under ℓ -WL for any ℓ . This therefore yields another polynomial-time solvable problem that cannot be captured by Weisfeiler-Leman.

We note that, using similar constructions, the same can be proved for several other hard problems, e.g., for counting certain (induced) subgraphs [Neu24a, LB24, DMW24, CN25, DMW25] or dominating set [GGR24].

9 Conditional Lower Bounds

In this section, we prove different hardness results for the Isomorphic-Priors model. As our source of hardness, we use the Graph Isomorphism problem.

Definition 9.1 (Graph Isomorphism (GI)). In the Graph Isomorphism problem we are given a pair of graphs $I = (G_0, G_1)$ and have to decide if $G_0 \cong G_1$.

Two problem instances $I = (G_0, G_1)$ and $J = (H_0, H_1)$ are isomorphic if $G_0 \cong H_0$ and $G_1 \cong H_1$.

Up to this date, it is not known whether GI is in P despite a lot of effort from many researchers [Koz78]. We can therefore use GI as a source of hardness and show that many problems are not polynomial time solvable inside the Isomorphic-Priors model unless $\text{GI} \in \mathsf{P}$, which would be a breakthrough result.

Furthermore, it is not known if GI is in coMA .³⁴ Again, we can use this as a source of hardness. We show in Section 9.1 that GI is not polynomial time solvable within our model unless $\text{GI} \in \text{coMA}$.

Next, we are utilizing the hardness of GI to obtain further hardness results within our model for “normal” decision problems Π . To this end, we associate each problem Π with a specific promise version of the same problem that we denote as Iso- Π . See Section 9.2 for a formal definition. We

³³By [CFI92, Theorem 6.4], we obtain $\text{CFI}(E_n, \text{odd}) \equiv_{\mathcal{C}_{\ell+1}} \text{CFI}(E_n, \text{even})$. By [CFI92, Theorem 5.2], this is equivalent to $\text{CFI}(E_n, \text{odd}) \equiv_{\ell\text{-WL}} \text{CFI}(E_n, \text{even})$.

³⁴The complexity class MA consists of all languages L for which there exist a polynomial-time deterministic Turing machine M and polynomials p, q such that for every input string x of length $n := |x|$,

- if $x \in L$ then there exist a $z \in \{0, 1\}^{q(n)}$ with $\Pr_{y \in \{0, 1\}^{p(n)}}[M(x, y, z) = 1] \geq 2/3$,
- if $x \notin L$ then for all $z \in \{0, 1\}^{q(n)}$ we obtain $\Pr_{y \in \{0, 1\}^{p(n)}}[M(x, y, z) = 0] \geq 2/3$.

The class coMA consists of all languages whose complement is in MA .

start by showing that for **GI** the new problem **Iso-GI** is **GI**-complete (i.e., as hard as **GI**). Therefore, **Iso-GI** can be used as a source hardness within our model.

Having a source of hardness, we introduce in Section 9.3 the notion of an *isomorphism-preserving reduction*. This new kind of reduction allows us to propagate the hardness results from **Iso-GI** to other **Iso-II** problems as long as there exists an isomorphism-preserving reduction between **GI** and **II**. We then use isomorphism-preserving reductions to obtain hardness results for each problem **Iso-II** where **II** is an element of

$\mathcal{H} := \{0/1\text{-Integer Linear Programming, Gap-}k\text{-Center}_{1,2}, \text{Clique, Clique Cover, Coloring, 3-Coloring, 3D-Matching, Exact Cover, Feedback Arc Set, Feedback Vertex Set, Directed Hamiltonian Cycle, Undirected Hamiltonian Cycle, Hitting Set, IS, MaxCut, } k\text{-Median, SAT, 3-SAT, Max 2-SAT, Set Cover, Set Packing, Steiner Tree, Gap-TSP}_{1,c}, \text{Metric TSP, VC}\}.$

The problem definitions can be found in Section B.1. The hardness of most problems can be shown by utilizing standard reductions from the literature. However, to obtain hardness of **3-SAT**, we need to take a non-standard route through **3-Coloring**. For this, we need a reduction by Lovász. See Section 9.4 for more details. Finally, utilizing the hardness of **Iso-GI** again, we then show in Section 9.5 that each problem in $\mathcal{H}$ cannot be solved efficiently in our model. To be more specific, we obtain two different kind of hardness results.

We show that no problem from $\mathcal{H}$ can be solved with fast preprocessing and queries, unless **GI** $\in$ **P**.

Remark 9.2. Note that in the following theorem, we state only the dependence of the running time on n , although it may also depend on k . Here, “polynomial in n ” means polynomial in n with k held fixed. The same convention applies to space requirements and to the remaining theorems in this section.

Theorem 9.3. *Unless **GI** $\in$ **P**, for every problem $\Pi \in \mathcal{H}$, the problem Π cannot be solved inside the Isomorphic-Priors model with k prior instances each of size at most n with preprocessing and query time polynomial in n , even when $k = 1$ prior instance is given.*

Under the stronger assumption **GI** $\notin$ **coMA**, we can also rule out a data structure of polynomial size and query time.

Theorem 9.4. *Unless **GI** $\in$ **coMA**, for every problem $\Pi \in \mathcal{H}$, the problem Π cannot be solved inside the Isomorphic-Priors model with k prior instances each of size at most n with a data structure of size polynomial in n , and query time polynomial in n , even when $k = 1$ prior instance is given.*

After showing hardness for decision problems, we move on in Section 9.6 to show different hardness results for approximation problems. To this end, we first show that **Max- k -Coverage** cannot be efficiently approximated better than $1 - 1/e$ in our model, unless **Max 3-SAT** can be approximated arbitrarily well. The assumption on **Max 3-SAT** serves as our analogue to the PCP theorem. We have not been able to formulate the PCP theorem directly, because it is unclear how to define an appropriate notion of isomorphism for probabilistically checkable proofs.

Theorem 9.5. *Assume that there is an $\varepsilon > 0$ such that **Gap-Max-3-SAT** $_{1,1-\varepsilon}$ cannot be solved in the Isomorphic-Priors model with k prior instances each of size at most n with preprocessing time polynomial in k and n and query time polynomial in n , even when $k = 1$ prior instance is given.*

*Then there is an $\varepsilon' > 0$ such that **Max- k -Coverage** cannot be approximated within $(1 - 1/e + \varepsilon')$ in the Isomorphic-Priors model with preprocessing and query time polynomial in n , even when $k = 1$ prior instance is given. Also, there is an $\varepsilon'' > 0$ such that **k -Median** cannot be approximated within $(1 + 2/e - \varepsilon'')$ in the Isomorphic-Priors model with preprocessing and query time polynomial in n , even when $k = 1$ prior instance is given.*

The analogous implication can be shown for approximating **Max- k -Coverage** with a small data structure.

Theorem 9.6. *Assume that there is an $\varepsilon > 0$ such that $\text{Gap-Max-3-SAT}_{1,1-\varepsilon}$ cannot be solved in the Isomorphic-Priors model with k prior instances each of size at most n and query time polynomial in n , even when $k = 1$ prior instance is given.*

*Then there is an $\varepsilon' > 0$ such that **Max- k -Coverage** cannot be approximated within $(1 - 1/e + \varepsilon')$ in the Isomorphic-Priors model with data structure of size polynomial in n and query time polynomial in n , even when $k = 1$ prior instance is given. Also, there is an ε'' such that **k -Median** cannot be approximated better than $(1 + 2/e - \varepsilon'')$ in the Isomorphic-Priors model with data structure of size polynomial in n and query time polynomial in n , even when $k = 1$ prior instance is given.*

9.1 Hardness Results for GI

We show that GI cannot be solved in our setting with a small data structure, unless $\text{GI} \in \text{coMA}$. The following Lemma is inspired by [LT92, Corollary 4.4].

Lemma 9.7. *Unless $\text{GI} \in \text{coMA}$, GI cannot be solved in the Isomorphic-Priors model with k prior instances each having at most n many vertices with a data structure of polynomial size in n and query time polynomial in n , even when $k = 1$ prior instance is given.*

This statement is even true for a randomized data structure such that the query is correct with probability $5/6$.

Proof. Assume that there is an algorithm consisting of the preprocessing phase **Pre** and query phase **Query** in the Isomorphic-Priors model that solves GI with a data structure of size $\text{poly}_k(n)$ ³⁵ and query time $\text{poly}_k(n)$, and in particular works for $k = 1$.

We show that this implies $\overline{\text{GI}} \in \text{MA}$, and hence $\text{GI} \in \text{coMA}$.

An MA protocol for $\overline{\text{GI}}$. On a tuple of input graphs (G_0, G_1) , Merlin sends Arthur a string D of length $\text{poly}(n)$, claimed to be a valid data structure for the single prior instance

$$I^* := (G_0, G_0).$$

Arthur repeats the following test twice

1. choose a uniform random bit $b \in \{0, 1\}$ and a uniform random permutation π of $[n]$,
2. form the query instance
$$J := (G_0, \pi(G_b)),$$
3. compute $z := \text{Query}(D, J)$ and define $\hat{b} := 0$ iff $z = 1$ (otherwise $\hat{b} := 1$),
4. accept this round iff $\hat{b} = b$.

Arthur accepts iff the majority of the t rounds accept.

Completeness. Assume $G_0 \not\cong G_1$. Let Merlin send the actual data structure $D = \text{Pre}(\{I^*\})$. If $b = 0$, then there exists an isomorphism $\varphi : G_0 \rightarrow \pi(G_0)$, hence $J = (G_0, \pi(G_0))$ is isomorphic to I^* via (id, φ) and $\text{GI}(J) = 1$. Thus correctness forces $\text{Query}(D, J) = 1$, so $\hat{b} = 0 = b$ with probability at least $5/6$. If $b = 1$, then $G_0 \not\cong \pi(G_1)$, so $\text{GI}(J) = 0$ and moreover J is not isomorphic to I^* . By correctness on non-matching queries, $\text{Query}(D, J)$ is either 0 or $\perp$ with probability at least $5/6$, hence $z \neq 1$ and therefore $\hat{b} = 1 = b$ with probability at least $5/6$. Thus, each round accepts with probability at least $5/6$. Therefore, Arthur accepts with probability at least $25/36 > 2/3$.

³⁵This means that the size is polynomial in n if k is fixed.

Soundness. Assume $G_0 \cong G_1$. Then the distributions of $\pi(G_0)$ and $\pi(G_1)$ are identical, hence the random bit b is independent of the pair (D, J) , and therefore also independent of $\hat{b}$ (which is a function of (D, J) and the internal randomness of **Query**). Consequently, in each round we have $\Pr[\hat{b} = b] \leq \frac{1}{2}$, and Arthur accepts with probability at most $1/4 < 2/3$.

Therefore $\overline{\text{Gl}} \in \text{MA}$, which implies $\text{Gl} \in \text{coMA}$.

By contrapositive, unless $\text{Gl} \in \text{coMA}$, no such polynomial-size data structure algorithm for Gl exists in the Isomorphic-Priors model (even for $k = 1$). $\square$

9.2 Iso-Problems and Gl-Completeness

We introduce the promise problem $\text{Iso-}\Pi$ and show properties that will be used in later sections to show hardness results in our model.

Definition 9.8. For a decision problem Π , define the promise problem $\text{Iso-}\Pi = (\text{Iso-}\Pi_{\text{YES}}, \text{Iso-}\Pi_{\text{NO}})$ over pairs of instances (I, J) by

$$\text{Iso-}\Pi_{\text{YES}} := \{(I, J) \mid I \in \Pi \wedge J \cong I\}, \quad \text{Iso-}\Pi_{\text{NO}} := \{(I, J) \mid I \in \Pi \wedge J \notin \Pi\}.$$

The input is promised to lie in $\text{Iso-}\Pi_{\text{YES}} \cup \text{Iso-}\Pi_{\text{NO}}$.

We recall the definition of polynomial-time many-one reductions.

Definition 9.9. We say that there is a *polynomial-time many-one reduction* from Π to Π' if there exist a polynomial-time computable function f such that $X \in \Pi$ if and only if $f(X) \in \Pi'$.

We say that Π is **Gl-hard** if there is a polynomial-time many-one reduction from Gl to Π . We say that Π is **Gl-complete** if there Π is **Gl-hard** and there is a polynomial-time many-one reduction from Π to Gl .

We show that for $\Pi = \text{Gl}$, $\text{Iso-}\Pi$ is just a reformulation of Gl .

Lemma 9.10. *Iso-Gl is Gl-complete.*

Proof. Recall that Iso-Gl is the promise problem on pairs (I, J) of Gl -instances such that $I \in \text{Gl}$ and either $J \cong I$ (YES) or $J \notin \text{Gl}$ (NO).

(1) **Iso-Gl $\in \text{Gl}$.** Given an input (I, J) , where $I = (G_0, G_1)$ and $J = (H_0, H_1)$, run a Gl -test to decide whether $I \cong J$, i.e. whether $G_0 \cong H_0$ and $G_1 \cong H_1$. If $I \cong J$ output YES, otherwise output NO. This is correct under the promise: in a YES-instance we have $J \cong I$, while in a promised instance with $J \not\cong I$ we cannot be in $\text{Iso-Gl}_{\text{YES}}$, hence must be in $\text{Iso-Gl}_{\text{NO}}$.

(2) **Gl-hardness.** We give a polynomial-time many-one reduction from Gl to Iso-Gl . Given a Gl -instance $I = (G, H)$, output the pair

$$f(G, H) := ((G, G), (G, H)).$$

Note that $(G, G) \in \text{Gl}$ always. If $G \cong H$, then $(G, H) \cong (G, G)$, so $f(G, H) \in \text{Iso-Gl}_{\text{YES}}$. If $G \not\cong H$, then $(G, H) \notin \text{Gl}$, so $f(G, H) \in \text{Iso-Gl}_{\text{NO}}$. Hence f is a valid reduction and $\text{Gl} \leq_m \text{Iso-Gl}$.

Conclusion. We have shown $\text{Iso-GI} \in \text{GI}$ and $\text{GI} \leq_m \text{Iso-GI}$, therefore Iso-GI is GI -complete. $\square$

We conclude this section by showing that the promise problem is at most as hard as GI independent of the problem it is based on.

Lemma 9.11. *Let Π be a decision problem. Then $\text{Iso-}\Pi$ is in GI .*

Proof. We give a polynomial-time many-one reduction from $\text{Iso-}\Pi$ to GI .

On input (I, J) , output the pair (I, J) as an instance of GI (i.e., the question whether $I \cong J$). This mapping is polynomial time.

We claim it is correct under the promise defining $\text{Iso-}\Pi$. If $(I, J) \in \text{Iso-}\Pi_{\text{YES}}$, then $J \cong I$, hence (I, J) is a YES-instance of GI . If $(I, J) \in \text{Iso-}\Pi_{\text{NO}}$, then $I \in \Pi$ and $J \notin \Pi$. Since Π is isomorphism-invariant, $J \cong I$ would imply $J \in \Pi$, a contradiction; hence $J \not\cong I$, and (I, J) is a NO-instance of GI .

Therefore, deciding $(I, J) \in \text{GI}$ decides $(I, J) \in \text{Iso-}\Pi$, and thus $\text{Iso-}\Pi \leq_m \text{GI}$, i.e. $\text{Iso-}\Pi$ is in GI . $\square$

9.3 Isomorphism-Preserving Reductions

A tool we use to show the hardness is isomorphism-preserving reductions

Definition 9.12. We call a function R that maps instances of Π to instances of Π' an *isomorphism-preserving reduction* if:

- R is polynomial time computable.
- R is *correct*, that is $I \in \Pi$ is a YES-instance of Π if and only if $R(I)$ is a YES-instance of Π' .
- R is *isomorphism-preserving*, that is if $I_1, I_2 \in \Pi$ are isomorphic in Π then $R(I_1), R(I_2)$ are isomorphic in Π' .

We write $\Pi \leq_p^\cong \Pi'$ if such a reduction exists. Further, we say that a problem Π' is GI -hard if $\text{GI} \leq_p^\cong \Pi'$.

The property of isomorphism-preserving reductions is that they can be used to show GI -hardness of promise problems.

Lemma 9.13. *Let Π and Π' be decision problems. If $\text{Iso-}\Pi$ is GI -hard and $\Pi \leq_p^\cong \Pi'$, then $\text{Iso-}\Pi'$ is GI -hard.*

Proof. Since $\text{Iso-}\Pi$ is GI -hard, there exists a polynomial-time reduction $g : \text{GI} \leq_m \text{Iso-}\Pi$.

It therefore suffices to give a polynomial-time reduction $f : \text{Iso-}\Pi \leq_m \text{Iso-}\Pi'$.

Let (I, J) be an instance of $\text{Iso-}\Pi$. Since $\Pi \leq_p^\cong \Pi'$, there exists an isomorphism-preserving many-one reduction R from Π to Π' . Define

$$f(I, J) := (R(I), R(J)).$$

We check that f maps promised instances to promised instances and preserves YES/NO.

YES case. If $(I, J) \in \text{Iso-}\Pi_{\text{YES}}$, then $I \in \Pi$ and $J \cong I$. By correctness, $R(I) \in \Pi'$, and by isomorphism preservation, $R(J) \cong R(I)$. Hence $f(I, J) \in \text{Iso-}\Pi'_{\text{YES}}$.

NO case. If $(I, J) \in \text{Iso-}\Pi_{\text{NO}}$, then $I \in \Pi$ and $J \notin \Pi$. By correctness of R , we have $R(I) \in \Pi'$ and $R(J) \notin \Pi'$. Hence $f(I, J) \in \text{Iso-}\Pi'_{\text{NO}}$.

Thus f is a valid many-one reduction $\text{Iso-}\Pi \leq_m \text{Iso-}\Pi'$. By transitivity,

$$\text{GI} \leq_m \text{Iso-}\Pi \leq_m \text{Iso-}\Pi',$$

so $\text{Iso-}\Pi'$ is GI-hard. $\square$

Thus, by using isomorphism-preserving reductions, we can obtain GI-completeness for many different Iso- Π problems. To this end, we use the reductions shown in Figure 3. We use this property to show that the promise problems based on the problems in $\mathcal{H}$ are GI-complete.

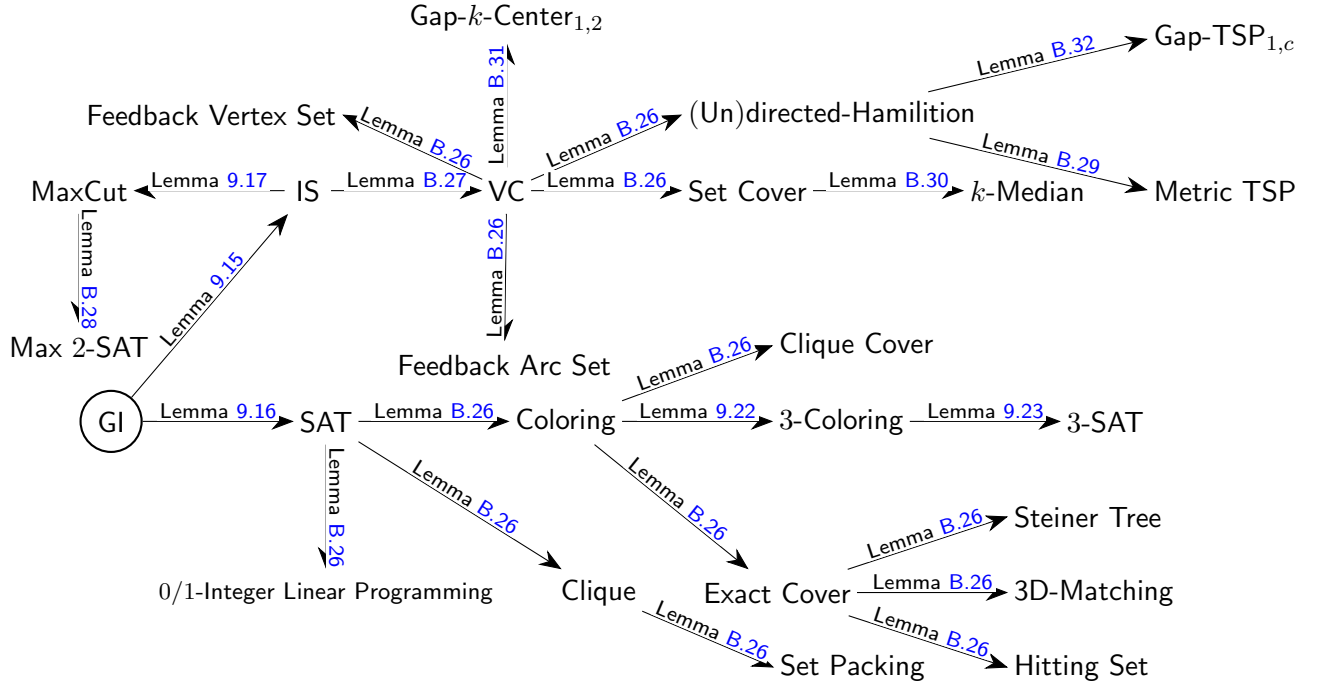


Figure 3: A tree that shows the different isomorphism-preserving reductions. An edge from A to B means $A \leq_p^{\cong} B$.

Theorem 9.14. *For every $\Pi \in \mathcal{H}$, the promise problem $\text{Iso-}\Pi$ is GI-complete.*

Proof. Fix $\Pi \in \mathcal{H}$. By Lemma 9.11, $\text{Iso-}\Pi \in \text{GI}$. Next, we use the fact that isomorphism-preserving reductions are transitive together with Lemma 9.15, Lemma 9.16, Lemma 9.17, Lemma 9.22, Lemma 9.23, Lemma B.26, Lemma B.27, Lemma B.28, Lemma B.29, Lemma B.30, Lemma B.31 and Lemma B.32. Thus, we obtain $\text{GI} \leq_p^{\cong} \Pi$ for each $\Pi \in \mathcal{H}$ (see Figure 3). By Lemma 9.10, Iso-GI is GI-hard. Hence, the theorem follows from Lemma 9.13. $\square$

The proof of Theorem 9.14 relies on having all necessary isomorphism-preserving reductions. We moved to proofs of most of our reductions to Section B since these reductions are standard reductions from the literature. However, we exemplify how our isomorphism-preserving reductions work by present three different ones in this section.

Lemma 9.15. $\text{GI} \leq_p^{\cong} \text{IS}$.

Proof. We give an isomorphism-preserving polynomial-time reduction from **GI** to the decision problem $\text{IS} = \{(K, k) \mid K \text{ has an independent set of size at least } k\}$.

Let (G, H) be an instance of **GI**. If $|V(G)| \neq |V(H)|$, output a fixed NO-instance of **IS** (e.g. $(K, 2)$ where K is a single edge). Hence assume $|V(G)| = |V(H)| = n$.

Reduction R . We construct the conflict graph $K = K(G, H)$ with

$$V(K) := V(G) \times V(H),$$

and for distinct $(u, v), (u', v') \in V(K)$ put an edge between them iff at least one of the following holds:

1. (*Function constraint*) $u = u'$ or $v = v'$.
2. (*Adjacency inconsistency*) $\mathbf{1}[(u, u') \in E(G)] \oplus \mathbf{1}[(v, v') \in E(H)] = 1$.

Define

$$R(G, H) := (K(G, H), n).$$

This is computable in polynomial time.

Correctness. We claim that

$$(G, H) \in \text{GI} \iff (K(G, H), n) \in \text{IS}.$$

If $\varphi : V(G) \rightarrow V(H)$ is an isomorphism, then $S := \{(u, \varphi(u)) : u \in V(G)\}$ is an independent set of size n in $K(G, H)$: type-(1) edges are avoided because φ is a bijection, and type-(2) edges are avoided because φ preserves adjacency. Hence $(K(G, H), n) \in \text{IS}$.

Conversely, if $K(G, H)$ has an independent set S of size n , then constraint (1) implies that S defines a bijection $\varphi : V(G) \rightarrow V(H)$ by $\varphi(u) = v$ iff $(u, v) \in S$. Independence of S implies no type-(2) edge occurs between $(u, \varphi(u))$ and $(u', \varphi(u'))$, hence $(u, u') \in E(G)$ iff $(\varphi(u), \varphi(u')) \in E(H)$ for all $u \neq u'$. Therefore φ is an isomorphism and $(G, H) \in \text{GI}$.

Isomorphism preservation. View **GI** instances as attributed graphs $\langle G, H \rangle$ (disjoint union with L/R vertex colors). If $(G, H) \cong (G', H')$, there exist isomorphisms $\psi_G : G \rightarrow G'$ and $\psi_H : H \rightarrow H'$. Define $\Psi : V(K(G, H)) \rightarrow V(K(G', H'))$ by

$$\Psi(u, v) := (\psi_G(u), \psi_H(v)).$$

Then Ψ preserves edges of type (1) (it preserves equality of coordinates) and of type (2) (since ψ_G, ψ_H preserve adjacency), hence $K(G, H) \cong K(G', H')$. Also $n = |V(G)| = |V(G')|$, so the parameter $k = n$ is preserved. Thus $R(G, H) \cong R(G', H')$ as **IS** instances.

Therefore R is an isomorphism-preserving reduction and $\text{GI} \leq_p^{\cong} \text{IS}$. $\square$

Lemma 9.16. $\text{GI} \leq_p^{\cong} \text{SAT}$.

Proof. We define a polynomial-time, isomorphism-preserving many-one reduction from **GI** to **SAT**.

Reduction. Let (G, H) be an instance of **GI**. If $|V(G)| \neq |V(H)|$, output a fixed unsatisfiable CNF (e.g. $(x) \wedge (\neg x)$). Hence assume $|V(G)| = |V(H)| = n$.

Introduce Boolean variables

$$x_{u,v} \quad \text{for each } u \in V(G), v \in V(H),$$

with the intended meaning “ $\varphi(u) = v$ ”.

We construct a CNF formula $\Phi(G, H)$ consisting of the following clauses.

Bijection constraints.

1. Each u maps somewhere:

$$\bigvee_{v \in V(H)} x_{u,v} \quad \text{for all } u \in V(G).$$

2. Each u maps to at most one v :

$$(\neg x_{u,v} \vee \neg x_{u,v'}) \quad \text{for all } u \in V(G), v \neq v' \in V(H).$$

3. Injectivity (each v is used by at most one u):

$$(\neg x_{u,v} \vee \neg x_{u',v}) \quad \text{for all } v \in V(H), u \neq u' \in V(G).$$

Adjacency consistency constraints. For every pair of distinct vertices $u, u' \in V(G)$ and distinct vertices $v, v' \in V(H)$ such that

$$\mathbf{1}[(u, u') \in E(G)] \oplus \mathbf{1}[(v, v') \in E(H)] = 1,$$

add the clause

$$(\neg x_{u,v} \vee \neg x_{u',v'}).$$

This construction is polynomial-time and yields a CNF formula of size polynomial in n .

Correctness. We show that $\Phi(G, H)$ is satisfiable if and only if $G \cong H$.

($\Rightarrow$) Suppose $\varphi : V(G) \rightarrow V(H)$ is an isomorphism. Set $x_{u, \varphi(u)} = \text{true}$ for all $u \in V(G)$ and all other variables to false. Then the bijection constraints are satisfied. For adjacency consistency, for any $u \neq u'$ let $v = \varphi(u)$ and $v' = \varphi(u')$. Since φ is an isomorphism,

$$(u, u') \in E(G) \iff (v, v') \in E(H),$$

so the XOR condition is false and no adjacency-consistency clause is violated. Hence the assignment satisfies $\Phi(G, H)$.

($\Leftarrow$) Conversely, let A satisfy $\Phi(G, H)$. By constraints (1) and (2), for each $u \in V(G)$ there is exactly one $v \in V(H)$ with $A(x_{u,v}) = \text{true}$; define $\varphi(u) = v$. Constraint (3) implies that φ is injective, and since $|V(G)| = |V(H)|$ it is a bijection. Now fix distinct $u, u' \in V(G)$ and let $v = \varphi(u)$, $v' = \varphi(u')$. If exactly one of $(u, u') \in E(G)$ and $(v, v') \in E(H)$ holds, then the corresponding clause $(\neg x_{u,v} \vee \neg x_{u',v'})$ would be falsified (both literals false), contradicting satisfaction. Therefore

$$(u, u') \in E(G) \iff (\varphi(u), \varphi(u')) \in E(H),$$

so φ is an isomorphism $G \cong H$.

Isomorphism preservation. We take “isomorphism” between SAT instances to mean: there is a variable renaming (and clause permutation) mapping one CNF to the other. If $(G, H) \cong (G', H')$ as GI instances, there exist isomorphisms $\psi_G : V(G) \rightarrow V(G')$ and $\psi_H : V(H) \rightarrow V(H')$. Define the variable renaming ρ by

$$\rho(x_{u,v}) := x_{\psi_G(u), \psi_H(v)}.$$

Then ρ maps the three families of bijection constraints for (G, H) to the corresponding families for (G', H') , and it maps each adjacency-consistency clause to the corresponding clause in $\Phi(G', H')$. Hence $\Phi(G, H) \cong \Phi(G', H')$ as SAT instances, so the reduction is isomorphism-preserving.

Therefore $\text{GI} \leq_p^{\cong} \text{SAT}$. □

Lemma 9.17. $\text{IS} \leq_p^{\cong} \text{MaxCut}$.

Proof. We use the standard gadget reduction from Independent Set to MaxCut from Barak’s online textbook [Bar23] and verify that it is also isomorphism-preserving.

Decision versions. We view **IS** as the decision problem on instances (G, k) asking whether G has an independent set of size at least k , and **MaxCut** as the decision problem on instances (H, T) asking whether H has a cut of size at least T .

Reduction. Given an instance (G, k) of **IS**, let $G = (V, E)$ and $m := |E|$. We construct a graph $H = R(G)$ as follows (cf. [Bar23]): add a distinguished vertex s^* adjacent to every $v \in V$; and for each edge $e = \{u, v\} \in E$, attach a constant-size gadget by adding two new vertices e_u, e_v and the five edges

$$\{e_u, e_v\}, \quad \{s^*, e_u\}, \quad \{s^*, e_v\}, \quad \{u, e_u\}, \quad \{v, e_v\}.$$

(No other edges are added; in particular, we do not keep the original edges of G in H .) We set the **MaxCut** threshold to

$$T := k + 4m,$$

and output the instance (H, T) . Barak proves that G has an independent set of size at least k if and only if H has a cut of size at least T , so this is a valid polynomial-time many-one reduction [Bar23].

Isomorphism preservation. Let $\varphi : G \rightarrow G'$ be a graph isomorphism and note that it preserves the parameter k and the number of edges m . We extend φ to an isomorphism $\psi : R(G) \rightarrow R(G')$ by mapping $s^* \mapsto s^*$, each original vertex $u \mapsto \varphi(u)$, and for each edge $e = \{u, v\} \in E(G)$ mapping the gadget vertices $e_u \mapsto (\varphi(e))_{\varphi(u)}$ and $e_v \mapsto (\varphi(e))_{\varphi(v)}$, where $\varphi(e) = \{\varphi(u), \varphi(v)\} \in E(G')$. Since the gadget adjacencies are defined purely in terms of incidences to s^* , u , v , and the internal gadget edge $\{e_u, e_v\}$, this mapping preserves all edges. Hence $R(G) \cong R(G')$ and the threshold T is preserved.

Therefore the reduction is isomorphism-preserving, i.e. $\text{IS} \leq_p^{\cong} \text{MaxCut}$. $\square$

9.4 The Hardness of 3-SAT

To establish the hardness of 3-SAT, we need a reduction other than the one introduced by Karp in [Kar72], as Karp's reduction is not isomorphism-preserving. Instead, we prove hardness through the following sequence of reductions: $\text{Coloring} \leq_p^{\cong} \text{3-Coloring} \leq_p^{\cong} \text{3-SAT}$

9.4.1 Lovász's Reduction for Colorability

The following construction goes back to László Lovász (see [Lov73]). We give a full description of Lovász's construction here since the paper is hard to find.³⁶ See Figure 4 for an example of the construction.

Definition 9.18 (See [Lov73]). Given a graph G with vertex set $V(G)$ and edge set $E(G)$. Further, let $k \geq 3$ be an integer. We construct a new graph $\mathcal{L}(G, k)$ with:

- $V(\mathcal{L}(G, k)) := V \uplus C \uplus W \uplus \{G_g, G_b\}$ with
 - $V := \{v^{(i)} \mid v \in V(G), i \in [k]\}$
 - $C := \{c_v^{(i)} \mid v \in V(G), i \in [k]\}$
 - $W := \{e_u^{(i)}, e_v^{(i)}, e_g^{(i)} \mid e = \{u, v\} \in E(G), i \in [k]\}$
- $E(\mathcal{L}(G, k)) := E_b \uplus E_g \uplus E_C \uplus E_W$ with
 - $E_b := \{G_b, G_g\} \cup \{\{v^{(i)}, G_b\} \mid v \in V(G), i \in [k]\}$

³⁶We refer the interested reader to the following website for further information: <https://users.ensc.concordia.ca/~chvatal/notes/color.html>

$$\begin{aligned}
& - E_g := \{\{e_g^{(i)}, G_g\} \mid e \in E(G), i \in [k]\} \\
& - E_C := \{\{c_v^{(i)}, c_v^{(i+1 \bmod k)}\} \mid v \in V(G), i \in [k]\} \cup \{\{v^{(i)}, c_v^{(i)}\} \mid v \in V(G), i \in [k]\} \\
& - \\
& E_W = \bigcup_{\substack{e \in E(G) \\ e = \{u, v\}}} \bigcup_{i=1}^k \left\{ \{e_u^{(i)}, e_v^{(i)}\}, \{e_v^{(i)}, e_g^{(i)}\}, \{e_g^{(i)}, e_u^{(i)}\}, \{e_u^{(i)}, u^{(i)}\}, \{e_v^{(i)}, v^{(i)}\} \right\}
\end{aligned}$$

We show that $\mathcal{L}(\cdot, \cdot)$ describes a correct polynomial-time many-one reduction.

Lemma 9.19. *Given a graph G and an odd integer $k \geq 3$, then $\mathcal{L}(G, k)$ satisfies the following properties:*

- $\mathcal{L}(G, k)$ has $O(k \cdot (|V(G)| + |E(G)|))$ many vertices and edges.
- $\mathcal{L}(G, k)$ can be constructed in time $O(k \cdot (|V(G)| + |E(G)|))$.
- $\mathcal{L}(G, k)$ is 3-colorable if and only if G is k -colorable.

Proof. Write $n := |V(G)|$ and $m := |E(G)|$. It is easy to check that $\mathcal{L}(G, k)$ has $O(k \cdot (n + m))$ many vertices and edges and that we can construct $\mathcal{L}(G, k)$ in time $O(k \cdot (n + m))$. In the following, we assume that $\mathcal{L}(G, k)$ is 3-colorable with a function $\tilde{c}: V(\mathcal{L}(G, k)) \rightarrow \{r, g, b\}$ (i.e., the colors red, green, blue). We show that G is k -colorable.

Without loss of generality, we assume that the vertex $G_g \in \mathcal{L}(G, k)$ is colored g and $G_b \in \mathcal{L}(G, k)$ is colored b (we can do this since G_g and G_b are connected). Next, observe that $\tilde{c}(v^{(i)}) \in \{r, g\}$ since each $v^{(i)}$ is connected to G_b . Next, we show the following claim.

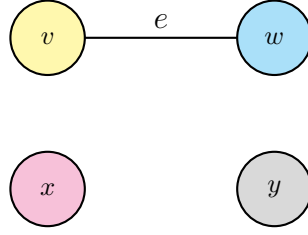
Claim 9.20. *For every $v \in V(G)$ there exists an index $i \in [k]$ such that $\tilde{c}(v^{(i)}) = g$.*

Proof. Assume otherwise, then $\tilde{c}(v^{(i)}) = r$ for all $i \in [k]$. Since k is odd, we obtain that the vertices $\{c_v^{(i)} \mid i \in [k]\}$ induce an odd cycle of size at least 3. Now, each odd cycle of size 3 is 3-colorable but not two colorable. Thus there is an index $i \in [k]$ with $\tilde{c}(c^{(i)}) = r$. However, this is not possible since $v^{(i)}$ is adjacent to $c^{(i)}$. Thus, our assumption that $\tilde{c}(v^{(i)}) = r$, for all $i \in [k]$, is wrong and therefore there exist an $i \in [k]$ with $\tilde{c}(v^{(i)}) = g$. $\square$

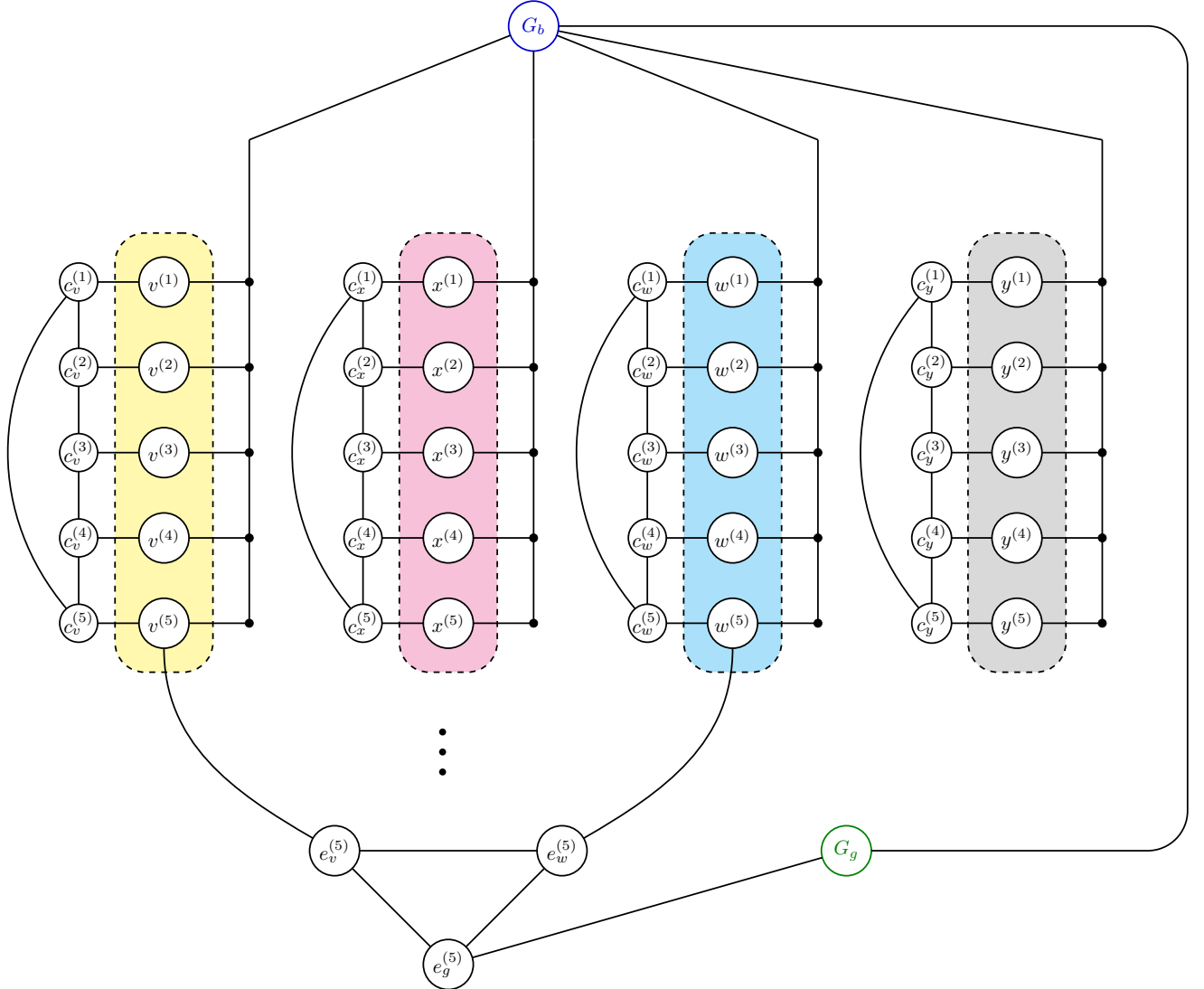
Now, Claim 9.20 yields a function $f: V(G) \rightarrow [k]$ that maps each vertex v to the smallest value $i \in [k]$ with $\tilde{c}(v^{(i)}) = g$. We show that f is a k -coloring for G . Assume that f is not a proper 3-coloring then there is an edge $e = \{u, v\} \in E(G)$ with $f(u) = f(v) = i$. By construction, we obtain $\tilde{c}(v^{(i)}) = g = \tilde{c}(u^{(i)})$. Further, note that $\tilde{c}(G_g) = g$. Thus, each vertex in the triangle $\{e_u^{(i)}, e_v^{(i)}, e_g^{(i)}\}$ is adjacent to a vertex with color g . Hence, we can only use the colors $\{r, b\}$ to color $\{e_u^{(i)}, e_v^{(i)}, e_g^{(i)}\}$ which is not possible since the vertices form a triangle in $\mathcal{L}(G, k)$. Therefore, $\tilde{c}$ is not a proper 3-coloring which is a contradiction.

Next, let $f: V(G) \rightarrow [k]$ be a k -coloring for G . We define the the following mapping on the subset $V' := \{G_g, G_b\} \cup \{v^{(i)} \mid v \in V(G), i \in [k]\}$ of vertices of $\mathcal{L}(G, k)$.

$$c': V' \rightarrow \{r, g, b\}, x \mapsto \begin{cases} g & \text{if } x = G_g \\ b & \text{if } x = G_b \\ g & \text{if } x = v^{(i)} \text{ and } f(v) = i \\ r & \text{if } x = v^{(i)} \text{ and } f(v) \neq i \end{cases}$$



(a) A undirected graph G with 4 vertices. Note that the vertices are actually uncolored.



(b) The graph $\mathcal{L}(G, 5)$. Note that we only show one of the five edge gadgets for the edge $e \in E(G)$.

Figure 4: Example of G and $\mathcal{L}(G, 5)$

We show that c' can be extended to a proper 3-coloring that is defined on all vertices of $\mathcal{L}(G, k)$. First note that under c' no two adjacent vertices in V' have the same color (the only relevant case is between G_g and $v^{(i)}$ with $f(v) = i$ which are non-adjacent by construction).

Let $v \in V(G)$ then $\{c_v^{(i)} \mid i \in [k]\}$ induces an odd cycle in $\mathcal{L}(G, k)$. By construction there are vertices $v^{(s)}$ that are colored r and a vertex $v^{(t)}$ that is colored g . Thus, we have three colors available to color the odd cycle $\{c_v^{(i)} \mid i \in [k]\}$.

Now, let $e = \{u, v\} \in E(G)$ and $i \in [k]$ then $\{e_u^{(i)}, e_v^{(i)}, e_g^{(i)}\}$ induces a triangle in $\mathcal{L}(G, k)$. Now, by construction not every node in $\{e_u^{(i)}, e_v^{(i)}, e_g^{(i)}\}$ can be adjacent to a vertex that is colored g . Otherwise, $f(v) = i = f(u)$ which is not possible. Thus, we have three colors available to color the triangle $\{e_u^{(i)}, e_v^{(i)}, e_g^{(i)}\}$. Meaning that we can extend c' . $\square$

The reduction $\mathcal{L}(\cdot, \cdot)$ is isomorphism-preserving.

Lemma 9.21. *Let G and H be two graphs and $k \geq 3$ be an odd positive integer. If $G \cong H$ then $\mathcal{L}(G, k) \cong \mathcal{L}(H, k)$.*

Proof. Let $\varphi: V(G) \rightarrow V(H)$ be an isomorphism from G to H . Write $V(\mathcal{L}(G, k)) := V \uplus C \uplus W \uplus \{G_g, G_b\}$. For $v \in V(G)$, we write $\tilde{v} := \varphi(v)$. For $e \in \{u, v\}$, we write $\tilde{e} := \{\varphi(u), \varphi(v)\} \in E(H)$. Note that $v \rightarrow \tilde{v}$ and $e \rightarrow \tilde{e}$ both define one-to-one mappings between $V(G)$ and $V(H)$, and between $E(G)$ and $E(H)$, respectively. We define

$$\psi: V(\mathcal{L}(G, k)) \rightarrow V(\mathcal{L}(H, k)), x \mapsto \begin{cases} H_g & \text{if } x = G_g \\ H_b & \text{if } x = G_b \\ \tilde{v}^{(i)} & \text{if } x = v^{(i)} \\ c_{\tilde{v}}^{(i)} & \text{if } x = c_v^{(i)} \\ \tilde{e}_v^{(i)} & \text{if } x = e_v^{(i)} \\ \tilde{e}_b^{(i)} & \text{if } x = e_b^{(i)} \end{cases}.$$

We show that ψ defines an isomorphism. First, note that ψ is a bijection. Let us show that $\{x, y\} \in E(\mathcal{L}(G, k))$ if and only if $\{\psi(x), \psi(y)\} \in E(\mathcal{L}(H, k))$. By construction we already know that $|E(\mathcal{L}(G, k))| = |E(\mathcal{L}(H, k))|$, thus it is actually enough to show that $\{x, y\} \in E(\mathcal{L}(G, k))$ implies $\{\psi(x), \psi(y)\} \in E(\mathcal{L}(H, k))$.³⁷

We consider a case distinction.

- Case $x = G_b$ or $y = G_b$: Without loss of generality, we assume $x = G_b$. If $\{G_b, y\} \in \mathcal{L}(G, k)$ then either $y = G_g$ or $y = v^{(i)}$ thus $\{H_g, \psi(y)\} \in E(\mathcal{L}(H, k))$.
- Case $(x = G_g \text{ or } y = G_g)$ and $x \neq G_b$ and $y \neq G_b$: Without loss of generality, we assume $x = G_g$. If $\{G_g, y\} \in \mathcal{L}(G, k)$ then $y = e_g^{(i)}$ thus $\{\psi(G_g), \psi(y)\} = \{H_g, \tilde{e}_g^{(i)}\} \in E(\mathcal{L}(H, k))$.
- Case $x, y \in V \uplus C \uplus W$ and $(x \in V, y \in C \text{ or } x \in C, y \in W)$: Without loss of generality, we assume $x = v^{(i)}$ and $y = c_u^{(j)}$ for some $u, v \in V$ and $i, j \in [k]$. If $\{v^{(i)}, c_u^{(j)}\} \in \mathcal{L}(G, k)$ then $v = u$ and $i = j$. Thus, $\{\psi(x), \psi(y)\} = \{\tilde{v}^{(i)}, c_{\tilde{v}}^{(i)}\} \in \mathcal{L}(H, k)$.
- Case $x, y \in V \uplus C \uplus W$ and $(x \in V, y \in W \text{ or } x \in W, y \in V)$: Without loss of generality, we assume $x = v^{(i)}$ and $y = e_c^{(j)}$ for some $v \in V$, $e = \{a, b\} \in E(G)$ and $i, j \in [k]$. If

³⁷This implies that the mapping $\psi_E: E(\mathcal{L}(G, k)) \rightarrow E(\mathcal{L}(H, k)), \{x, y\} \mapsto \{\psi(x), \psi(y)\}$ is well-defined. Further, ψ_E is injective since ψ is. Lastly, $|E(\mathcal{L}(G, k))| = |E(\mathcal{L}(H, k))|$ yields that ψ_E is a bijection and thus ψ is an isomorphism.

$\{v^{(i)}, e_c^{(j)}\} \in \mathcal{L}(G, k)$ then $v = c$, $i = j$ and either $a = v$ or $b = v$. Thus, $\{\psi(x), \psi(y)\} = \{\tilde{v}^{(i)}, \tilde{e}_v^{(i)}\} \in \mathcal{L}(H, k)$.

- Case $x, y \in V \uplus C \uplus W$ and $x \in C$ and $y \in C$: Without loss of generality, we assume $x = c_v^{(i)}$ and $y = c_u^{(j)}$ for $u, v \in V$ and $i, j \in [k]$. If $\{c_v^{(i)}, c_u^{(j)}\} \in \mathcal{L}(G, k)$ then $u = v$ and either $j = i + 1$ or $i = j + 1$. Without loss of generality, assume $j = i + 1$, now $\{\psi(x), \psi(y)\} = \{c_v^{(i)}, c_v^{(i+1)}\} \in \mathcal{L}(H, k)$.
- Case $x, y \in V \uplus C \uplus W$ and $x \in E$ and $y \in E$: Without loss of generality, we assume $x = e_c^{(i)}$ and $y = d_{c'}^{(j)}$ for some $e, d \in E(G)$, $i, j \in [k]$. If $\{e_c^{(i)}, d_{c'}^{(j)}\} \in \mathcal{L}(G, k)$ then $e = d$ and $i = j$ and $c \neq c'$. Thus, $\{\psi(x), \psi(y)\} = \{\tilde{e}_c^{(i)}, \tilde{e}_{c'}^{(i)}\} \in \mathcal{L}(H, k)$.
- Otherwise: In all remaining cases we have $x, y \in V \uplus C \uplus W$ with $\{x, y\} \notin E(\mathcal{L}(G, k))$. For example, if $x, y \in V$ then $\psi(x) = \tilde{v}^{(i)}$ and $\psi(y) = \tilde{u}^{(j)}$ for $v, u \in V$ and $i, j \in [k]$. Note that $\{\tilde{v}^{(i)}, \tilde{u}^{(j)}\} \notin E(\mathcal{L}(H, k))$.

This shows that ψ is an isomorphism which proves that lemma. $\square$

We extend Lovász's construction by handling even values of k and conclude a isomorphism-preserving reduction to 3-Coloring.

Lemma 9.22. $\text{Coloring} \leq_p^{\cong} \text{3-Coloring}$.

Proof. We first define a helper function $A: \mathcal{G} \rightarrow \mathcal{G}$ that takes as input an graph G and outputs a graph G' that is obtained by adding a new vertex x to $V(G)$ and connecting x to all vertices in G . Observe that $A(G)$ is computable in time $O(|V(G)| + |E(G)|)$. Further, note that $A(G)$ is $(k + 1)$ -colorable if and only if G is k -colorable. Also note that $G \cong H$ if and only if $A(G) \cong A(H)$. Next, we define

$$f: \mathcal{G} \times \mathbb{N}_{\geq 3} \rightarrow \mathcal{G}, (G, k) \mapsto \begin{cases} \mathcal{L}(G, k) & \text{if } k \text{ is odd} \\ \mathcal{L}(A(G), k + 1) & \text{if } k \text{ is even} \end{cases}.$$

We show that f defines a polynomial time many-one reduction from Coloring to 3-Coloring is isomorphism-preserving. First, observe that f is polynomial time computable due to Lemma 9.19. Next, let G be a graph and k be a number, we show $(G, k) \in \text{Coloring}$ if and only if $f(G, k) \in \text{3-Coloring}$. If k is odd then this directly follows from Lemma 9.19. If k is even then G is k -colorable if and only if $A(G)$ is $(k + 1)$ -colorable. Now, Lemma 9.19 yields that $A(G)$ is $(k + 1)$ -colorable if and only if $\mathcal{L}(A(G), k + 1)$ is 3-colorable, proving the claim.

Lastly, we show that f is isomorphism-preserving. To this end, let $k \geq 3$ be an integer and G, H be graphs with $G \cong H$. If k is odd then Lemma 9.21 implies $f(G, k) \cong f(H, k)$. If k is even then we obtain $A(G) \cong A(H)$. Again, Lemma 9.21 yields $f(G, k) \cong f(H, k)$, proving the lemma. $\square$

9.4.2 Reduction from 3-Coloring to 3-SAT

We give an isomorphism-preserving reduction from 3-Coloring to 3-SAT.

Lemma 9.23. $\text{3-Coloring} \leq_p^{\cong} \text{3-SAT}$.

Proof. We define a polynomial-time, isomorphism-preserving many-one reduction from $\text{3-Coloring} \leq_p^{\cong} \text{3-SAT}$.

Reduction. let G be a graph, we define a 3-CNF $f(G)$ in the following way:

- For each $v \in V(G)$ we define variables v_r, v_g, v_b . For each edge $e \in E(G)$, we define variables d_e^1, d_e^2, d_e^3 . Thus, $f(G)$ has $3(|V(G)| + |E(G)|)$ many variables.
- For each $v \in V(G)$, we define the clause $(v_r \vee v_g \vee v_b)$.

For each $e = \{u, v\} \in E(G)$ and $c \in \{r, g, b\}$, we define the clause $(\neg v_c \vee \neg u_c \vee d_e^1)$. Further, we define the clauses

$$(\neg d_e^1 \vee d_e^2 \vee d_e^3), (\neg d_e^1 \vee d_e^2 \vee \neg d_e^3), (\neg d_e^1 \vee \neg d_e^2 \vee d_e^3), (\neg d_e^1 \vee \neg d_e^2 \vee \neg d_e^3).$$

Note that these clauses are constructed in a way that ensures $d_e^1 = 0$. This is done to reduce $(\neg v_c \vee \neg u_c \vee d_e^1)$ to $(\neg v_c \vee \neg u_c)$. In total $f(G)$ has $|V(G)| + 7|E(G)|$ many clauses.

Note that $f(G)$ is a 3-CNF that is obviously computable in time $O(|V(G)| + |E(G)|)$.

Correctness. We show that G is 3-colorable if and only if $f(G)$ is solvable.

Let $\tilde{c}: V(G) \rightarrow \{r, g, b\}$ be a valid 3-coloring of G . We use $\tilde{c}$ to construct a satisfying assignment for $f(G)$. We define an assignment $\mathbf{x}$ via

$$x \mapsto \begin{cases} 1 & \text{if } x = v_c \text{ for } c \in \{r, g, b\} \text{ and } \tilde{c}(v) = c \\ 0 & \text{otherwise} \end{cases}.$$

Now, let $e = \{u, v\} \in E(G)$ and assume that $\mathbf{x}$ does not satisfy $(\neg v_c \vee \neg u_c \vee d_e^1)$. Since $d_e^1 = 0$, this is only possible if $v_c = 1$ and $u_c = 1$. However, this would imply that $\tilde{c}(v) = \tilde{c}(u)$ which is not possible because $\tilde{c}$ is a proper 3-coloring. Thus, $\mathbf{x}$ satisfies $(\neg v_c \vee \neg u_c \vee d_e^1)$. Further, it is easy to check that $\mathbf{x}$ satisfies all other constraints. Hence, $f(G)$ is satisfiable.

Next, let $\mathbf{x}$ be a satisfying assignment for $f(G)$. Note that $\mathbf{x}$ can only satisfy the constraints

$$(\neg d_e^1 \vee d_e^2 \vee d_e^3), (\neg d_e^1 \vee d_e^2 \vee \neg d_e^3), (\neg d_e^1 \vee \neg d_e^2 \vee d_e^3), (\neg d_e^1 \vee \neg d_e^2 \vee \neg d_e^3),$$

by setting $d_e^1 = 0$ for each $e \in E(G)$. We show how to construct a 3-coloring for G using $\mathbf{x}$. Due to the constraint $(v_r \vee v_g \vee v_b)$, for each $v \in V(G)$ there is a $c \in \{r, g, b\}$ with $v_c = 1$ with respect to $\mathbf{x}$. We define $\tilde{c}(v) = c$ (if $v_c = 1$ is true for multiple $c \in \{r, g, b\}$ then we choose arbitrarily). We show that $\tilde{c}$ defines a proper 3-coloring on $V(G)$. Assume otherwise, then there is a edge $e = \{u, v\} \in E(G)$ with $\tilde{c}(v) = c = \tilde{c}(u)$. However, since $d_e^1 = 0$, this would yield that the constraint $(\neg v_c \vee \neg u_c \vee d_e^1)$ is false which is a contradiction. Thus, $\tilde{c}$ defines a proper 3-coloring.

Isomorphism preservation. We show that if G, H are two graphs with $G \cong H$ then $f(G) \cong f(H)$.

Let $\varphi: V(G) \rightarrow V(H)$ be an isomorphism between G and H . For $v \in V(G)$, we write $\tilde{v} := \varphi(v)$. For $e \in E(G)$, we write $\tilde{e} := \{\varphi(u), \varphi(v)\} \in E(H)$. Note that $v \rightarrow \tilde{v}$ and $e \rightarrow \tilde{e}$ both define one-to-one mappings between $V(G)$ and $V(H)$, and between $E(G)$ and $E(H)$, respectively. We write $V(f(G))$ for the set of variables in $f(G)$. Let

$$\psi: V(f(G)) \rightarrow V(f(H)), x \mapsto \begin{cases} \tilde{v}_c & \text{if } x = v_c \\ d_e^i & \text{if } x = d_e^i \end{cases}.$$

It is now easy to check that ψ is an isomorphism between $E(G)$ and $E(H)$.

Thus, f defines a polynomial time many-one reduction from 3-Coloring to 3-SAT is isomorphism-preserving. Hence, $3\text{-Coloring} \leq_p^{\cong} 3\text{-SAT}$. $\square$

9.5 Hardness in the Isomorphism-Priors Model

We show that the GI -hardness of Iso-II implies that Π cannot be solved in our model with fast preprocessing, unless $\text{GI} \in \text{P}$.

Lemma 9.24. *Let Π be a decision problem such that Iso-II is GI -hard. Unless $\text{GI} \in \text{P}$, Π cannot be solved inside the Isomorphic-Priors model with k prior instances each of size at most n with preprocessing and query time polynomial in n , even when $k = 1$ prior instance is given.*

Proof. Assume that there exists an algorithm for Π in the Isomorphic-Priors model with preprocessing time $\text{poly}_k(n)$ and query time $\text{poly}_k(n)$, and that it is correct on all valid queries. In particular, consider the case $k = 1$.

We show that this implies $\text{Iso-II} \in \text{P}$. Given an input (G, H) to Iso-II , run the model's preprocessing phase on the single prior instance G together with $L = \Pi(G)$ (i.e., $L = 1$ if and only if G is a YES-instance), obtaining a data structure D . Then query the model on H and obtain an output

$$z := \text{Query}(D, H) \in \{0, 1, \perp\}.$$

We decide Iso-II by outputting YES iff $z = 1$, and NO otherwise (in particular, treat $z = \perp$ as NO). Correctness holds under the promise defining Iso-II :

- If $(G, H) \in \text{Iso-II}_{\text{YES}}$, then $G \in \Pi$ and $H \cong G$. Since Π is invariant under isomorphism, $\Pi(H) = \Pi(G) = 1$, so the model must output $z = \Pi(H) = 1$, and we accept.
- If $(G, H) \in \text{Iso-II}_{\text{NO}}$, then $G \in \Pi$ and $H \notin \Pi$, i.e. $\Pi(H) = 0$. By correctness of the model algorithm on non-matching queries, on input H it outputs either $z = \Pi(H) = 0$ or the special symbol $\perp$. In both cases we reject.

The running time is polynomial, since preprocessing on one instance and one query are polynomial in n .

Thus $\text{Iso-II} \in \text{P}$. Since by assumption Iso-II is GI -hard, this implies $\text{GI} \in \text{P}$. Therefore, unless $\text{GI} \in \text{P}$, no such algorithm exists for Π in the Isomorphic-Priors model (even for $k = 1$). $\square$

We show that a problem, that can be reduced to GI with an isomorphism-preserving reduction, cannot be solved in our model with a small data structure, unless $\text{GI} \in \text{coMA}$.

Lemma 9.25. *Let Π be a decision problem such that $\text{GI} \leq_p^{\cong} \Pi$. Assume $\text{GI} \notin \text{coMA}$. Then Π cannot be solved inside the Isomorphic-Priors model with k prior instances each of size at most n with a data structure of size polynomial in n and query time polynomial in n , even when $k = 1$ prior instance is given.*

Proof. Assume for contradiction that Π can be solved in the Isomorphic-Priors model with preprocessing and query time polynomial and with a data structure of size $\text{poly}_k(n)$.

Since $\text{GI} \leq_p^{\cong} \Pi$, there exists an isomorphism-preserving many-one reduction R from GI -instances to Π -instances such that for every GI -instance I ,

$$I \in \text{GI} \iff R(I) \in \Pi.$$

We build an algorithm that solves GI in the Isomorphic-Priors model with a polynomial-size data structure (already for $k = 1$), contradicting Lemma 9.7 unless $\text{GI} \in \text{coMA}$.

GI preprocessing (one prior instance). Given a single prior GI-instance $I = (G_1, G_2)$ together with $L = \text{GI}(I)$,³⁸ compute the corresponding Π -instance $X := R(I)$. Since R is a many-one reduction, we have $\Pi(X) = \text{GI}(I) = L$,³⁹ so we can use the same value L for X . Run Π -preprocessing on the singleton database $\{(X, L)\}$ to obtain a data structure D .

GI query. Given a query GI-instance J , compute the Π -instance $Y := R(J)$ and query the Π -algorithm to get

$$z := \text{Query}_\Pi(D, Y) \in \{0, 1, \perp\}.$$

Output z .

Correctness. If $J \cong I$, then by isomorphism preservation $Y \cong X$, hence the query is matching and the Π -algorithm must output $z = \Pi(Y) = \text{GI}(J)$, so we output the correct answer. If $J \not\cong I$, then either the Π -algorithm outputs $\Pi(Y)$ or $\perp$; in either case, the above rule outputs the correct value for $\text{GI}(J)$ or $\perp$, both are correct outputs for J in the Isomorphic-Priors model.

Thus we obtain a GI solver in the Isomorphic-Priors model with a polynomial-size data structure, contradicting Lemma 9.7 unless $\text{GI} \in \text{coMA}$. $\square$

Proof of Theorem 9.3. Fix $\Pi \in \mathcal{H}$ and assume for contradiction that Π can be solved in the Isomorphic-Priors model with preprocessing time polynomial in k and n and query time polynomial in n , even for $k = 1$.

By Theorem 9.14, the promise problem Iso- Π is GI-hard. Applying Lemma 9.24, we conclude that $\text{GI} \in \text{P}$.

Taking the contrapositive, unless $\text{GI} \in \text{P}$ no such Isomorphic-Priors model algorithm for Π exists (even for $k = 1$). Since Π was arbitrary in the set $\mathcal{H}$, the claim follows for all three problems. $\square$

Proof of Theorem 9.4. Fix $\Pi \in \mathcal{H}$ and assume for contradiction that Π can be solved in the Isomorphic-Priors model with a data structure of size $\text{poly}(k, n)$.

By Lemma 9.25, it suffices to show that $\text{GI} \leq_p^\cong \Pi$. For $\Pi = \text{IS}$ this holds by Lemma 9.15, and for $\Pi = \text{SAT}$ it holds by Lemma 9.16. For $\Pi = \text{MaxCut}$, we have $\text{GI} \leq_p^\cong \text{IS}$ by Lemma 9.15 and $\text{IS} \leq_p^\cong \text{MaxCut}$ by Lemma 9.17; by composition of isomorphism-preserving reductions, $\text{GI} \leq_p^\cong \text{MaxCut}$.

Hence in all three cases $\text{GI} \leq_p^\cong \Pi$, and Lemma 9.25 implies $\text{GI} \in \text{coMA}$, contradicting the assumption. Therefore, unless $\text{GI} \in \text{coMA}$, no such polynomial-size data structure algorithm for Π exists (even for $k = 1$). $\square$

9.6 Proof of Hardness for Gap-Max- k -Coverage

In this section, we show that Max- k -Coverage cannot be approximated better than $(1 - 1/e)$ in our model, unless Max 3-SAT can be approximated arbitrarily well in our model. We achieve this by first observing that Feige's k -prover system works for general Max 3-SAT and is not limited to bounded-occurrences. This is necessary because bounded occurrences would imply that the underlying isomorphism problem can be solved fast. We then go on and show that this implies a isomorphism-preserving reduction.

Lemma 9.26 (Feige's k -prover system for general Max 3-SAT). *Consider the k -prover proof system of Section 2.3 in [Fei98] applied to an arbitrary 3CNF formula ϕ (no bounded-occurrence assumption). Let $\varepsilon \in (0, 1)$.*

³⁸That is, $L = 1$ if $G_1 \cong G_2$ and zero, otherwise.

³⁹Here, $\Pi(x) = 1$ if $x \in \Pi$ and zero otherwise.

1. If ϕ is satisfiable, then there is a prover strategy that causes the verifier to always strongly accept.
2. If $OPT(\phi) \leq 1 - \varepsilon$ (i.e., no assignment satisfies more than a $(1 - \varepsilon)$ -fraction of clauses), then the verifier weakly accepts with probability at most $k^2 \cdot 2^{-c\ell}$, where $c > 0$ depends only on ε (and on the constant answer length), and ℓ is the repetition parameter of the verifier.

Proof. The verifier samples ℓ clauses uniformly and independently and chooses one distinguished variable per clause, exactly as in [Fei98, Sec. 2.3]. The weak predicate is “some pair of provers is consistent” (on the induced assignment to the ℓ distinguished-variable positions), and the strong predicate is “every pair is consistent” [Fei98, Sec. 2.3].

Completeness. If ϕ is satisfiable, fix any satisfying assignment and let every prover answer consistently with it. Then the induced assignments to the distinguished-variable positions are identical for all provers, so the verifier always strongly accepts (identical to Feige’s argument) [Fei98, Lem. 2.3.1].

Soundness: reducing to two provers and parallel repetition. Assume $OPT(\phi) \leq 1 - \varepsilon$ and suppose the verifier weakly accepts with probability at least δ . Then there exist two provers whose answers are consistent with probability at least δ/k^2 (by averaging over the $\binom{k}{2}$ pairs), exactly as in Feige [Fei98, Lem. 2.3.1]. By the Hamming-distance property of the code used to distribute queries, there are at least $\ell/6$ coordinates in which one of these provers receives a clause and the other receives the distinguished variable from that clause (again exactly as in Feige) [Fei98, Lem. 2.3.1]. Fix the questions on the remaining $5\ell/6$ coordinates to maximize acceptance; the acceptance probability remains at least δ/k^2 . Now ignore those fixed coordinates. This yields a strategy for a two-prover system that succeeds with probability at least δ/k^2 on $\ell/6$ independent coordinates, i.e., on $\ell/6$ parallel repetitions of the underlying two-prover test.

Base two-prover error does not need bounded occurrence. Feige’s Proposition 2.2.1 (stated for 3CNF-5) bounds the optimal acceptance probability of the base two-prover test by $1 - \varepsilon/3$ when at most a $(1 - \varepsilon)$ -fraction of clauses are simultaneously satisfiable [Fei98, Prop. 2.2.1]. Its proof uses only: (i) the second prover’s strategy induces a global assignment x , (ii) a random clause is unsatisfied by x with probability at least ε , and (iii) conditioned on such a clause, if the first prover makes it satisfied by changing at least one literal, then a uniformly random distinguished variable from that clause disagrees with probability at least $1/3$. This argument does not depend on how many times a variable appears in the formula (variables may occur unboundedly often).

Hence the base two-prover system has constant error at most $1 - \varepsilon/3$, independent of n and of occurrence bounds.

Parallel repetition. By Raz’s parallel repetition theorem (Feige’s Theorem 2.2.2), repeating a one-round two-prover system t times independently in parallel reduces the acceptance probability to at most 2^{-ct} for some constant $c > 0$ that depends only on the base error and the constant answer length [Fei98, Thm. 2.2.2]. Taking $t = \ell/6$, we obtain

$$\delta/k^2 \leq 2^{-c\ell},$$

after renaming constants, and thus $\delta \leq k^2 \cdot 2^{-c\ell}$.

This proves the claimed soundness bound for arbitrary 3CNF formulas, without any bounded-occurrence assumption. $\square$

This k -prover systems can be used for an isomorphism-preserving reduction.

Lemma 9.27. *For every $\varepsilon, \varepsilon' \in (0, 1/3)$, there is an isomorphism-preserving reduction*

$$\text{Gap-Max-3-SAT}_{1, 1-\varepsilon} \leq_p^{\cong} \text{Gap-Max-}k\text{-Coverage}_{1, 1-1/e+\varepsilon'}.$$

Proof. Fix $\varepsilon \in (0, 1/3)$. We construct an isomorphism-preserving mapping from 3CNF formulas to Max- k -Coverage instances, following Feige [Fei98], and use Lemma 9.26 (Feige’s Lemma 2.3.1 without bounded-occurrence) as the only nontrivial black box about the underlying k -prover system.

No regularization. We do *not* apply any bounded-occurrence / regularization preprocessing: such steps typically require non-canonical choices (e.g., ordering occurrences), and are therefore not isomorphism-preserving in general.

Step 1: The reduction $\Phi \mapsto (U(\Phi), \mathcal{S}(\Phi), k')$. Let Φ be a 3CNF instance. Consider the k -prover verifier from [Fei98, Sec. 2.3] with repetition parameter ℓ (to be chosen later as a function of ε). Let $\mathcal{R}$ be the set of random strings of the verifier; write $R := |\mathcal{R}|$. For each $r \in \mathcal{R}$, the verifier determines (for each prover $i \in [k]$) a question $q_i(r)$ (a tuple of clause/variable identifiers as in [Fei98, Sec. 2.3]). Let Q be the number of possible questions a prover can receive (as in Feige).

Partition systems. Fix parameters (m, L, k, d) as in Feige’s partition-system lemma (Lemma 3.2 in [Fei98]), with $L = 2^\ell$. For each $r \in \mathcal{R}$, create an independent copy of the partition system

$$B_r := B(m, L, k, d)$$

on a ground set of size m , equipped with L partitions indexed by strings $p \in \{0, 1\}^\ell$. Each such partition consists of k disjoint parts, which we denote

$$\mathcal{P}_{r,p} = \{B(r, p, 1), \dots, B(r, p, k)\}, \quad \bigcup_{i=1}^k B(r, p, i) = B_r.$$

Universe. Define the universe as the disjoint union of all these ground sets:

$$U(\Phi) := \bigsqcup_{r \in \mathcal{R}} (\{r\} \times B_r),$$

so $|U(\Phi)| = mR$.

Sets. For each prover $i \in [k]$, each question q that prover i may receive, and each possible answer a to that question, define a set $S(q, a, i) \subseteq U(\Phi)$ by

$$S(q, a, i) := \bigcup_{\substack{r \in \mathcal{R}: \\ q_i(r) = q}} (\{r\} \times B(r, p_r(a), i)),$$

where $p_r(a) \in \{0, 1\}^\ell$ is the induced assignment to the ℓ distinguished variables selected under randomness r (exactly as in Feige’s reduction). Let

$$\mathcal{S}(\Phi) := \{S(q, a, i) : i \in [k], q \in \mathcal{Q}_i, a \in \mathcal{A}_{i,q}\}.$$

Budget. Set

$$k' := k^Q,$$

corresponding to choosing, for each prover, one answer for each of its Q possible questions (as in Feige’s Max- k -Cover hardness, Section 5 of [Fei98]).

We output the Max- k -Coverage instance

$$R(\Phi) := (U(\Phi), \mathcal{S}(\Phi), k').$$

Step 2: Gap correctness (black-box). We argue informally, following Feige.

YES case. If $OPT(\Phi) = 1$, then by Lemma 9.26 there is a prover strategy that causes the verifier to *strongly* accept on every random string. Fix that strategy and choose the k' sets that correspond to the provers' answers on all possible questions (one answer per question per prover, hence k^Q choices total). For each r , strong acceptance implies that the induced strings $p_r(\cdot)$ are consistent across provers, and therefore the chosen sets cover, inside the block $\{r\} \times B_r$, all m points (because the k parts of $\mathcal{P}_{r,p}$ partition B_r). Hence the chosen k' sets cover all of $U(\Phi)$, so $OPT(R(\Phi)) = 1$.

NO case. If $OPT(\Phi) \leq 1 - \varepsilon$, then by Lemma 9.26 the verifier weakly accepts with probability at most δ , where δ can be made exponentially small in ℓ . Feige's analysis of the reduction (using the partition-system lemma) shows that any family of k' sets corresponds to a (possibly inconsistent) prover strategy, and that the fraction of universe elements covered is at most $1 - 1/e + O(\delta) + O(1/d)$ (cf. [Fei98, Sec. 4–5]). Choosing ℓ and d large enough as functions of ε makes the additive terms $O(\delta) + O(1/d)$ arbitrarily small; in particular, we can choose parameters so that the covered fraction is at most $1 - 1/e + \varepsilon'$. Therefore $OPT(R(\Phi)) \leq 1 - 1/e + \varepsilon'$ in the NO case.

Altogether, $\Phi \in \text{Gap-Max-3-SAT}_{1,1-\varepsilon'}$ implies $R(\Phi) \in \text{Gap-Max-}k\text{-Coverage}_{1,1-1/e+\varepsilon'}$.

Step 3: Isomorphism preservation. Let $\alpha : \Phi \rightarrow \Phi'$ be an isomorphism of 3CNF instances (renaming variables and permuting clauses, preserving literal signs). We build an isomorphism between $R(\Phi) = (U, \mathcal{S}, k')$ and $R(\Phi') = (U', \mathcal{S}', k')$.

The verifier's random string $r \in \mathcal{R}$ specifies a pattern of queries (which clause indices are chosen, which variables are distinguished, etc.). Under α , clause/variable identifiers are renamed, and hence each question q and answer a is renamed canonically to $\alpha(q)$ and $\alpha(a)$. Define a bijection $\beta_U : U \rightarrow U'$ by

$$\beta_U(r, x) := (r, x) \quad \text{for } x \in B_r,$$

i.e. we keep the partition-system point and randomness index, but interpret the clause/variable identifiers inside the indexing data via α (equivalently: we identify $\mathcal{R}$ across Φ and Φ' by the same randomness and query pattern, differing only by renamed identifiers). Define a bijection $\beta_S : \mathcal{S} \rightarrow \mathcal{S}'$ by

$$\beta_S(S(q, a, i)) := S(\alpha(q), \alpha(a), i).$$

By construction, for every universe element (r, x) and every set $S(q, a, i)$, we have

$$(r, x) \in S(q, a, i) \iff (r, x) \in S(\alpha(q), \alpha(a), i),$$

because membership is defined solely by (i) whether $q_i(r) = q$ and (ii) which part $B(r, p_r(a), i)$ is selected, and both are invariant under renaming clause/variable identifiers. Hence (β_U, β_S) preserves incidence, so it is an isomorphism of set systems. Therefore $R(\Phi) \cong R(\Phi')$.

Conclusion. The mapping $\Phi \mapsto R(\Phi)$ is polynomial-time, isomorphism-preserving, and maps YES-instances of $\text{Gap-Max-3-SAT}_{1,1-\varepsilon}$ to YES-instances of $\text{Gap-Max-}k\text{-Coverage}_{1,1-1/e+\varepsilon'}$ and NO-instances to NO-instances of $\text{Gap-Max-}k\text{-Coverage}_{1,1-1/e+\varepsilon'}$. This proves the lemma. $\square$

We use this isomorphism-preserving reduction to conclude the proofs of Theorems 9.5 and 9.6.

Proof of Theorems 9.5 and 9.6. We show the contraposition. Assume there is an $\varepsilon > 0$ such that $\text{Gap-Max-}k\text{-Coverage}_{1,1-1/e+\varepsilon}$ can be solved in the Isomorphic-Priors model with preprocessing and query time polynomial in n even when $k = 1$ prior instance is given.

Then for every $\varepsilon' > 0$, $\text{Gap-Max-3-SAT}_{1,1-\varepsilon'}$ can be solved in the Isomorphic-Priors model with preprocessing and query time polynomial in n even when $k = 1$ prior instance is given.

This algorithm just computes for all formulas Φ that are given in the preprocessing and query phase the **Gap- k -Coverage** instance $R(\Phi)$, using the reduction from Lemma 9.27, and applies the preprocessing and query algorithm for **Gap-Max- k -Coverage**_{1, 1-1/e+ ϵ} to them.

The same argument can be made for data structures of polynomial size.

Because in [GK99] it is argued that the reduction in Lemma B.30 is also a reduction from **Gap-Max- k -Coverage**_{1, 1-1/e+ ϵ} and **Gap- k -Median**_{1, 1+2/e+2 ϵ} . We can repeat both arguments above for k -Median. $\square$

10 Conclusion

We conclude by presenting the proofs of our main theorems, referring to the relevant results established throughout the paper, and by discussing some hypotheses within our model and their consequences.

10.1 Proof of the Main Theorems

Proof of Theorem 1.1. By Theorem 7.2, we can solve **CSP** inside the Isomorphic-Priors model with $\tilde{O}(km)$ preprocessing time and $\tilde{O}(m)$ query time. By Theorem 6.3, we can solve **CST** randomized inside the Isomorphic-Priors model with $\tilde{O}(kn^\omega)$ preprocessing time and $\tilde{O}(n^\omega)$ query time.

The extensions from **Constrained Shortest Path** to **Constrained Shortest Path and ℓ_p -Shortest Path**, and from **Constrained Spanning Tree** to **Constrained Spanning Tree and ℓ_p -Spanning Tree**, are both straightforward generalizations of the presented algorithm. To this end, note that the technique of walk lifting (see Lemma 7.8) also works in this setting. $\square$

Proof of Theorem 1.2. By Theorem A.6 there is an algorithm that solves **Replacement Path** inside the Isomorphic-Priors model using $\tilde{O}(km)$ preprocessing time and $\tilde{O}(m)$ query time. By Theorem 6.2, there is an algorithm that solves **Negative Triangle** inside the Isomorphic-Priors model randomized using $\tilde{O}(kn^\omega)$ preprocessing time and $\tilde{O}(n^\omega)$ query time. $\square$

Proof of Theorem 1.3. By Theorem 7.4, we can solve **Energy Game** inside the Isomorphic-Priors model with $\tilde{O}(km)$ preprocessing time and $\tilde{O}(m)$ query time. Our Weisfeiler-Leman techniques also yield the same result for **Parity Game**, **Mean Payoff Game**, and **Simple Stochastic Game**. To be more specific, we show in Theorem A.2 that this can be extended to value-determined games (a general concept including the above games). $\square$

Proof of Theorem 1.4. The proof is a direct consequence of Theorem 9.3 and Theorem 9.4. The approximation results follow from the corresponding gap results. $\square$

10.2 k -Clique Hypothesis and Consequence

One of the foundational hypotheses in fine-grained complexity [Vas18] states that, for every fixed integer $k \geq 3$, no randomized algorithm can detect a k -clique in an n -vertex graph in $O(n^{\frac{\omega k}{3}-\epsilon})$ time for any constant $\epsilon > 0$. It is natural to ask whether this fundamental computational barrier translates to the Isomorphic-Priors model. We observe that if this hypothesis extends to our setting, then **Graph Isomorphism** is not polynomial-time solvable.

Recall our hypothesis from earlier:

Hypothesis 3.1. (*Isomorphic-Priors k -Clique Hypothesis*) For every fixed integer $k \geq 3$ and constant $\epsilon > 0$, no isomorphic-priors algorithm can detect a k -clique in an n -vertex graph with both preprocessing and query times $O(n^{\omega k/3-\epsilon})$, even with $O(1)$ prior graphs.

Observation 10.1. *Under Hypothesis 3.1, Graph Isomorphism cannot be solved in polynomial time (i.e. $\text{GI} \notin \text{P}$).*

Proof sketch. Assume for contradiction that Graph Isomorphism can be solved in $O(n^b)$ time for some constant $b \geq 1$. Let $k = \lceil 10b \rceil$. We construct an isomorphic-priors algorithm for k -Clique to handle $O(1)$ prior graphs that requires $O(1)$ preprocessing time and $O(n^b)$ query time.

During preprocessing, the algorithm does nothing. Upon receiving a query graph H , it simply runs the $O(n^b)$ -time GI algorithm to test whether $H \cong G_i$ for any i . If such G_i exists, it outputs the precomputed answer $\text{OPT}(G_i)$; otherwise, it correctly declares $H \not\cong G_i$ for any i . This satisfies the correctness guarantees of the Isomorphic-Priors model.

Since we chose $k = \lceil 10b \rceil$, our query time is $O(n^b) = O(n^{k/10})$. Because the matrix multiplication exponent $\omega \geq 2$, we have $\frac{\omega k}{3} \geq \frac{2k}{3}$. Since $k = \lceil 10b \rceil \geq 10$, $n^{k/10}$ is asymptotically faster than $O(n^{\frac{\omega k}{3} - 0.001})$. This implies we can solve k -Clique in our model in $O(n^{\frac{\omega k}{3} - 0.001})$ time, which contradicts Hypothesis 3.1. $\square$

Note that the implication $\text{GI} \notin \text{P}$ in the observation above remains true under weaker variants of the hypothesis, specifically as long as k -Clique requires $\Omega(n^{f(k)})$ time in our model for any monotonically increasing, unbounded function $f(k)$. Consequently, refuting Hypothesis 3.1 would serve as an intermediate step toward a polynomial-time algorithm for Graph Isomorphism. Conversely, resolving the hypothesis, or any of its weaker variants, in the affirmative would prove that no such algorithm exists (i.e., $\text{GI} \notin \text{P}$).

10.3 Acknowledgments and AI Disclosure

The authors thank Maximilian Thiessen for helpful comments on an earlier draft of this paper.

The authors used Gemini and ChatGPT as interactive writing and proofreading assistants, including for figure generation. While preparing the camera-ready version for FOCS 2026, ChatGPT was also used to suggest references, discuss technical formulations and possible extensions, and assist with the formulation and proof of the observation relating the Isomorphic-Priors model to invariant graph predictors (Section 4). These interactions informed revisions to the abstract, introduction, related-work and open-problems discussions, and the observation mentioned above. The authors substantially revised the generated material and take full responsibility for the manuscript, including all mathematical claims, proofs, and references.

Separately, after the FOCS 2026 submission, the authors used the ChatGPT models 5.6-So1 and 6-Pro to explore alternative proofs of results in this paper and extensions to other problems. No additional proofs or results from these post-submission experiments are included in this paper.

Bibliography

- [AAN82] V. Aggarwal, Yash P. Aneja, and K. P. K. Nair. Minimal spanning tree subject to a side constraint. *Comput. Oper. Res.*, 9(4):287–296, 1982.
- [AAN83] Yash P. Aneja, V. Aggarwal, and K. P. K. Nair. Shortest chain subject to side constraints. *Networks*, 13(2):295–302, 1983.
- [ABK25] Vikrant Ashvinkumar, Aaron Bernstein, and Adam Karczmarz. Faster approximation algorithms for restricted shortest paths in directed graphs. In *SODA*, pages 5263–5277. SIAM, 2025.
- [AD19] Albert Atserias and Anuj Dawar. Definable inapproximability: new challenges for duplicator. *J. Log. Comput.*, 29(8):1185–1210, 2019.

- [AFKV22] Vikraman Arvind, Frank Fuhlbrück, Johannes Köbler, and Oleg Verbitsky. On the weisfeiler-leman dimension of fractional packing. *Inf. Comput.*, 288:104803, 2022.
- [AFKV25] Vikraman Arvind, Frank Fuhlbrück, Johannes Köbler, and Oleg Verbitsky. On a hierarchy of spectral isomorphism invariants. *Comput. Complex.*, 34(2):19, 2025.
- [AGV15] Amir Abboud, Fabrizio Grandoni, and Virginia Vassilevska Williams. Subcubic equivalences between graph centrality problems, APSP and diameter. In *SODA*, pages 1681–1697. SIAM, 2015.
- [Alf18] Abdo Y. Alfakih. *Euclidean Distance Matrices and Their Applications in Rigidity Theory*. Springer Cham, 2018.
- [Alo21] Noga Alon. Explicit expanders of every degree and size. *Comb.*, 41(4):447–463, 2021.
- [AM13] Albert Atserias and Elitza N. Maneva. Sherali-adams relaxations and indistinguishability in counting logics. *SIAM J. Comput.*, 42(1):112–137, 2013.
- [ATHK21] Saman Ahmadi, Guido Tack, Daniel D Harabor, and Philip Kilby. A fast exact algorithm for the resource constrained shortest path problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 12217–12224, 2021.
- [Bab16] László Babai. Graph isomorphism in quasipolynomial time [extended abstract]. In *STOC*, pages 684–697. ACM, 2016.
- [Bab19] László Babai. Canonical form for graphs in quasipolynomial time: preliminary report. In *STOC*, pages 1237–1246. ACM, 2019.
- [Bar23] Boaz Barak. Introduction to theoretical computer science. Online textbook (frozen release v0.95 / Fall 2023), 2023. Fall 2023 release (v0.95), dated 2023-07-24. Accessed 2026-02-21.
- [BBG17] Christoph Berkholz, Paul S. Bonsma, and Martin Grohe. Tight lower and upper bounds for the complexity of canonical colour refinement. *Theory Comput. Syst.*, 60(4):581–614, 2017.
- [BC79] Kellogg S Booth and Charles J Colbourn. *Problems polynomially equivalent to graph isomorphism*. Computer Science Department, University of Waterloo, 1979.
- [BC89] John E. Beasley and Nicos Christofides. An algorithm for the resource constrained shortest path problem. *Networks*, 19(4):379–394, 1989.
- [BCF⁺11] Sam Buss, Yijia Chen, Jörg Flum, Sy-David Friedman, and Moritz Müller. Strong isomorphism reductions in complexity theory. *J. Symb. Log.*, 76(4):1381–1402, 2011.
- [Ber12] Aaron Bernstein. Near linear time $(1 + \epsilon)$ -approximation for restricted shortest paths in undirected graphs. In *SODA*, pages 189–201. SIAM, 2012.
- [BFZB23] Giorgos Bouritsas, Fabrizio Frasca, Stefanos Zafeiriou, and Michael M. Bronstein. Improving graph neural network expressivity via subgraph isomorphism counting. *IEEE Trans. Pattern Anal. Mach. Intell.*, 45(1):657–668, 2023.
- [BLM⁺08] Yael Berstein, Jon Lee, Hugo Maruri-Aguilar, Shmuel Onn, Eva Riccomagno, Robert Weismantel, and Henry P. Wynn. Nonlinear matroid optimization and experimental design. *SIAM J. Discret. Math.*, 22(3):901–919, 2008.

- [BMNT23] Joakim Blikstad, Sagnik Mukhopadhyay, Danupon Nanongkai, and Ta-Wei Tu. Fast algorithms via dynamic-oracle matroids. In *STOC*, pages 1229–1242. ACM, 2023.
- [Bod90] Hans L Bodlaender. Polynomial algorithms for graph isomorphism and chromatic index on partial k-trees. *Journal of Algorithms*, 11(4):631–643, 1990.
- [Bök19] Jan Böker. Color refinement, homomorphisms, and hypergraphs. In Ignasi Sau and Dimitrios M. Thilikos, editors, *Graph-Theoretic Concepts in Computer Science - 45th International Workshop, WG 2019, Vall de Núria, Spain, June 19-21, 2019, Revised Papers*, volume 11789 of *Lecture Notes in Computer Science*, pages 338–350. Springer, 2019.
- [BV07] Henrik Björklund and Sergei G. Vorobyov. A combinatorial strongly subexponential strategy improvement algorithm for mean payoff games. *Discret. Appl. Math.*, 155(2):210–229, 2007.
- [CC82] A. Cardon and Maxime Crochemore. Partitioning a graph in $o(|a| \log^2 |v|)$. *Theor. Comput. Sci.*, 19:85–98, 1982.
- [CDHL16] Krishnendu Chatterjee, Wolfgang Dvorák, Monika Henzinger, and Veronika Loitzenbauer. Conditionally optimal algorithms for generalized büchi games. In *MFCS*, volume 58 of *LIPICs*, pages 25:1–25:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2016.
- [CFI92] Jin-yi Cai, Martin Fürer, and Neil Immerman. An optimal lower bound on the number of variables for graph identification. *Comb.*, 12(4):389–410, 1992.
- [CJK⁺17] Cristian S. Calude, Sanjay Jain, Bakhadyr Khoussainov, Wei Li, and Frank Stephan. Deciding parity games in quasipolynomial time. In Hamed Hatami, Pierre McKenzie, and Valerie King, editors, *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2017, Montreal, QC, Canada, June 19-23, 2017*, pages 252–263. ACM, 2017.
- [CKL⁺22] Li Chen, Rasmus Kyng, Yang P. Liu, Richard Peng, Maximilian Probst Gutenberg, and Sushant Sachdeva. Maximum flow and minimum-cost flow in almost-linear time. In *FOCS*, pages 612–623. IEEE, 2022.
- [CLS⁺19] Deeparnab Chakrabarty, Yin Tat Lee, Aaron Sidford, Sahil Singla, and Sam Chiu-wai Wong. Faster matroid intersection. In *FOCS*, pages 1146–1168. IEEE Computer Society, 2019.
- [CMM26] Charlie Carlson, Yury Makarychev, and Ron Mosenzon. Hardness of approximation for shortest path with vector costs. In *SODA*, pages 3905–3935. SIAM, 2026.
- [CMT26] Julia Chuzhoy, Ron Mosenzon, and Ohad Trabelsi. Faster algorithms for global minimum vertex-cut in directed graphs. In *Proceedings of the 2026 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 5586–5623. SIAM, 2026.
- [CN25] Radu Curticapean and Daniel Neuen. Counting small induced subgraphs: Hardness via fourier analysis. In Yossi Azar and Debmalaya Panigrahi, editors, *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2025, New Orleans, LA, USA, January 12-15, 2025*, pages 3677–3695. SIAM, 2025.
- [Con92] Anne Condon. The complexity of stochastic games. *Inf. Comput.*, 96(2):203–224, 1992.

- [CY00] Fan Chung and Chao Yang. On polynomials of spanning trees. *Annals of Combinatorics*, 4(1):13–25, 2000.
- [DGH⁺13] Anuj Dawar, Erich Grädel, Bjarki Holm, Eryk Kopczynski, and Wied Pakusa. Definability of linear equation systems over groups and rings. *Log. Methods Comput. Sci.*, 9(4), 2013.
- [DGR18] Holger Dell, Martin Grohe, and Gaurav Rattan. Lovász meets Weisfeiler and Leman. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *45th International Colloquium on Automata, Languages, and Programming, ICALP 2018, Prague, Czech Republic, July 9-13, 2018*, volume 107 of *LIPIcs*, pages 40:1–40:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018.
- [Din07] Irit Dinur. The PCP theorem by gap amplification. *J. ACM*, 54(3):12, 2007.
- [DMW24] Simon Döring, Dániel Marx, and Philip Wellnitz. Counting small induced subgraphs with edge-monotone properties. In Bojan Mohar, Igor Shinkar, and Ryan O’Donnell, editors, *Proceedings of the 56th Annual ACM Symposium on Theory of Computing, STOC 2024, Vancouver, BC, Canada, June 24-28, 2024*, pages 1517–1525. ACM, 2024.
- [DMW25] Simon Döring, Dániel Marx, and Philip Wellnitz. From graph properties to graph parameters: Tight bounds for counting on small subgraphs. In Yossi Azar and Debmalaya Panigrahi, editors, *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2025, New Orleans, LA, USA, January 12-15, 2025*, pages 3637–3676. SIAM, 2025.
- [Dvo10] Zdenek Dvorák. On recognizing graphs by numbers of homomorphisms. *J. Graph Theory*, 64(4):330–342, 2010.
- [EJ91] E. Allen Emerson and Charanjit S. Jutla. Tree automata, mu-calculus and determinacy (extended abstract). In *32nd Annual Symposium on Foundations of Computer Science, San Juan, Puerto Rico, October 1-4, 1991*, pages 368–377. IEEE Computer Society, 1991.
- [Fei98] Uriel Feige. A threshold of $\ln n$ for approximating set cover. *J. ACM*, 45(4):634–652, 1998.
- [FH15] Stefan Felsner and Daniel Heldt. Lattice path enumeration and toeplitz matrices. *J. Integer Seq.*, 18(1):15.1.3, 2015.
- [Fox26] James Fox. A note on all-pairs shortest paths with isomorphic priors, 2026. Available at <https://j-f-ox.github.io/all-pairs-shortest-paths-isomorphic-priors.pdf>.
- [Für10] Martin Fürer. On the power of combinatorial and spectral invariants. *Linear algebra and its applications*, 432(9):2373–2380, 2010.
- [GGR24] Andreas Göbel, Leslie Ann Goldberg, and Marc Roth. The weisfeiler-leman dimension of conjunctive queries. *Proc. ACM Manag. Data*, 2(2):86, 2024.
- [GJ79] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [GK99] Sudipto Guha and Samir Khuller. Greedy strikes back: Improved facility location algorithms. *J. Algorithms*, 31(1):228–248, 1999.

- [GLR⁺20] Pawel Gawrychowski, Martin Lange, Narad Rampersad, Jeffrey O. Shallit, and Marek Szykula. Existential length universality. In *STACS*, volume 154 of *LIPIcs*, pages 16:1–16:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.
- [GM15] Martin Grohe and Dániel Marx. Structure theorem and isomorphism test for graphs with excluded topological subgraphs. *SIAM J. Comput.*, 44(1):114–159, 2015.
- [GN21] Martin Grohe and Daniel Neuen. Recent advances on the graph isomorphism problem. In Konrad K. Dabrowski, Maximilien Gadouleau, Nicholas Georgiou, Matthew Johnson, George B. Mertzios, and Daniël Paulusma, editors, *Surveys in Combinatorics, 2021: Invited lectures from the 28th British Combinatorial Conference, Durham, UK, July 5-9, 2021*, pages 187–234. Cambridge University Press, 2021.
- [GNS23] Martin Grohe, Daniel Neuen, and Pascal Schweitzer. A faster isomorphism test for graphs of small degree. *SIAM J. Comput.*, 52(6):S18–1, 2023.
- [GNSW20] Martin Grohe, Daniel Neuen, Pascal Schweitzer, and Daniel Wiebking. An improved isomorphism test for bounded-tree-width graphs. *ACM Trans. Algorithms*, 16(3):34:1–34:31, 2020.
- [GRS22] Martin Grohe, Gaurav Rattan, and Tim Seppelt. Homomorphism tensors and linear equations. In Mikolaj Bojanczyk, Emanuela Merelli, and David P. Woodruff, editors, *49th International Colloquium on Automata, Languages, and Programming, ICALP 2022, Paris, France, July 4-8, 2022*, LIPIcs, pages 70:1–70:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.
- [GS85] Harold N. Gabow and Matthias F. M. Stallmann. Efficient algorithms for graphic matroid intersection and parity (extended abstract). In *ICALP, Lecture Notes in Computer Science*, pages 210–220. Springer, 1985.
- [Hal07] Nir Halman. Simple stochastic games, parity games, mean payoff games and discounted payoff games are all lp-type problems. *Algorithmica*, 49(1):37–50, 2007.
- [Han80] Pierre Hansen. Bicriterion path problems. In G. Fandel and T. Gal, editors, *Multiple Criteria Decision Making: Theory and Application*, pages 109–127. Springer-Verlag, Berlin, 1980.
- [Has92] Refael Hassin. Approximation schemes for the restricted shortest path problem. *Math. Oper. Res.*, 17(1):36–42, 1992.
- [HCP04] Sung-Pil Hong, Sung-Jin Chung, and Bum Hwan Park. A fully polynomial bicriteria approximation scheme for the constrained spanning tree problem. *Oper. Res. Lett.*, 32(3):233–239, 2004.
- [HW74] John E Hopcroft and Jin-Kue Wong. Linear time algorithm for isomorphism of planar graphs (preliminary report). In *Proceedings of the sixth annual ACM symposium on Theory of computing*, pages 172–184, 1974.
- [HZ80] Gabriel Y Handler and Israel Zang. A dual algorithm for the constrained shortest path problem. *Networks*, 10(4):293–309, 1980.
- [IL90] Neil Immerman and Eric Lander. Describing graphs: A first-order approach to graph canonization. In *Complexity theory retrospective*, pages 59–81. Springer, 1990.

- [Ita77] Alon Itai. Finding a minimum circuit in a graph. In *STOC*, pages 1–10. ACM, 1977.
- [Jur98] Marcin Jurdzinski. Deciding the winner in parity games is in $UP \cap co-UP$. *Inf. Process. Lett.*, 68(3):119–124, 1998.
- [Kar72] Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Proceedings of a symposium on the Complexity of Computer Computations, held March 20-22, 1972, at the IBM Thomas J. Watson Research Center, Yorktown Heights, New York, USA*, The IBM Research Symposia Series, pages 85–103. Plenum Press, New York, 1972.
- [KBH⁺21] Devin Kreuzer, Dominique Beaini, William L. Hamilton, Vincent Létourneau, and Prudencio Tossou. Rethinking graph transformers with spectral attention. In *NeurIPS*, pages 21618–21629, 2021.
- [Kie20a] Sandra Kiefer. *Power and Limits of the Weisfeiler-Leman Algorithm*. PhD thesis, RWTH Aachen University, 2020.
- [Kie20b] Sandra Kiefer. The weisfeiler-leman algorithm: an exploration of its power. *ACM SIGLOG News*, 7(3):5–27, 2020.
- [KL14] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography, Second Edition*. CRC Press, 2014.
- [Koz78] Dexter Kozen. A clique problem equivalent to graph isomorphism. *SIGACT News*, 10(2):50–52, 1978.
- [KP19] Nicolas Keriven and Gabriel Peyré. Universal invariant and equivariant graph neural networks. In *NeurIPS*, pages 7090–7099, 2019.
- [LB24] Matthias Lanzinger and Pablo Barceló. On the power of the weisfeiler-leman test for graph motif parameters. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.
- [LNP⁺25] Jason Li, Danupon Nanongkai, Debmalya Panigrahi, Thatchaphol Saranurak, and Sorrachai Yingchareonthawornchai. Vertex connectivity in poly-logarithmic max-flows. *J. ACM*, 72(4):28:1–28:34, 2025.
- [Lov73] László Lovász. Coverings and coloring of hypergraphs. In *Proceedings of the Fourth Southeastern Conference on Combinatorics, Graph Theory, and Computing*, pages 3–12. Utilitas Mathematica, Winnipeg, 1973.
- [LPPS17] Daniel Lokshtanov, Marcin Pilipczuk, Michał Pilipczuk, and Saket Saurabh. Fixed-parameter tractable canonization and isomorphism test for graphs of bounded treewidth. *SIAM J. Comput.*, 46(1):161–189, 2017.
- [LR01] Dean H. Lorenz and Danny Raz. A simple efficient approximation scheme for the restricted shortest path problem. *Oper. Res. Lett.*, 28(5):213–219, 2001.
- [LT92] Antoni Lozano and Jacobo Torán. On the non-uniform complexity of the graph isomorphism problem. In Klaus Ambos-Spies, Steven Homer, and Uwe Schöning, editors, *Complexity Theory: Current Research, Dagstuhl Workshop, February 2-8, 1992*, pages 245–271. Cambridge University Press, 1992.

- [Luk82] Eugene M Luks. Isomorphism of graphs of bounded valence can be tested in polynomial time. *Journal of computer and system sciences*, 25(1):42–65, 1982.
- [LXZ24] Shi Li, Chenyang Xu, and Ruilong Zhang. Polylogarithmic approximations for robust s-t path. In *ICALP*, volume 297 of *LIPICs*, pages 106:1–106:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.
- [MBSL19] Haggai Maron, Heli Ben-Hamu, Nadav Shamir, and Yaron Lipman. Invariant and equivariant graph networks. In *ICLR (Poster)*. OpenReview.net, 2019.
- [MOT24] Yury Makarychev, Max Ovsiankin, and Erasmo Tani. Approximation algorithms for ℓ_p -shortest path and ℓ_p -group steiner tree. In *ICALP*, volume 297 of *LIPICs*, pages 111:1–111:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.
- [MRF⁺19] Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and leman go neural: Higher-order graph neural networks. In *AAAI*, pages 4602–4609. AAAI Press, 2019.
- [MRS⁺98] Madhav V Marathe, Ramamoorthi Ravi, Ravi Sundaram, SS Ravi, Daniel J Rosenkrantz, and Harry B Hunt III. Bicriteria network design problems. *Journal of algorithms*, 28(1):142–171, 1998.
- [MST24] Rupak Majumdar, Irmak Saglam, and K. S. Thejaswini. Rabin games and colourful universal trees. In *TACAS (3)*, volume 14572 of *Lecture Notes in Computer Science*, pages 213–231. Springer, 2024.
- [MZ00] Kurt Mehlhorn and Mark Ziegelmann. Resource constrained shortest paths. In *ESA*, volume 1879 of *Lecture Notes in Computer Science*, pages 326–337. Springer, 2000.
- [MZ01] Kurt Mehlhorn and Mark Ziegelmann. CNOP - A package for constrained network optimization. In *ALLENEX*, volume 2153 of *Lecture Notes in Computer Science*, pages 17–31. Springer, 2001.
- [Neu24a] Daniel Neuen. Homomorphism-distinguishing closedness for graphs of bounded tree-width. In Olaf Beyersdorff, Mamadou Moustapha Kanté, Orna Kupferman, and Daniel Lokshtanov, editors, *41st International Symposium on Theoretical Aspects of Computer Science, STACS 2024, Clermont-Ferrand, France, March 12-14, 2024*, *LIPICs*, pages 53:1–53:12. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.
- [Neu24b] Daniel Neuen. Isomorphism testing for graphs excluding small topological subgraphs. *ACM Trans. Algorithms*, 20(3):25, 2024.
- [NSW25] Jesper Nederlof, Céline M. F. Swennenhuis, and Karol Węgrzycki. A subexponential time algorithm for makespan scheduling of unit jobs with precedence constraints. In *SODA*, pages 535–552. SIAM, 2025.
- [NY15] Moni Naor and Eylon Yogev. Bloom filters in adversarial environments. In *CRYPTO (2)*, *Lecture Notes in Computer Science*, pages 565–584. Springer, 2015.
- [Ott17] Martin Otto. *Bounded Variable Logics and Counting: A Study in Finite Models*, volume 9 of *Lecture Notes in Logic*. Cambridge University Press, 2017.
- [PT87] Robert Paige and Robert Endre Tarjan. Three partition refinement algorithms. *SIAM J. Comput.*, 16(6):973–989, 1987.

- [PY00] Christos H. Papadimitriou and Mihalis Yannakakis. On the approximability of trade-offs and optimal access of web sources. In *FOCS*, pages 86–92. IEEE Computer Society, 2000.
- [RG96] R. Ravi and Michel X. Goemans. The constrained minimum spanning tree problem (extended abstract). In *SWAT*, volume 1097 of *Lecture Notes in Computer Science*, pages 66–75. Springer, 1996.
- [RS23] Gaurav Rattan and Tim Seppelt. Weisfeiler-leman and graph spectra. In Nikhil Bansal and Viswanath Nagarajan, editors, *Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SODA 2023, Florence, Italy, January 22-25, 2023*, pages 2268–2285. SIAM, 2023.
- [Sch03] Alexander Schrijver. *Combinatorial Optimization: Polyhedra and Efficiency*. Springer, 2003.
- [Sho83] Andrew W. Shogan. Constructing a minimal-cost spanning tree subject to resource constraints and flow requirements. *Networks*, 13(2):169–190, 1983.
- [Skr21] Saveliy V. Skresanov. The probabilistic Weisfeiler-Leman algorithm. *CoRR*, abs/2112.13045, 2021.
- [SSvL⁺11] Nino Shervashidze, Pascal Schweitzer, Erik Jan van Leeuwen, Kurt Mehlhorn, and Karsten M. Borgwardt. Weisfeiler-lehman graph kernels. *J. Mach. Learn. Res.*, 12:2539–2561, 2011.
- [Sto12] Sabine Storandt. Route planning for bicycles - exact constrained shortest paths made practical via contraction hierarchy. In *ICAPS*. AAAI, 2012.
- [Tho97] Wolfgang Thomas. Languages, automata, and logic. In Grzegorz Rozenberg and Arto Salomaa, editors, *Handbook of Formal Languages, Volume 3: Beyond Words*, pages 389–455. Springer, 1997.
- [Tuc24] Jamie Tucker-Foltz. Inapproximability of unique games in fixed-point logic with counting. *Log. Methods Comput. Sci.*, 20(2), 2024.
- [Vad12] Salil P. Vadhan. Pseudorandomness. *Found. Trends Theor. Comput. Sci.*, 7(1-3):1–336, 2012.
- [Vas18] Virginia Vassilevska Williams. On some fine-grained questions in algorithms and complexity. In *Proceedings of the international congress of mathematicians: Rio de janeiro 2018*, pages 3447–3487. World Scientific, 2018.
- [vdBCP⁺23] Jan van den Brand, Li Chen, Richard Peng, Rasmus Kyng, Yang P. Liu, Maximilian Probst Gutenberg, Sushant Sachdeva, and Aaron Sidford. A deterministic almost-linear time algorithm for minimum-cost flow. In *FOCS*, pages 503–514. IEEE, 2023.
- [vdBLL⁺21] Jan van den Brand, Yin Tat Lee, Yang P. Liu, Thatchaphol Saranurak, Aaron Sidford, Zhao Song, and Di Wang. Minimum cost flows, mdps, and ℓ_1 -regression in nearly linear time for dense instances. In *STOC*, pages 859–869. ACM, 2021.
- [vdBLN⁺20] Jan van den Brand, Yin Tat Lee, Danupon Nanongkai, Richard Peng, Thatchaphol Saranurak, Aaron Sidford, Zhao Song, and Di Wang. Bipartite matching in nearly-linear time on moderately dense graphs. In *FOCS*, pages 919–930. IEEE, 2020.

- [VW10] Virginia Vassilevska Williams and Ryan Williams. Subcubic equivalences between path, matrix and triangle problems. In *FOCS*, pages 645–654. IEEE Computer Society, 2010.
- [WW23] Libin Wang and Raymond Chi-Wing Wong. QHL: A fast algorithm for exact constrained shortest path search on road networks. *Proc. ACM Manag. Data*, 1(2):155:1–155:25, 2023.
- [WXYL16] Sibow Wang, Xiaokui Xiao, Yin Yang, and Wenqing Lin. Effective indexing for approximate constrained shortest path queries on large road networks. *Proc. VLDB Endow.*, 10(2):61–72, 2016.
- [XHLJ19] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *ICLR*. OpenReview.net, 2019.
- [Xue00] Guoliang Xue. Primal-dual algorithms for computing weight-constrained shortest paths and weight-constrained minimum spanning trees. In *Conference Proceedings of the 2000 IEEE International Performance, Computing, and Communications Conference (Cat. No. 00CH37086)*, pages 271–277. IEEE, 2000.
- [YRZ⁺24] Jiaming Yin, Weixiong Rao, Qinpei Zhao, Chenxi Zhang, and Pan Hui. Learn to optimize the constrained shortest path on large dynamic graphs. *IEEE Trans. Mob. Comput.*, 23(3):2456–2469, 2024.
- [YY98] Gang Yu and Jian Yang. On the robust shortest path problem. *Computers & operations research*, 25(6):457–468, 1998.
- [ZGD⁺24] Bohang Zhang, Jingchu Gai, Yiheng Du, Qiwei Ye, Di He, and Liwei Wang. Beyond weisfeiler-lehman: A quantitative framework for GNN expressiveness. In *ICLR*. OpenReview.net, 2024.

A Further Problems that are Invariant under 1-WL

We first provide the deferred proof of Lemma 7.14 concerning color-consistent strategies for Energy Game. We then give further examples of problems that are invariant under 1-WL and can therefore be solved in near-linear time in the Isomorphic-Priors model.

A.1 Color-Consistent Strategies for Energy Game

Proof of Lemma 7.14. Let $h(v)$ be the minimum initial energy required to win from v . By assumption, σ wins from every starting vertex v with initial energy $h(v)$. The shadow-play argument using Neighborhood Preservation in the proof of Theorem 7.12 shows that vertices of the same stable 1-WL color have the same minimum initial energy. Thus h is constant on each color class.

For each color class C , choose an arbitrary representative $r_C \in C$ and define $H(C) := h(r_C)$. For each stable color class $C \subseteq V_1$ and each $u \in C$, choose

$$\hat{\sigma}(u) \in \{v \in V \mid (u, v) \in E, w(u, v) = w(r_C, \sigma(r_C)), \chi_\infty^G(v) = \chi_\infty^G(\sigma(r_C))\}.$$

This set is nonempty by Neighborhood Preservation.

Let $(u, v) \in E$ be an edge consistent with $\hat{\sigma}$, meaning that either $u \in V_2$ or $v = \hat{\sigma}(u)$. Denote $C = \chi_\infty^G(u)$ and $D = \chi_\infty^G(v)$. There exists a σ -consistent edge (r_C, z) with $w(r_C, z) = w(u, v)$ and

$\chi_\infty^G(z) = D$: for $u \in V_1$, take $z = \sigma(r_C)$; for $u \in V_2$, use Neighborhood Preservation. Since σ wins from r_C with energy $H(C)$, we have $h(r_C) + w(u, v) \geq h(z)$ and therefore $w(u, v) \geq H(D) - H(C)$.

Now fix any initial vertex v_0 and energy $E_0 \geq h(v_0)$, and consider any resulting infinite walk $\pi = v_0 v_1 v_2 \dots$ consistent with $\hat{\sigma}$. Applying the preceding inequality to each edge (v_k, v_{k+1}) yields a telescoping sum

$$E_k = E_0 + \sum_{j=0}^{k-1} w(v_j, v_{j+1}) \geq H(\chi_\infty^G(v_k)) \geq 0 \quad \text{for all } k \geq 0.$$

Hence $\hat{\sigma}$ is optimal from every starting vertex and has the required consistency property. The construction of $\hat{\sigma}$ takes linear time. $\square$

A.2 More Games on Graphs

We have shown in Section 7 that **Energy Game** can be solved efficiently in the Isomorphic-Priors model using the 1-WL algorithm. In the following, we extend them to more general games.

Definition A.1 (Value-determined Game). An instance of Value-determined Game is defined by a tuple $G = (V, E, V_1, V_2, V_r, \delta, w, v_0, W, p)$, where

- (V, E) is a finite directed graph with no terminal states,
- (V_1, V_2, V_r) is a partition of V , where Player 1 controls V_1 , Player 2 controls V_2 , and V_r are random nodes,
- δ is a transition probability function for random nodes: for each $u \in V_r$, $\delta(u)$ is a probability distribution over $\text{Succ}(u) = \{v \mid (u, v) \in E\}$.
- $w : V \rightarrow \Sigma$ is a label function.
- v_0 is the designated initial vertex.
- $W \subseteq \Sigma^\omega$ is the Borel objective for Player 1.
- $p \in [0, 1]$ is a probability threshold.

A play $\pi = v_0 v_1 v_2 \dots$ is formed by moving a token: at step $k \geq 0$, if $v_k \in V_i, i \in \{1, 2\}$, Player i chooses an outgoing edge $(v_k, v_{k+1}) \in E$ and moves the token to v_{k+1} ; if $v_k \in V_r$, then the arena samples a node v_{k+1} from $\delta(v_k)$, and moves the token to v_{k+1} . We slightly abuse the label function by denoting $w(\pi) := w(v_0)w(v_1)w(v_2) \dots$. We denote $\pi(\sigma, \tau)$ as the (randomized) play where Player 1 uses strategy σ and Player 2 uses strategy τ . We say

- Player 1 wins the game from v_0 if $\sup_\sigma \inf_\tau \Pr(w(\pi(\sigma, \tau)) \in W) \geq p$,
- Player 2 wins the game from v_0 if $\inf_\tau \sup_\sigma \Pr(w(\pi(\sigma, \tau)) \in W) < p$.

The game is value-determined, if $\sup_\sigma \inf_\tau \Pr(w(\pi(\sigma, \tau)) \in W) = \inf_\tau \sup_\sigma \Pr(w(\pi(\sigma, \tau)) \in W)$, which means that exactly one of the Players wins. For a value-determined game G , we define $OPT(G) = \text{YES}$ if Player 1 wins from v_0 , and $OPT(G) = \text{NO}$ otherwise.

We view a value-determined game as an attributed directed graph in the sense of Section 5. All instance-specific information can be encoded using vertex and edge attributes. In particular, the threshold p , which is a global parameter of the instance, can be encoded by including p in the attribute of every vertex. We regard the Borel objective W as a fixed global parameter. Thus, Value-determined Game fits the model of 5.

Theorem A.2 (Value-Determined Game in the Isomorphic-Priors Model). *There is an algorithm that solves the Value-determined Game problem with k prior instances each having at most n vertices and m edges in $O(k(n + m) \log n)$ preprocessing time and $O((n + m) \log n)$ query time.*

Similar to solving **Energy Game** in the Isomorphic-Priors setting, we only need to show that if two games are 1-WL equivalent, then the winning player is the same.

We initialize 1-WL with the vertex types and labels, and use transition probabilities as labels on edges leaving random nodes. All other edges receive a distinguished dummy label. Colors are compared using a common refinement scheme. Strategies may be history-dependent and randomized.

Theorem A.3. *Let G and G' be two instances of Value-determined Games with the same Borel objective W . If $G \equiv_{1\text{-WL}} G'$ and $\chi_\infty^G(v_0) = \chi_\infty^{G'}(v'_0)$, then the winners of G and G' are the same.*

Proof. By symmetry, it suffices to show that the value of G' is at least the value of G . Fix an arbitrary strategy σ for Player 1 in G . For simplicity, we use w for the label functions of both games.

For every pair of equally colored vertices u, u' , fix a color-preserving bijection $b_{u,u'} : \text{Succ}(u) \rightarrow \text{Succ}(u')$. At random nodes, additionally require $\delta(u)(v) = \delta'(u')(b_{u,u'}(v))$. Such bijections exist by Neighborhood Preservation, since the refinement also includes transition probabilities as edge labels.

Fix a common infinite random tape, viewed as a sequence $(U_k)_{k \geq 0}$ of independent uniform random variables on $[0, 1]$, independent of the players' randomization. At corresponding random nodes u, u' , order the successors of u arbitrarily and order those of u' according to $b_{u,u'}$. At step k , both games use U_k to sample from the corresponding cumulative probability intervals. The two transitions then have the correct marginal distributions and lead to successors matched by $b_{u,u'}$. This random tape is used only for the coupling and is not observed by the players.

We define σ' dynamically by maintaining a simulated “shadow play” in G' . Let the current finite prefix of a play in G' be $\pi' = (v'_0, v'_1, \dots, v'_k)$ and the shadow play in G be $\pi = (v_0, v_1, \dots, v_k)$. We show by induction on k that $\chi_\infty^G(v_i) = \chi_\infty^{G'}(v'_i)$ for all $i \leq k$.

Base case. For $k = 0$, the claim holds by assumption.

Inductive step. Assume the claim holds for all $i \leq k$. Since v'_k and v_k have the same color, they have the same vertex type.

- If v_k and v'_k are Player 1 nodes, we first move the token in G : the strategy σ selects a successor v_{k+1} according to $\sigma(\pi)$. Then we move the token in G' to $v'_{k+1} = b_{v_k, v'_k}(v_{k+1})$. By the choice of the bijection, $\chi_\infty^{G'}(v'_{k+1}) = \chi_\infty^G(v_{k+1})$.
- If v_k and v'_k are Player 2 nodes, we first move the token in G' : Player 2 selects an arbitrary successor v'_{k+1} of v'_k . We then move the token in G to $v_{k+1} = b_{v_k, v'_k}^{-1}(v'_{k+1})$. Again, $\chi_\infty^G(v_{k+1}) = \chi_\infty^{G'}(v'_{k+1})$.
- If v_k and v'_k are random nodes, the arena samples $v'_{k+1} \sim \delta'(v'_k)$. Upon observing v'_{k+1} , the strategy σ' appends to the shadow play the vertex $v_{k+1} := b_{v_k, v'_k}^{-1}(v'_{k+1})$. This implements the coupling above: since $\delta'(v'_k)(b_{v_k, v'_k}(u)) = \delta(v_k)(u)$ for every $u \in \text{Succ}(v_k)$, the induced shadow transition has distribution $\delta(v_k)$. Moreover, color preservation gives $\chi_\infty^G(v_{k+1}) = \chi_\infty^{G'}(v'_{k+1})$. Thus σ' reconstructs the shadow move from the observed move in G' , without observing the random tape.

The fixed local bijections give a bijection between finite histories in the two games. Both directions of this correspondence depend only on the histories already observed. Therefore, σ' is a legal history-dependent strategy.

Now fix an arbitrary strategy τ' for Player 2 in G' . Using the history correspondence, define a strategy τ in G by transporting the choices of τ' through the inverse local bijections. This is also a legal history-dependent strategy.

Under these strategy pairs, corresponding finite histories have equal probabilities. At player nodes, this follows from the construction of the strategies; at random nodes, it follows from the probability-preserving coupling. Recall that colors determine the labels of the nodes along a play. Thus the two games induce the same distribution on label sequences, and

$$\Pr(w(\pi'(\sigma', \tau')) \in W) = \Pr(w(\pi(\sigma, \tau)) \in W).$$

Here $\pi(\sigma, \tau)$ and $\pi'(\sigma', \tau')$ denote the resulting infinite plays in G and G' , respectively.

Since this holds for every τ' , we have

$$\inf_{\tau'} \Pr(w(\pi'(\sigma', \tau')) \in W) \geq \inf_{\tau} \Pr(w(\pi(\sigma, \tau)) \in W).$$

Since σ was arbitrary, it follows that

$$\sup_{\sigma'} \inf_{\tau'} \Pr(w(\pi'(\sigma', \tau')) \in W) \geq \sup_{\sigma} \inf_{\tau} \Pr(w(\pi(\sigma, \tau)) \in W).$$

By symmetry, the values of G and G' are equal. Since both games are value-determined and have the same threshold p , their winners are the same. $\square$

Proof of Theorem A.2. The first part follows directly from Theorems A.3 and 8.4. For the second part, observe that Theorem A.3 provides an explicit method for constructing a winning strategy on H given a strategy for a prior graph G_i such that $G_i \equiv_{1\text{-WL}} H$. $\square$

Generalized Büchi games. A generalized Büchi game is equipped with a family of sets $F_1, \dots, F_r \subseteq V$, and Player 1 wins if every set F_i is visited infinitely often. Generalized Büchi games can be viewed as a special case of value-determined games. Indeed, if the sets $F_1, \dots, F_r$ come with fixed identifiers, we may label every vertex v by the set

$$w(v) := \{i \in [r] \mid v \in F_i\},$$

and choose the Borel objective that requires every index $i \in [r]$ to occur infinitely often. Hence, Theorem A.2 immediately applies to this representation.

There is, however, a subtlety in this formulation. Encoding the indices $i \in [r]$ as part of the vertex labels makes the identities of the Büchi sets part of the input attributes. Consequently, an isomorphism is required to preserve every set F_i individually. This is stronger than the arguably more natural notion of isomorphism for generalized Büchi games, under which the Büchi sets themselves may be permuted. Namely, one would only require the existence of a graph isomorphism φ and a permutation π of $[r]$ such that

$$v \in F_i \iff \varphi(v) \in F'_{\pi(i)}$$

for every $v \in V$ and $i \in [r]$.

A natural attempt to model this stronger symmetry is to introduce one unlabeled auxiliary vertex for each Büchi set and encode membership by incidence edges. In such a representation, graph isomorphisms may freely permute the auxiliary vertices, exactly as desired. However, our current 1-WL argument does not immediately extend to this representation: the shadow-play construction preserves local colors and transition behavior, but does not provide a single globally consistent matching between the Büchi sets of the two games. Such a consistent matching appears necessary in order to conclude that visiting every Büchi set infinitely often is preserved.

Rabin games. A similar distinction arises for Rabin games. A Rabin game is equipped with a family of pairs $(E_1, F_1), \dots, (E_r, F_r)$, where $E_i, F_i \subseteq V$, and Player 1 wins if there exists an index $i \in [r]$ such that E_i is visited only finitely often and F_i is visited infinitely often. If the pairs come with fixed identifiers, we may label every vertex v by

$$w(v) := (\{i \in [r] \mid v \in E_i\}, \{i \in [r] \mid v \in F_i\}).$$

The Rabin condition is then a Borel objective on the resulting label sequence, so Theorem A.2 also applies to this representation.

As in the generalized Büchi case, however, this encoding requires isomorphisms to preserve every set E_i and F_i individually. Under the more natural notion of isomorphism, the Rabin pairs may be permuted, while the pairing and the roles of their two components are preserved. More precisely, one requires a game-graph isomorphism φ and a permutation π of $[r]$ such that

$$\varphi(E_i) = E'_{\pi(i)} \quad \text{and} \quad \varphi(F_i) = F'_{\pi(i)}$$

for every $i \in [r]$. Encoding the pairs and their two types of membership using auxiliary vertices and incidence edges allows this symmetry to be represented. However, the same obstacle remains: our current shadow-play construction does not provide a single globally consistent correspondence between the Rabin pairs, and therefore does not establish preservation of the Rabin objective in this representation.

Open questions. Under the natural notions of isomorphism described above, we leave open whether the Isomorphic-Priors model admits algorithms with $O(\text{poly}(k, n))$ preprocessing time and

1. $O(n^{2.99})$ query time for generalized Büchi games, when the Büchi sets may be arbitrarily permuted; and
2. $O(\text{poly}(n))$ query time for Rabin games, when the Rabin pairs may be arbitrarily permuted.

A.3 The Replacement Path Problem

The work of Vassilevska Williams and Williams [VW10] is widely regarded as an early landmark in fine-grained complexity. In particular, they showed that a number of fundamental problems with best known subcubic algorithms are tightly related via subcubic reductions, yielding an equivalence web among path, matrix, and triangle problems.

In this appendix, we show that, in our model, Replacement Path can be solved in $O((n+m) \log n)$ time.

Definition A.4 (Replacement Path). An instance of Replacement Path is defined by a tuple $G = (V, E, c, P)$, where

- (V, E) is a finite directed graph,
- $c: E \rightarrow \mathbb{R}$ is a cost function assigning a non-negative cost to each edge,
- $P = v_1 v_2 \dots v_k$ is a shortest path.

The solution for Replacement Path is a vector $OPT = (OPT_1, \dots, OPT_{k-1})$, where for each $i \in \{1, \dots, k-1\}$, OPT_i denotes the cost of the shortest v_1, v_k -path in $G_i = (V, E \setminus \{(v_i, v_{i+1})\})$.

Remark A.5. We may assume without loss of generality that G contains no negative cycles reachable from v_1 and to v_k . Indeed, since P is a shortest path, the distance between its endpoints is finite.

We view an instance $G = (V, E, c, P)$, where $P = v_1 v_2 \dots v_k$, of **Replacement Path** as an attributed graph with edge attributes $\alpha_E(e) = c(e)$ for all $e \in E$ and vertex attributes $\alpha_V(v_i) = i$ for all $i \in \{1, \dots, k\}$, and $\alpha_V(w) = 0$ for all other vertices $w \in V \setminus \{v_1, \dots, v_k\}$.

Theorem A.6 (Replacement Path in the Isomorphic-Priors Model). *There is an algorithm that solves Replacement Path with k prior instances each having at most n vertices and m edges in $O(k(n+m) \log n)$ preprocessing time and $O((n+m) \log n)$ query time. Additionally, if each prior G_i is equipped with a set of replacement paths, then the algorithm can also compute a set of replacement paths for H in $O((n+m) \log n)$ time.*

In the following theorem, we prove that **Replacement Path** is invariant under 1-WL equivalence.

Theorem A.7. *Let $G = (V, E, c: E \rightarrow \mathbb{R}, P)$ and $G' = (V', E', c': E' \rightarrow \mathbb{R}, P')$ be two instances of Replacement Path. If $G \equiv_{1\text{-WL}} G'$, then G and G' have the same optimal replacement path vector.*

Proof. Let $P = v_1, \dots, v_k$ and $P' = v'_1, \dots, v'_k$ be the designated shortest paths in G and G' , respectively. Observe that the initial coloring χ_0 assigns color i to v_i and to v'_i for every $i \in \{1, \dots, k\}$, and assigns color 0 to all remaining vertices. Let OPT and OPT' be the optimal replacement vectors for G and G' , respectively. We show that for every $i \in \{1, \dots, k-1\}$, $OPT_i = OPT'_i$.

Fix $i \in \{1, \dots, k-1\}$. Let $Q = u_1, \dots, u_m$ be a shortest v_1, v_k -path in $G_i = (V, E \setminus \{(v_i, v_{i+1})\})$. We construct a v'_1, v'_k -walk $Q' = u'_1, \dots, u'_m$ in $G'_i = (V', E' \setminus \{(v'_i, v'_{i+1})\})$ with the same sequence of stable colors as Q , and hence with the same total cost.

We define Q' inductively. Since $u_1 = v_1$, its initial color is 1. Because $G \equiv_{1\text{-WL}} G'$, there exists a vertex $u'_1 \in V'$ with $\chi_\infty^G(u_1) = \chi_\infty^{G'}(u'_1)$. By the Refinement property of 1-WL (see Lemma 7.6), u'_1 has the same initial color as u_1 , and therefore $\chi_0^{G'}(u'_1) = 1$, so $u'_1 = v'_1$.

Now suppose that u'_j has already been chosen so that $\chi_\infty^G(u_j) = \chi_\infty^{G'}(u'_j)$. Since $(u_j, u_{j+1}) \in E$, by Neighborhood Preservation (see Lemma 7.6), there exists an outgoing edge $(u'_j, u'_{j+1}) \in E'$ such that $\chi_\infty^G(u_{j+1}) = \chi_\infty^{G'}(u'_{j+1})$ and (u_j, u_{j+1}) and (u'_j, u'_{j+1}) have the same edge label, i.e., the same cost under c and c' , respectively. Continuing in this way, we obtain a walk Q' in G' from v'_1 to some vertex u'_m with $\chi_\infty^G(v_k) = \chi_\infty^{G'}(u'_m)$. By Refinement, $\chi_0^{G'}(u'_m) = \chi_0^G(v_k) = k$, and hence $u'_m = v'_k$.

Thus, Q' is a v'_1, v'_k -walk in G' and $c'(Q') = c(Q)$. We claim that Q' does not use the edge (v'_i, v'_{i+1}) . Suppose otherwise. Then there exists j such that $(u'_j, u'_{j+1}) = (v'_i, v'_{i+1})$. Since v'_i and v'_{i+1} are the unique vertices of initial colors $\chi_0^{G'}(v'_i) = i$ and $\chi_0^{G'}(v'_{i+1}) = i+1$, it follows from $\chi_\infty^G(u_j) = \chi_\infty^{G'}(u'_j)$ and $\chi_\infty^G(u_{j+1}) = \chi_\infty^{G'}(u'_{j+1})$, together with another application of Refinement, that $u_j = v_i$ and $u_{j+1} = v_{i+1}$. Hence $(u_j, u_{j+1}) = (v_i, v_{i+1})$, contradicting the fact that Q is a path in G_i .

Therefore, Q' is a valid v'_1, v'_k -walk in G'_i , and so, by Remark A.5, $OPT'_i \leq c'(Q') = c(Q) = OPT_i$. By symmetry, $OPT_i \leq OPT'_i$. Hence $OPT_i = OPT'_i$, as required. $\square$

Proof of Theorem A.6. The first part follows directly from Theorems A.7 and 8.4. For the second part, observe that Theorem A.7 provides an explicit method for constructing replacement paths on H given replacement paths for a prior graph G_i such that $G_i \equiv_{1\text{-WL}} H$. $\square$

A.4 Maximum Flow

Next, we show that **Maximum Flow** can be solved in near-linear time in our model, slightly improving over existing worst-case algorithms [CKL⁺22, vdBCP⁺23, vdBLL⁺21, vdBLN⁺20]. Towards this end, we again argue that 1-WL equivalent graphs admit the same maximum flow. We note that this also improves over a result from [AM13] where the same is shown for 2-WL.

Definition A.8. An instance of Maximum Flow is a tuple $G = (V, E, c, s, t)$ where (V, E) is a directed graph, $c: E(G) \rightarrow \mathbb{R}_{\geq 0}$ are edge capacities, and $s, t \in V$ are the source and sink vertex. A flow is a mapping $f: E(G) \rightarrow \mathbb{R}_{\geq 0}$ such that

- $f(e) \leq c(e)$ for all $e \in E(G)$, and
- $\sum_{u \in N_{in}(v)} f(u, v) = \sum_{w \in N_{out}(v)} f(v, w)$ for all $v \in V \setminus \{s, t\}$.

The objective is to find a flow f that maximizes $\text{val}(f) := \sum_{w \in N_{out}(s)} f(s, w)$.

Theorem A.9 (Maximum Flow in the Isomorphic-Priors Model). *There is an algorithm that solves Maximum Flow with k prior instances each having at most n vertices and m edges in $O(k(n+m) \log n)$ preprocessing time and $O((n+m) \log n)$ query time. Additionally, if each prior G_i is equipped with a maximum flow, then the maximum flow for the query graph can be found in $O((n+m) \log n)$ query time.*

For the proof, we require some additional basic tools. First, we generalize the neighborhood preservation property from Lemma 7.6.

Lemma A.10 (Folklore). *For every two (attributed) graphs $G = (V_G, E_G, \alpha_G^V, \alpha_G^E)$ and $H = (V_H, E_H, \alpha_H^V, \alpha_H^E)$ such that $G \equiv_{1\text{-WL}} H$, the following holds:*

1. **Neighborhood Multiset Preservation:** *For every $v \in V_G$ and $v' \in V_H$, if $\chi_\infty^G(v) = \chi_\infty^H(v')$, then for every stable color C and edge attribute ℓ , it holds that*

$$|\{u \in V_G \mid \alpha_G^E(u, v) = \ell, \chi_\infty^G(u) = C\}| = |\{u' \in V_H \mid \alpha_H^E(u', v') = \ell, \chi_\infty^H(u') = C\}|.$$

The analogous statement holds for outgoing edges.

Now let $G = (V, E, c, s, t)$ be an instance of Maximum Flow, and let χ_∞^G denote the coloring computed by 1-WL. We define an auxiliary instance $G/\chi_\infty = (V^*, E^*, c^*, s^*, t^*)$ via

- $V^* = \{\chi_\infty^G(v) \mid v \in V\}$,
- $E^* = \{(\chi_\infty^G(v), \chi_\infty^G(w)) \mid (v, w) \in E\}$,
- $c^*(a, b) = \sum_{v \in \chi_\infty^{-1}(a), w \in \chi_\infty^{-1}(b), (v, w) \in E} c(v, w)$,
- $s^* = \chi_\infty^G(s)$ and $t^* = \chi_\infty^G(t)$.

The next two lemmas prove that the value of the maximum flow is identical in G and G/χ_∞ .

Lemma A.11. *Let $G = (V, E, c, s, t)$ be an instance of Maximum Flow and let f be a flow in G . Then there exists a flow f^* in G/χ_∞ such that $\text{val}(f^*) \geq \text{val}(f)$.*

Proof. For $(a, b) \in E(G/\chi_\infty)$, we define

$$f^*(a, b) := \sum_{v \in \chi_\infty^{-1}(a), w \in \chi_\infty^{-1}(b), (v, w) \in E} f(v, w).$$

By definition, it immediately follows that $f^*(a, b) \leq c^*(a, b)$. Moreover, for $a \in V^* \setminus \{s^*, t^*\}$, it holds that

$$\begin{aligned}
\sum_{b \in N_{in}(a)} f^*(b, a) &= \sum_{b \in N_{in}(a)} \sum_{v \in \chi^{-1}(a), u \in \chi^{-1}(b), (u, v) \in E} f(u, v) \\
&= \sum_{v \in \chi^{-1}(a)} \sum_{u \in N_{in}(v)} f(u, v) \\
&= \sum_{v \in \chi^{-1}(a)} \sum_{w \in N_{out}(v)} f(v, w) \\
&= \sum_{b \in N_{out}(a)} \sum_{v \in \chi^{-1}(a), w \in \chi^{-1}(b), (v, w) \in E} f(v, w) \\
&= \sum_{b \in N_{out}(a)} f^*(a, b).
\end{aligned}$$

Hence, f^* is a flow in G/χ_∞ . Finally,

$$\text{val}(f^*) = \sum_{a \in N_{out}(s^*)} f(s^*, a) = \sum_{a \in N_{out}(s^*)} \sum_{w \in \chi^{-1}(a), (s, w) \in E} f(s, w) = \sum_{w \in N_{out}(s)} f(s, w) = \text{val}(f).$$

□

Lemma A.12. *Let $G = (V, E, c, s, t)$ be an instance of Maximum Flow, and let f^* be a flow in G/χ_∞ . Then there is a flow f in G such that $\text{val}(f) \geq \text{val}(f^*)$.*

Proof. For $(a, b) \in E(G/\chi_\infty)$, we define $E_{a,b} := \{(v, w) \in E \mid \chi_\infty^G(v) = a, \chi_\infty^G(w) = b\}$. We define

$$f(v, w) := \frac{f^*(a, b)}{c^*(a, b)} \cdot c(v, w),$$

for all $(v, w) \in E_{a,b}$. Clearly, $f(v, w) \leq c(v, w)$ for all $(v, w) \in E_{a,b}$, because $f^*(a, b) \leq c^*(a, b)$ due to f^* being a flow.

Now let $v \in \chi^{-1}(a)$ where a is an arbitrary color in the image of χ_∞^G . We claim that

$$\sum_{u \in N_{in}(v)} f(u, v) = \frac{1}{|\chi^{-1}(a)|} \sum_{b \in N_{in}(a)} f^*(b, a). \quad (4)$$

Indeed, we have that

$$\sum_{u \in N_{in}(v)} f(u, v) = \sum_{u' \in N_{in}(v')} f(u', v')$$

for all $v, v' \in \chi^{-1}(a)$ by Lemma A.10. It follows that

$$\begin{aligned}
|\chi^{-1}(a)| \cdot \sum_{u \in N_{in}(v)} f(u, v) &= \sum_{v' \in \chi^{-1}(a)} \sum_{u' \in N_{in}(v')} f(u', v') \\
&= \sum_{v' \in \chi^{-1}(a)} \sum_{b \in N_{in}(a)} \sum_{u' \in N_{in}(v'), \chi(u')=b} f(u', v') \\
&= \sum_{v' \in \chi^{-1}(a)} \sum_{b \in N_{in}(a)} \sum_{u' \in N_{in}(v'), \chi(u')=b} \frac{f^*(b, a)}{c^*(b, a)} \cdot c(u', v') \\
&= \sum_{b \in N_{in}(a)} \sum_{v' \in \chi^{-1}(a), u' \in \chi^{-1}(b), (u', v') \in E} \frac{f^*(b, a)}{c^*(b, a)} \cdot c(u', v') \\
&= \sum_{b \in N_{in}(a)} f^*(b, a).
\end{aligned}$$

Similarly, we obtain that

$$\sum_{w \in N_{out}(v)} f(v, w) = \frac{1}{|\chi^{-1}(a)|} \sum_{b \in N_{out}(a)} f^*(a, b). \quad (5)$$

Together, we conclude that

$$\sum_{u \in N_{in}(v)} f(u, v) = \frac{1}{|\chi^{-1}(a)|} \sum_{b \in N_{in}(a)} f^*(b, a) = \frac{1}{|\chi^{-1}(a)|} \sum_{b \in N_{out}(c)} f^*(a, b) = \sum_{w \in N_{out}(v)} f(v, w)$$

where the second equality holds since f^* is a flow. Also, we obtain that

$$\text{val}(f) = \sum_{w \in N_{out}(s)} f(s, w) = \sum_{b \in N_{out}(\chi(s))} f^*(\chi(s), b) = \text{val}(f^*)$$

since the color class containing s has size one. $\square$

Corollary A.13. *Let $G = (V, E, c: E \rightarrow \mathbb{R}, s, t)$ and $G' = (V', E', c': E' \rightarrow \mathbb{R}, s', t')$ be two instances of Maximum Flow. If $G \equiv_{1\text{-WL}} G'$, then G and G' have the same maximum flow value.*

Proof. If $G \equiv_{1\text{-WL}} G'$, then we obtain that $G/\chi_\infty = G'/\chi_\infty$. Now, the statement follows from Lemmas A.11 and A.12. $\square$

Proof of Theorem A.9. The first part follows directly from Theorem 8.4 and Corollary A.13. For the second part, observe that Lemmas A.11 and A.12 provide an explicit method for obtaining a flow on H given the flow for a prior graph G_i such that $G_i \equiv_{1\text{-WL}} H$. $\square$

B More Conditional Lower Bounds

B.1 Additional Problem Definitions

Definition B.1 (0/1-Integer Linear Programming). In the 0/1-Integer Linear Programming we are given an integer matrix C and an integer d and. We have to decide if there exist a 0/1-vector x with $Cx = d$.

Two problem instances (C, d) and (C', d') are isomorphic if there are permutation matrices P and Q with $PCQ = C'$ and $Pd = d'$.

Note that 0/1-Integer Linear Programming is closed under isomorphism since if x is a solution to $Cx = d$ if and only if $x' = Q^{-1}x$ is a solution to $C'x' = d'$. For the remaining problem checking that they are closed under isomorphism is easy to see.

Definition B.2 (k -Center). In the k -Center problem we are given a simple graph $G = (V, E, w)$ with edge weights $w: E \rightarrow \mathbb{N}$, a number $k \in \mathbb{N}$, and a value $c \in \mathbb{N}$. We have to decide if there is a set $X \subseteq V$ of size at most k such that every $v \in V$ has distance at most c to the closest point in X .

Two problem instances $(G = (V, E, w), k, c)$ and $(G' = (V', E', w'), k', c')$ are isomorphic if $k = k'$, $c = c'$ and G is isomorphic to G' .

Definition B.3 (Clique). In the Clique problem we are given a simple graph G and a value k . We have to decide if G contains k -clique (i.e., a subgraph with k vertices such that each vertex is connected to each other vertex)

Two problem instances (G, k) and (G', k') are isomorphic if G and G' are isomorphic as graphs and $k = k'$.

Definition B.4 (Clique Cover). In the Clique Cover problem we are given a simple graph G and a value k . We have to decide if G contains clique cover of size at most k (i.e., there is a collection of at most k many cliques that are all subgraphs of G and each edge of G is covered by at least one clique)

Two problem instances (G, k) and (G', k') are isomorphic if G and G' are isomorphic as graphs and $k = k'$.

Definition B.5 (Coloring). In the Coloring problem we are given a simple graph G and a value k . We have to decide if G is k -colorable.

Two problem instances (G, k) and (G', k') are isomorphic if G and G' are isomorphic as graphs and $k = k'$.

Definition B.6 (k -Coloring). In the k -Coloring problem we are given a simple graph G . We have to decide if G is k -colorable.

Two problem instances G and G' are isomorphic if G and G' are isomorphic as graphs.

Definition B.7 (3D-Matching). In the 3D-Matching problem we are given a set $U \subseteq [n]^3$. We have to decide if there exist a set $W \subseteq U$ with $|W| = n$ and no two elements of W agree in any coordinate.

Two problem instances (U, n) and (U', n') are isomorphic if there exist a bijection $f: [n] \rightarrow [n']$ with $U' = \{(f(x), f(y), f(z)) \mid (x, y, z) \in [n]^3\}$.

Definition B.8 (Exact Cover). In the Exact Cover problem we are given a number n and a values of subsets $S_1, \dots, S_m \subseteq [n]$. We have to decide if there are indices $j_1, \dots, j_s$ such that $S_{j_1}, \dots, S_{j_s}$ are disjoint and $\bigcup_{t=1}^s S_{j_t} = \bigcup_{i=1}^m S_i$.

Two problem instances $(S_1, \dots, S_m, n)$ and $(S'_1, \dots, S'_{m'}, n')$ are isomorphic if $m = m'$, $n = n'$ and there are bijective functions $f: [n] \rightarrow [n']$ and $g: [m] \rightarrow [m']$ such that for each $i \in [m]$ we obtain $f(S_i) = \{f(s) \mid s \in S_i\} = S'_{g(i)}$

Definition B.9 (Feedback Arc Set). In the Feedback Arc Set problem we are given a directed graph H and a number k . We have to decide if there exist a set of edges $S \subseteq E(G)$ with $|S| \leq k$ such that every cycle in G contains at least on edge in S .

Two problem instances (H, k) and (H', k') are isomorphic if $k = k'$ and $H \cong H'$ are isomorphic as graphs.

Definition B.10 (Feedback Vertex Set). In the Feedback Vertex Set problem we are given a directed graph H and a number k . We have to decide if there exist a set of vertices $R \subseteq V(G)$ with $|R| \leq k$ such that every cycle in G contains at least on vertex in R .

Two problem instances (H, k) and (H', k') are isomorphic if $k = k'$ and $H \cong H'$ are isomorphic as graphs.

Definition B.11 (Directed Hamiltonian Cycle). In the Directed Hamiltonian Cycle problem we are given a directed unweighted graph G . We have to decide if G contains a directed cycle which includes each node exactly once.

Two problem instances G and G' are isomorphic if G and G' are isomorphic as graphs.

Definition B.12 (Undirected Hamiltonian Cycle). In the Undirected Hamiltonian Cycle problem we are given a undirected unweighted graph G . We have to decide if G contains a directed cycle which includes each node exactly once.

Two problem instances G and G' are isomorphic if G and G' are isomorphic as graphs.

Definition B.13 (Hitting Set). In the Hitting Set problem we are given a number n and a values of subsets $S_1, \dots, S_k \subseteq [n]$. We have to decide if there is a set $W \subseteq [n]$ such that $|W \cap S_i| = 1$.

Two problem instances $(S_1, \dots, S_k, n)$ and $(S'_1, \dots, S'_{k'}, n')$ are isomorphic if $k = k'$, $n = n'$ and there are bijective functions $f: [n] \rightarrow [n']$ and $g: [k] \rightarrow [k']$ such that for each $i \in [k]$ we obtain $f(S_i) = \{f(s) \mid s \in S_i\} = S'_{g(i)}$

Definition B.14 (Independent Set (IS)). In the Clique problem we are given a simple graph G and a value k . We have to decide if G contains an independent set of size k (i.e., a subgraph with k vertices such that no vertex is connected to any other vertex)

Two problem instances (G, k) and (G', k') are isomorphic if G and G' are isomorphic as graphs and $k = k'$.

Definition B.15 (MaxCut). In the MaxCut problem we are given a simple graph G and a value k . We have to decide if there exist a subset of vertices $S \subseteq V(G)$ such that there are at least k many edges between S and $V(G) \setminus S$.

Two problem instances (G, k) and (G', k') are isomorphic if G and G' are isomorphic as graphs and $k = k'$.

Definition B.16 (k -Median). In the k -Median problem we are given a set of facilities $F = \{f_1, \dots, f_t\}$, a set of clients $C = \{c_1, \dots, c_s\}$, a function $d: F \times C \rightarrow \mathbb{R}_{\geq 0}$, that satisfies the triangle inequality, a number k and a number c . We have to decide if there exist a set $F' \subseteq F$ of size k such that

$$\sum_{i=1}^s \min_{f \in F'} d(f, c_i) \leq c.$$

Two problem instances (F, C, d, k, c) and (F', C', d', k', c') are isomorphic if $c = c'$, $k = k'$, and there are bijections $f: F \rightarrow F'$ and $g: C \rightarrow C'$ such that $d(x, y) = d'(f(x), g(y))$.

Definition B.17 (Satisfiability (SAT)). In the Satisfiability problem we are given a formula Φ in conjunctive normal form (CNF). We have to decide if Φ is decidable.

Two CNF formulas Φ, Φ' are isomorphic if Φ' can be obtained from Φ by a renaming of variables. Note that this renaming is consistent with negation (i.e., if we rename x to y then $\neg x$ is mapped to $\neg y$).

Definition B.18 (3-Satisfiability (3-SAT)). In the 3-Satisfiability problem we are given a formula Φ in conjunctive normal form (CNF) such that each clause contains exactly three literals. We have to decide if Φ is decidable.

Two CNF formulas Φ, Φ' are isomorphic if Φ' can be obtained from Φ by a renaming of variables. Note that this renaming is consistent with negation (i.e., if we rename x to y then $\neg x$ is mapped to $\neg y$).

Definition B.19 (Max k -SAT). In the Max k -SAT we are given a formula Φ in conjunctive normal form (CNF) such that each clause contains exactly k literals. Given an assignment x of variables we write $|\Phi(x)|$ for the number of clauses that are satisfied by x . We have to return $\max_x |\Phi(x)|$.

Two CNF formulas Φ, Φ' are isomorphic if Φ' can be obtained from Φ by a renaming of variables. Note that this renaming is consistent with negation (i.e., if we rename x to y then $\neg x$ is mapped to $\neg y$).

Definition B.20 (Set Cover). In the Set Cover problem we are given a number n , a list of subsets $S_1, \dots, S_m \subseteq [n]$, and a number k . We have to decide if there are indices $j_1, \dots, j_s$ such that $s \leq k$ and $\bigcup_{t=1}^s S_{j_t} = \bigcup_{i=1}^m S_i$.

Two problem instances $(S_1, \dots, S_m, n, k)$ and $(S'_1, \dots, S'_{m'}, n', k')$ are isomorphic if $m = m'$, $k = k'$, $n = n'$ and there are bijective functions $f: [n] \rightarrow [n']$ and $g: [k] \rightarrow [k']$ such that for each $i \in [m]$ we obtain $f(S_i) = \{f(s) \mid s \in S_i\} = S'_{g(i)}$.

Definition B.21 (Set Packing). In the Set Packing problem we are given a family of sets $\mathcal{S} = \{S_1, \dots, S_m\}$ with $S_i \subseteq [n]$ and a number k . We have to decide if there exist k many mutually disjoint sets in $\mathcal{S}$.

Two problem instances $(\mathcal{S} = (S_1, \dots, S_m), n, k)$ and $(\mathcal{S}' = (S'_1, \dots, S'_m), n', k')$ are isomorphic if $m = m'$, $k = k'$, $n = n'$ and there are bijective functions $f: [n] \rightarrow [n']$ and $g: [k] \rightarrow [k']$ such that for each $i \in [m]$ we obtain $f(S_i) = \{f(s) \mid s \in S_i\} = S'_{g(i)}$.

Definition B.22 (Steiner Tree). In the Steiner Tree problem we are given a weighted undirected graph G , set of vertices R , and a value k . We have to decide if G contains a subtree that contains each vertex in R and has cost (i.e. sum of edges) at most k .

Two problem instances (G, R, k) and (G', R', k') are isomorphic if $k = k'$ and there is an isomorphism $\varphi: V(G) \rightarrow V(G')$ from G to G' that respects the edge weights and φ maps R to R' .

Definition B.23 (Traveling Salesman Problem (TSP)). In the Traveling Salesman Problem we are given a weighted clique with n vertices where each edge-weight is non-negative. We have to return the length of the humilation cycle with the smallest length.

Two problem instances G and G' are isomorphic if the underlying graphs are isomorphic.

Definition B.24 (Metric Traveling Salesman Problem (Metric TSP)).

In the Metric Traveling Salesman Problem we are given a weighted clique with n vertices where each edge-weight is non-negative. Further the edge-weight satisfy the triangle inequality. We have to return the length of the humilation cycle with the smallest length.

Two problem instances G and G' are isomorphic if the underlying graphs are isomorphic.

Definition B.25 (Vertex-Cover (VC)). In the Vertex-Cover problem we are given a simple graph G and a value k . We have to decide if G contains a vertex cover of size at most k .

Two problem instances (G, k) and (G', k') are isomorphic if G and G' are isomorphic as graphs and $k = k'$.

B.2 Additional Isomorphism-Preserving Reductions

In the following we show many different isomorphism-preserving reductions. However, many of them are simple modifications of already known reductions.

Lemma B.26. *All of the following reductions do hold:*

- $\text{SAT} \leq_p^{\cong} \text{Coloring}$
- $\text{VC} \leq_p^{\cong} \text{Directed Hamiltonian Cycle}$
- $\text{VC} \leq_p^{\cong} \text{Set Cover}$
- $\text{VC} \leq_p^{\cong} \text{Feedback Vertex Set}$
- $\text{VC} \leq_p^{\cong} \text{Feedback Arc Set}$
- $\text{SAT} \leq_p^{\cong} \text{0/1-Integer Linear Programming}$
- $\text{SAT} \leq_p^{\cong} \text{Clique}$
- $\text{Coloring} \leq_p^{\cong} \text{Exact Cover}$
- $\text{Exact Cover} \leq_p^{\cong} \text{Steiner Tree}$
- $\text{Exact Cover} \leq_p^{\cong} \text{Hitting Set}$
- $\text{Exact Cover} \leq_p^{\cong} \text{3D-Matching}$
- $\text{Coloring} \leq_p^{\cong} \text{Clique Cover}$
- $\text{Clique} \leq_p^{\cong} \text{Set Packing}$

and $\text{Directed Hamiltonian Cycle} \leq_p^{\cong} \text{Undirected Hamiltonian Cycle}$.

Proof. All of these reductions can be obtained by using the corresponding reduction in [Kar72] and checking that the reduction is isomorphism-preserving. $\square$

Lemma B.27. $\text{IS} \leq_p^{\cong} \text{VC}$

Proof. A folklore result states that S is an independent set of a graph G if and only if $\bar{S} := V(G) \setminus S$ is a vertex cover of G . Thus, a function R that maps (G, k) to $(G, |V(G)| - k)$ is an isomorphism-preserving reduction from IS to VC. $\square$

Lemma B.28. $\text{MaxCut} \leq_p^\cong \text{Max 2-SAT}$

Proof. Let $(G = (V, E), k)$ be an instance of **MaxCut**. Let R be the function that maps (G, k) to Φ where $\{x_v \mid v \in V\}$ are the variables of $(\Phi, |E| + k)$ and $\bigcup_{\{v,u\} \in E} \{(x_v \vee x_u), (\neg x_v \vee \neg x_u)\}$.

If there is a set $S \subseteq V$ that cuts at least k edges, then the assignment

$$x_v = \begin{cases} 1 & , \text{ if } v \in S \\ 0 & , \text{ if } v \notin S \end{cases}$$

satisfies both clauses for a cut edge and one clause for every other edge. This implies that $|E| + k$ clauses are satisfied for this assignment.

If there is an assignment x_v that satisfies $|E| + k$ clauses, then the set $S = \{v \in V \mid x_v = 1\}$ cuts at least k edges because there are k edges for which both constraints are satisfied. These edges are cut by S .

Let $(G_1 = (V_1, E_1), k_1)$ and $(G_2 = (V_2, E_2), k_2)$ be isomorphic instances of **MaxCut** with isomorphism $\phi: V \rightarrow V$. Then, the formulas in $R(G_1, |E_1| + k_1)$ and $R(G_2, |E_2| + k_2)$ are isomorphic to each other due to renaming the variables using ϕ . Also it holds that

$$|E_1| + k_1 = |E_1| + k_2 = |E_2| + k_2.$$

$\square$

Lemma B.29. $\text{Undirected Hamiltonian Cycle} \leq_p^\cong \text{Metric TSP}$

Proof. Let $G = (V, E)$ be an instance of **Undirected Hamiltonian Cycle**. Let R be the function that maps G to $(G' = (V, \binom{V}{2}), w, |V|)$ such that

$$w(e) = \begin{cases} 1 & , \text{ if } e \in E \\ 2 & , \text{ if } e \notin E \end{cases}$$

for all $e \in \binom{V}{2}$. Note that this weight functions satisfies the triangle inequality because it has only values from $\{1, 2\}$.

If G has a Hamiltonian Cycle, then it has G' has a Hamiltonian Cycle of cost at most $|V|$.

If G has no Hamiltonian Cycle, then every Hamiltonian Cycle in G' uses an edge $e \notin E$ and thus is of cost $w(e) = 2$. Hence, every Hamiltonian Cycle in G' has cost at least $|V| + 1$.

Let G_1 and G_2 be isomorphic instances of **Undirected Hamiltonian Cycle** with isomorphism $\phi: V \rightarrow V$. Then, ϕ is also an isomorphism for $R(G_1)$ and $R(G_2)$. $\square$

This Lemma uses the reduction from [GK99].

Lemma B.30. $\text{Set Cover} \leq_p^\cong k\text{-Median}$.

Proof. Let $(S_1, \dots, S_m, n, k)$ be an instance of **Set Cover**. Let R be the function that maps $(S_1, \dots, S_m, n, k)$ to $([m], [n], d, k, n)$ and

$$d(i, j) = \begin{cases} 1 & , \text{ if } j \in S_i \\ 3 & , \text{ otherwise} \end{cases}$$

for all $i \in [m]$ and $j \in [n]$. In [GK99], they show that $R(S_1, \dots, S_m, n, k)$ is a YES-Instance of k -Median if and only if $(S_1, \dots, S_m, n, k)$ is a YES-Instance of Set Cover.

It remains to show that the reduction R is isomorphism-preserving. To this end, we start by considering two isomorphic instances $(S_1, \dots, S_m, n, k)$ and $(S'_1, \dots, S'_{m'}, n', k')$. By definition, there are two bijective functions $f: [n] \rightarrow [n']$ and $g: [m] \rightarrow [m']$ with $S'_{g(i)} = f(S_i)$. Then $R(S_1, \dots, S_m, n, k) = ([m], [n], d, k, n)$ and $R(S'_1, \dots, S'_{m'}, n', k') = ([m'], [n'], d', k', n')$ are isomorphic due to the same bijective functions f and g because for all $i \in [m]$ and $j \in [n]$ it holds that,

$$d(i, j) = 1 \Leftrightarrow j \in S_i \Leftrightarrow f(j) \in S'_{g(i)} \Leftrightarrow d(f(j), g(i)) = 1.$$

□

Lemma B.31. $\text{VC} \leq_p^{\cong} \text{Gap-k-Center}_{1,2}$

Proof. Let $((V, E), k)$ be an instance of VC. Let R be the function that maps $((V, E), k)$ to $((V, E, w), k)$ such that $w(e) = 1$ for all $e \in E$.

If there is a vertex cover X of size k in V , then it holds that

$$\max_{v \in V} d_{(V, E), w}(v, X) \leq 1.$$

If (V, E) does not contain a vertex cover of size k , then for all subsets $X \subseteq V$ of size k it holds that

$$\max_{v \in V} d_{(V, E), w}(v, X) \geq 2.$$

This is an isomorphism-preserving reduction because the graph is not changed when R is applied. □

Lemma B.32. $\text{Undirected Hamiltonian Cycle} \leq_p^{\cong} \text{Gap-TSP}_{1,c}$ For all $c > 1$.

Proof. Let $G = (V, E)$ be an instance of Undirected Hamiltonian Cycle. Let R be the function that maps G to $G' = (V, \binom{V}{2}, w)$ such that

$$w(e) = \begin{cases} 1/|V| & , \text{ if } e \in E \\ C & , \text{ if } e \notin E \end{cases}$$

for all $e \in \binom{V}{2}$.

If G has a Hamiltonian Cycle, then it has G' has a Hamiltonian Cycle of cost at most 1.

If G has no Hamiltonian Cycle, then every Hamiltonian Cycle in G' uses an edge $e \notin E$ and thus is of cost $w(e) = C$. Hence, every Hamiltonian Cycle in G' has cost at least C .

Let G_1 and G_2 be isomorphic instances of Undirected Hamiltonian Cycle with isomorphism $\phi: V \rightarrow V$. Then, ϕ is also an isomorphism for $R(G_1)$ and $R(G_2)$. □